



THE LAB CODER

Basic Level (Grove Hat)

Updated 13 Feb 2026

Content

Page	Unit	Topic
3	3002	<u>Sequence & Buzzer</u>
12	3003	<u>Concurrency & LCD</u>
22	3004	<u>Repeat Loops</u>
30	3005	<u>Conditional Loops & Button</u>
40	3006	<u>If Statement</u>
50	3007	<u>Comparison & AND Operators & Ultrasonic Sensor</u>
65	3008	<u>Comparison & OR Operators</u>
74	3009	<u>Basic 1 Project</u>
76	3010	<u>Logic & Motor</u>
84	3011	<u>Logic & Colour Sensor</u>
91	3012	<u>Nested If Statements</u>
99	3013 - 3015	<u>More Applications</u>
107	3016	<u>Basic 2 Project</u>
109		<u>Useful Structures</u>

3002 - Sequence & Buzzer

Learning

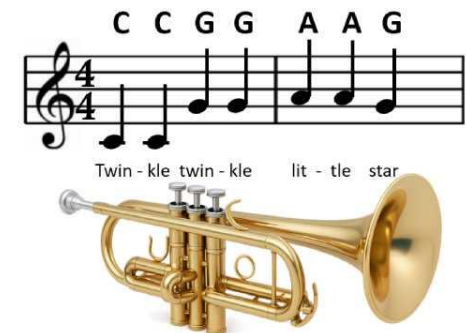
When solving problems and writing code, programmers begin by understanding the order in which actions occur. This is called **sequencing**.

In this unit, we will practice sequencing in **Google Blockly** using the **Buzzer module** of your robot.

In Blockly, the blocks run from **top to bottom**. You must form your blocks in the right sequence to get the desired results.

In class, we will explore the following:

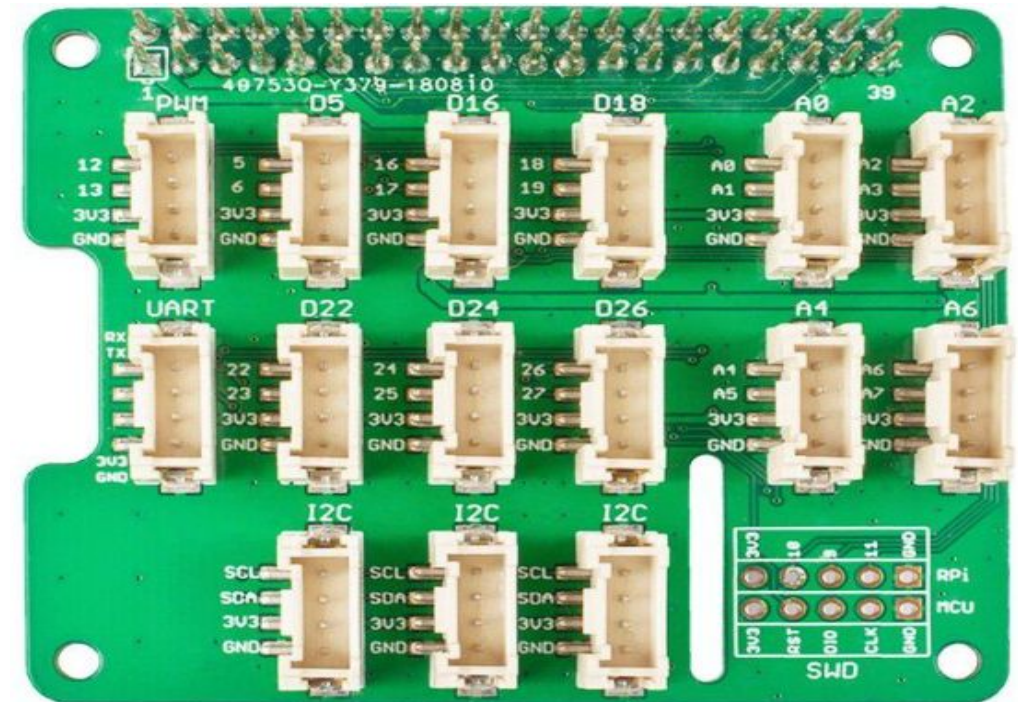
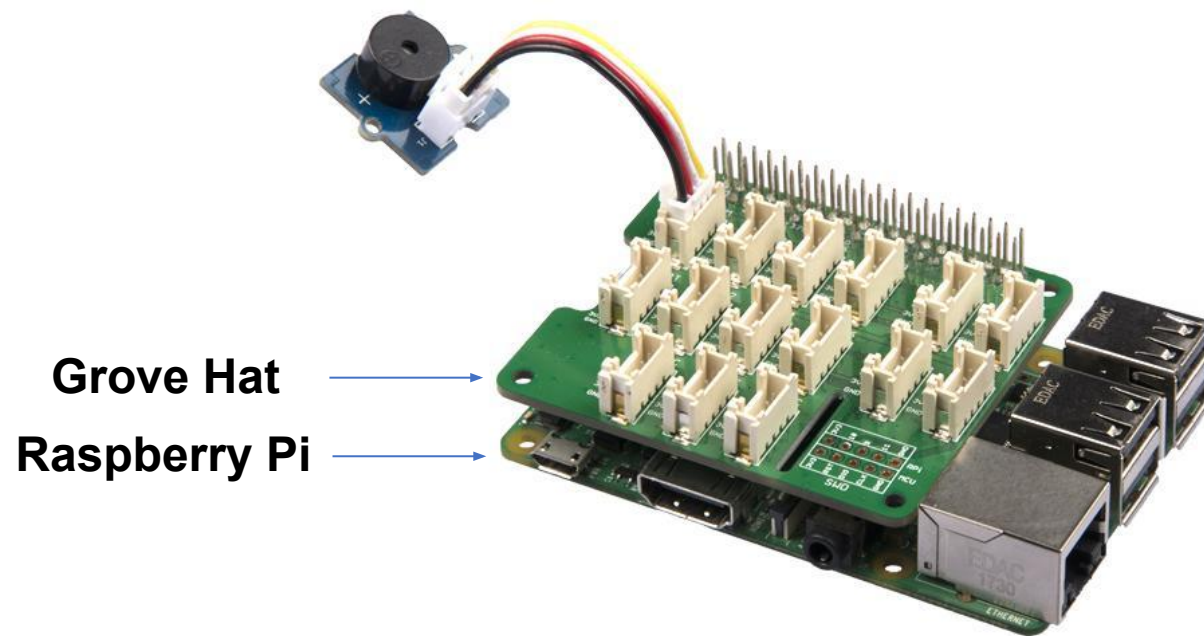
- Understand the Buzzer module and its blocks.
- Explore making sound sequencing using Note and Tone blocks.
- How to control the speed of beeping.
- Put together sound effects and play a piece of music.



3002 - Sequence & Buzzer

Robot

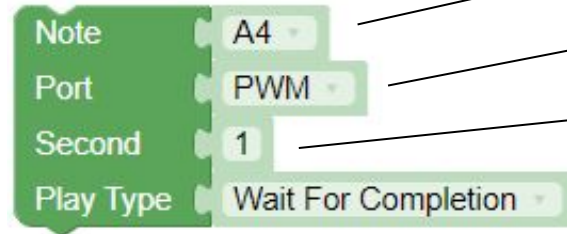
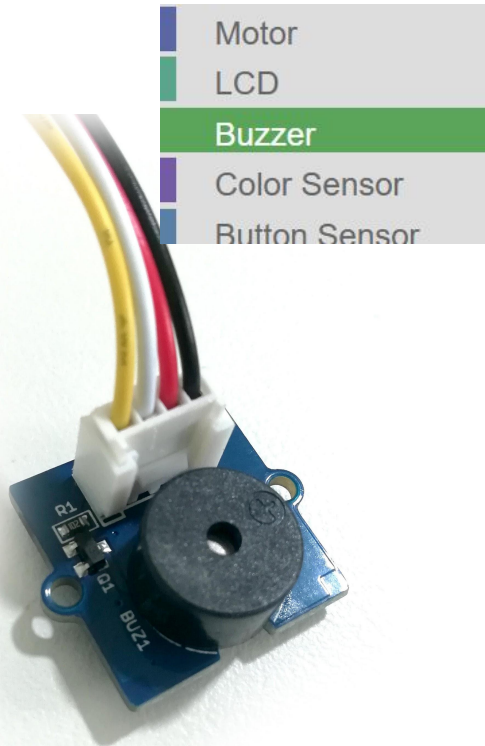
Raspberry Pi (the brain) + Grove Hat (ports for plugging sensors)



3002 - Sequence & Buzzer

Buzzer Sensor

For making a sound



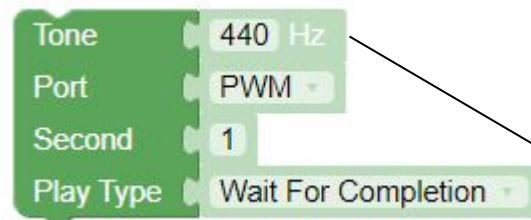
Notes to play (C4, D4, E4, F4, A4, B4). C4 is lowest.

Plug at which port

Duration of play

Set the Play Type:

- **"Wait For Completion"**
Wait until the sound ends then move to next block
- **"Play once"**
Move to next block while letting it play once. It will stop when the time is up.
- **"Loop"** (this option is not required)
Move to next block while keep looping the sound

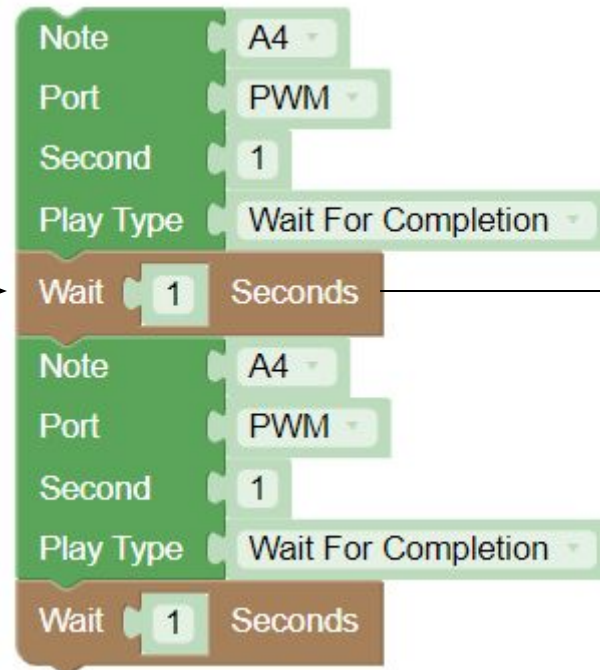
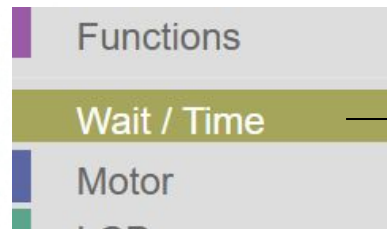
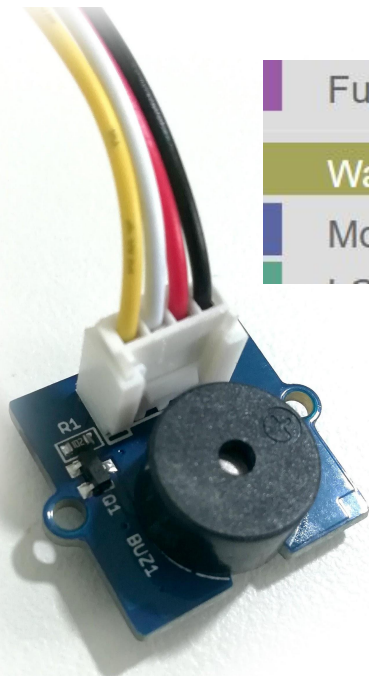


You can also use frequency instead of notes. The higher the sharper the tone.

3002 - Sequence & Buzzer

Buzzer Sequence

Making a **beeping** sound



To make a **BEEPING** sound, you must include a **wait block** between the sound blocks. Without it, it will give a continuous sound.

The **speed** of beeping is affected by the both durations of the sound and the wait.

Exercise Q1

Which of the following code snippets will make a **beeping sound**?

Diagram 1

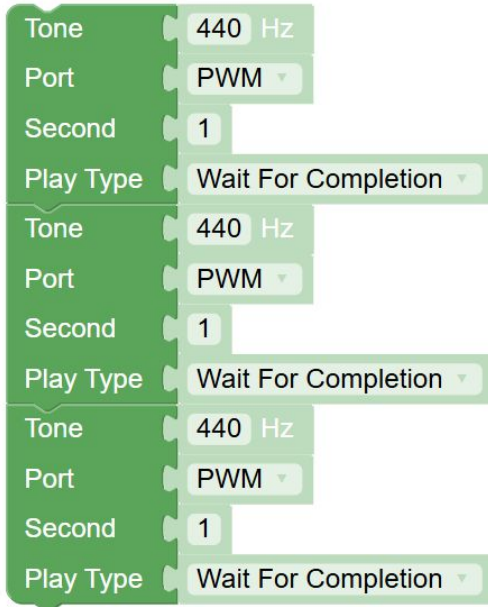
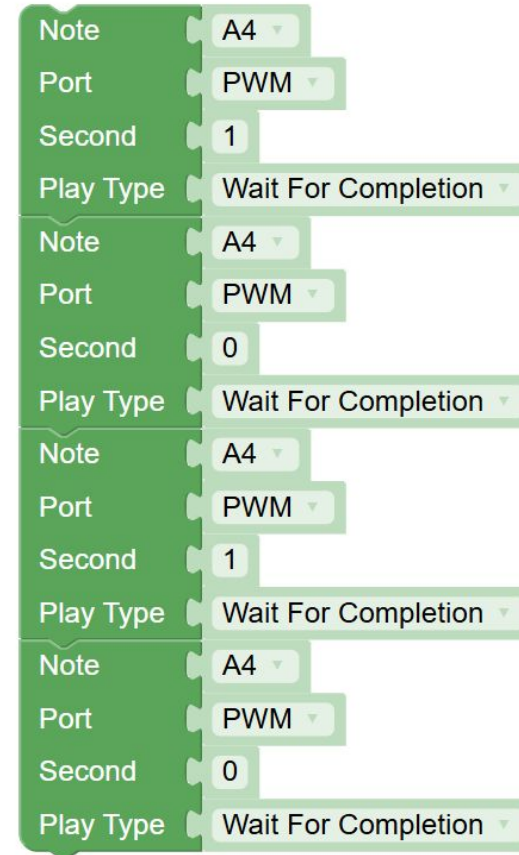


Diagram 2



Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) Diagrams 1, 2 and 3

Exercise Q2

Which of the following code snippets will make **one beep per second**?

Diagram 1

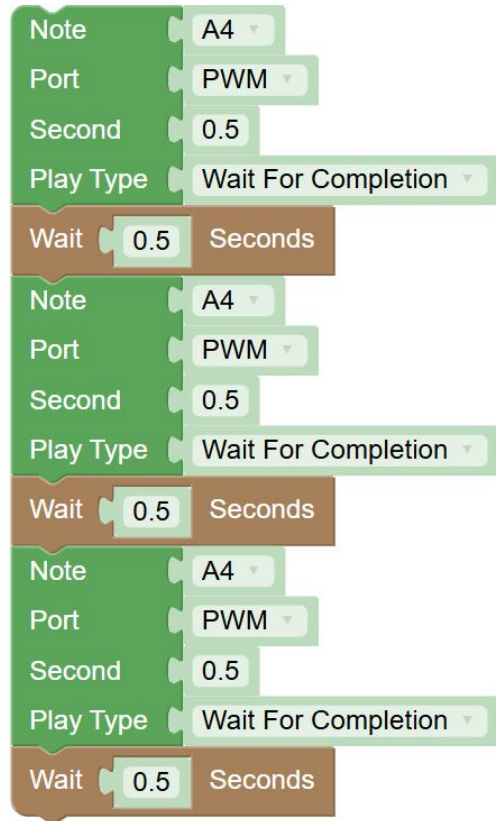


Diagram 2

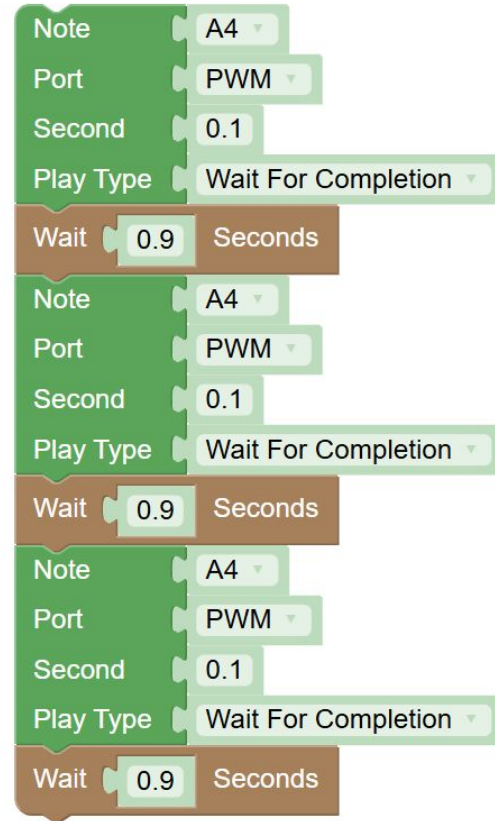
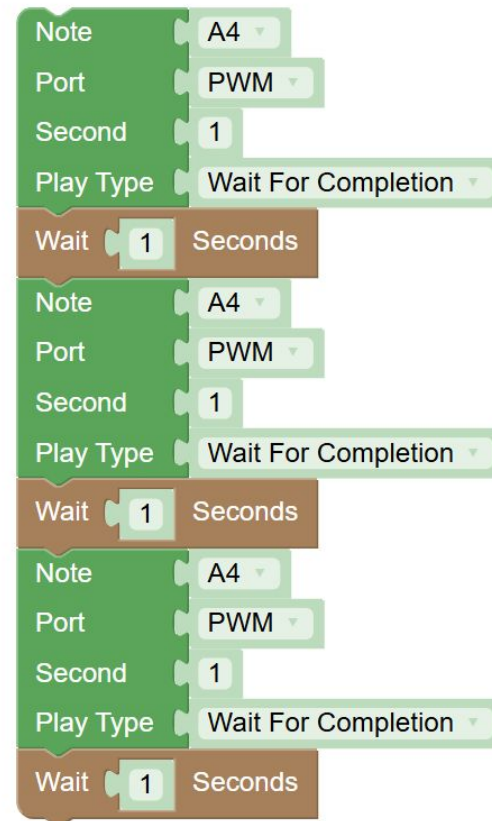


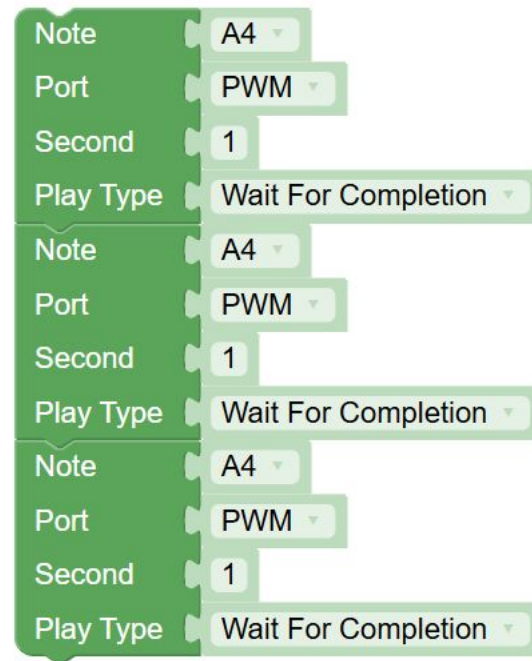
Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) Diagrams 1 and 2 only

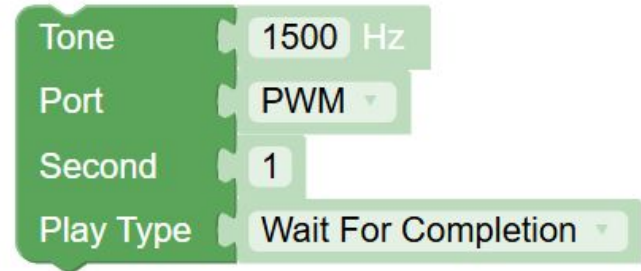
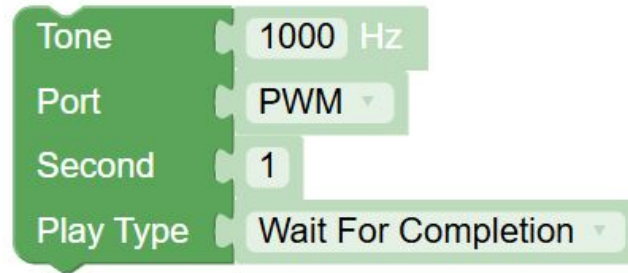
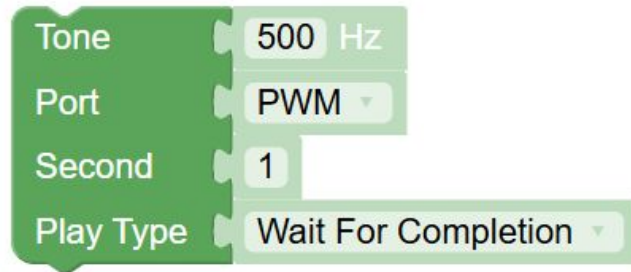
Exercise Q3

The code below is supposed to make a beeping sound but it sounds like a continuous sound. Add in suitable blocks to make it work.



Exercises Q4

Kenny wanted to make 3 different sounds in a row from high pitch to low pitch. He formed his code this way. Help Kenny to fix his code.



Answers

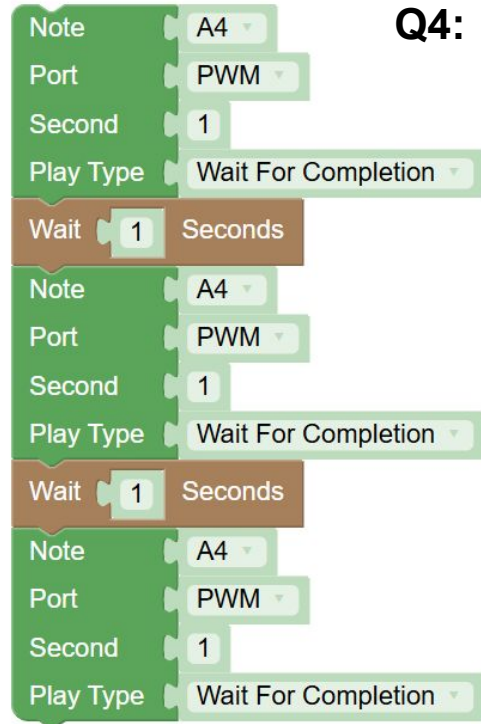
Q1: B

You need pause between sound blocks to beep. Otherwise, it will sound continuous.

Q2: D

The cycle time of a beep includes the wait duration as well. Diagram 3 is making one beep every 2 seconds.

Q3:



Q4:



3003 - Concurrency & LCD

Learning

Previously, you explored sequencing with the Buzzer module, learning about rhythm and sound effects.

In this unit, you'll dive into **concurrency**, making multiple actions happen together. We will learn about the LCD module and make sound and visual happen in order and concurrently for synchronised effects. We will also learn how to draw a simple flowchart.

In class, we will explore the following:

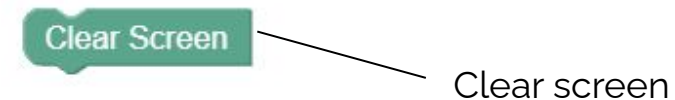
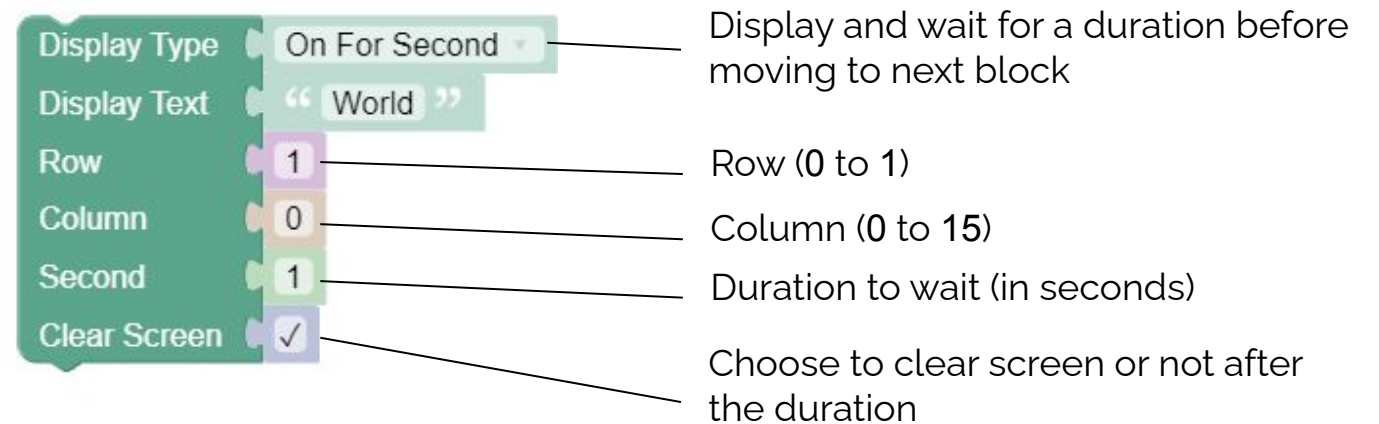
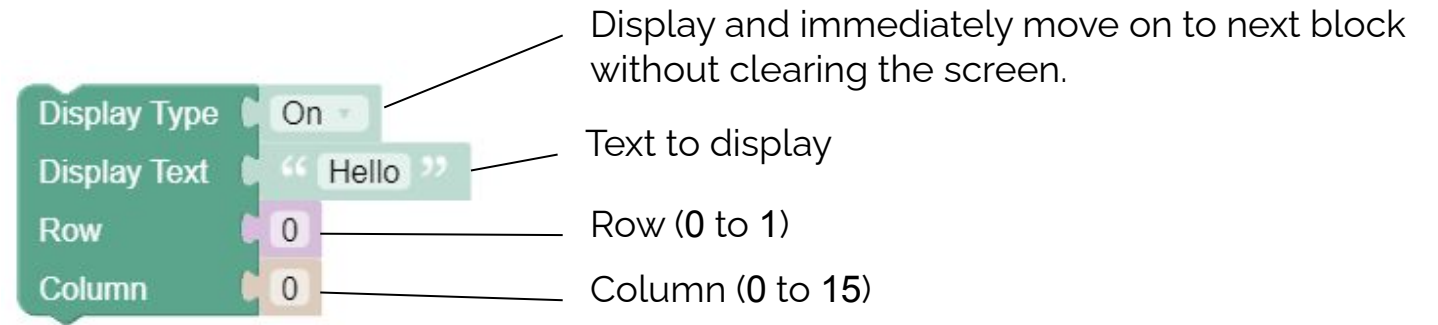
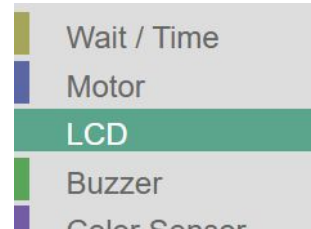
- Understand the LCD module and its blocks.
- Understand row and column and control the position of display content.
- Synchronise audio and visual output.
- Know what a flowchart is used for and practice drawing a simple one depicting sequence.
- Applications such as building codes for score board, counter and progress bar.



3003 - Concurrency & LCD

LCD

For displaying text



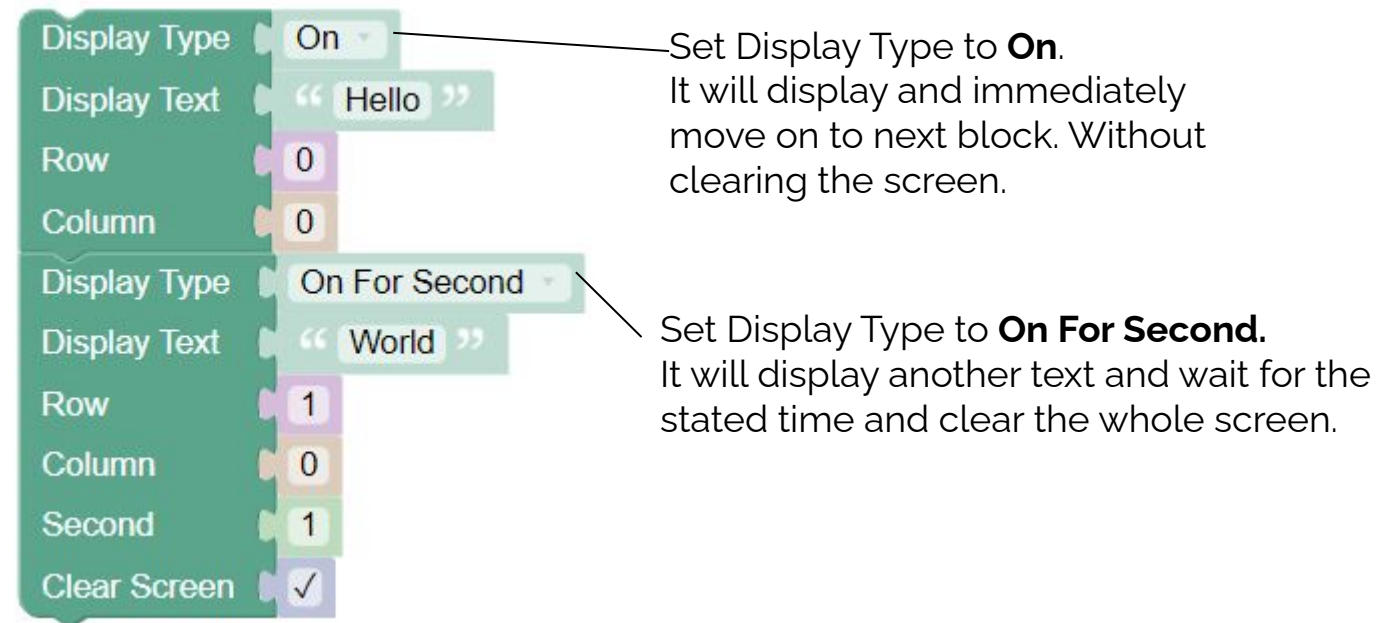
3003 - Concurrency & LCD

LCD

Displaying text **concurrently**



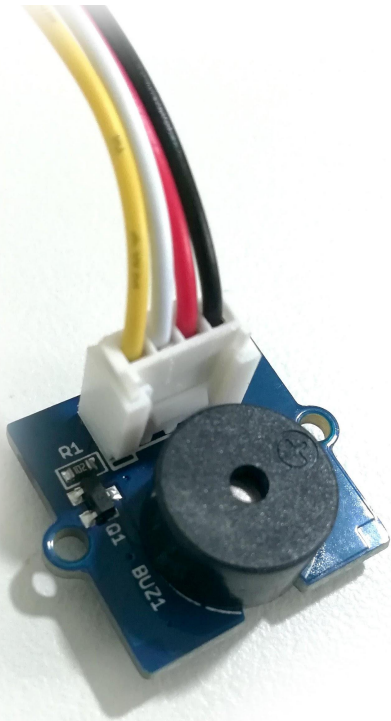
Putting the blocks in this manner will show the text concurrently (at the same time)



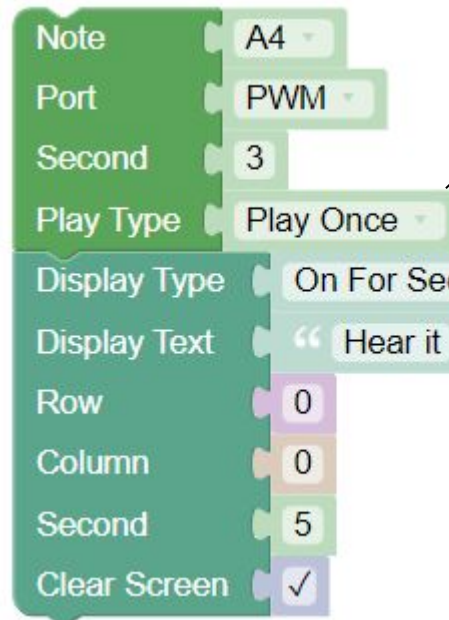
3003 - Concurrency & LCD

Buzzer Sensor

Play a sound and execute other blocks concurrently



You can set the sound to play for a duration while continue to perform the subsequent tasks.



Play Type: **Play Once**

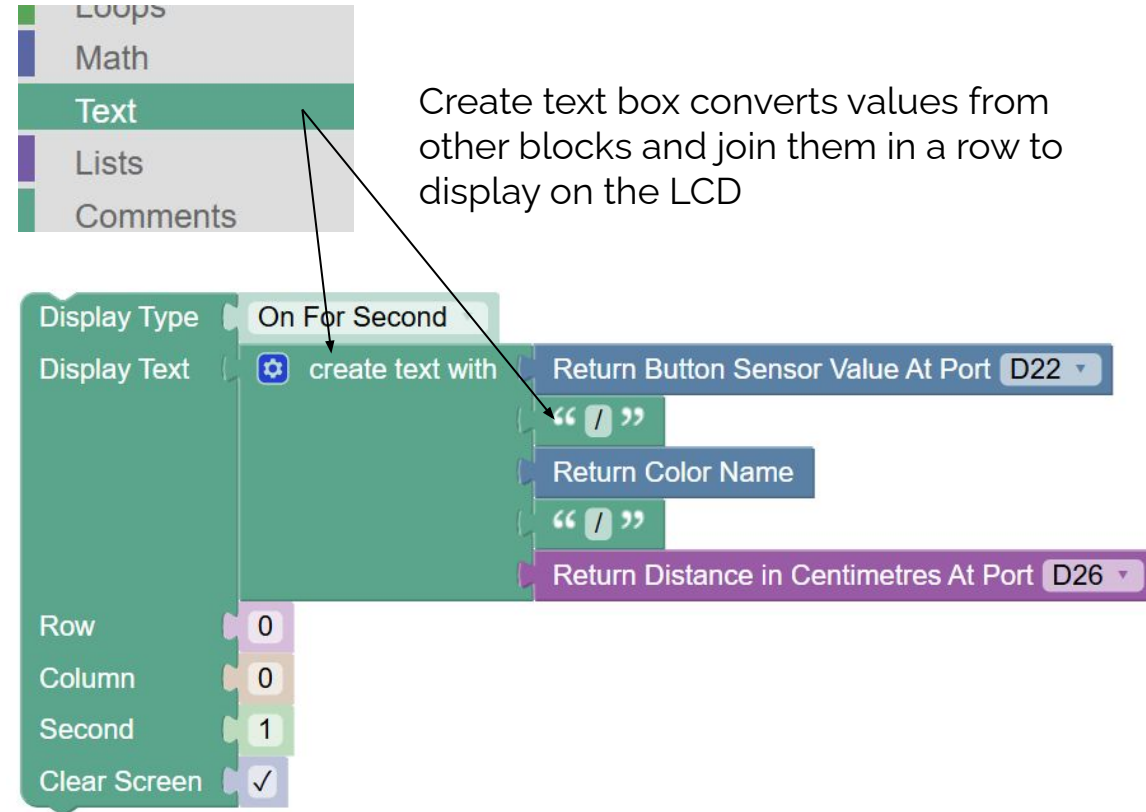
The sound will start playing and the next block will be executed immediately (it does not wait for 3 seconds)

The sound will continue playing at the background while the LCD display "Hear it". The sound will stop after 3 seconds (indicated in the sound block) while the LCD will continue to display for another 2 seconds (total 5 seconds as indicated in the LCD block).

3003 - Concurrency & LCD

LCD

For displaying values from other blocks in a row



Exercise Q1

Which of the following code snippets will display "**Hello World**" in **2 rows** as shown for **5 seconds**?

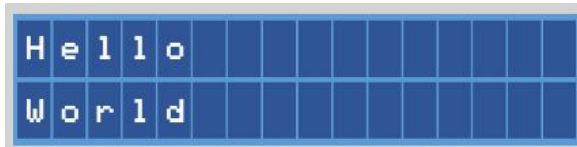


Diagram 1

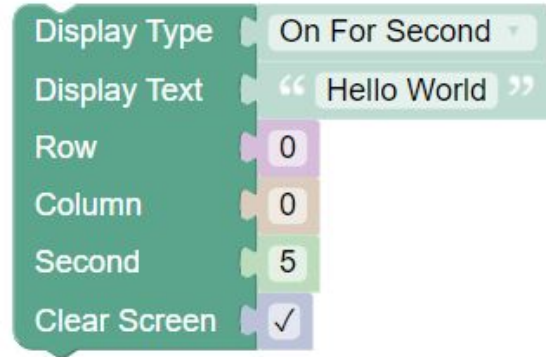


Diagram 2

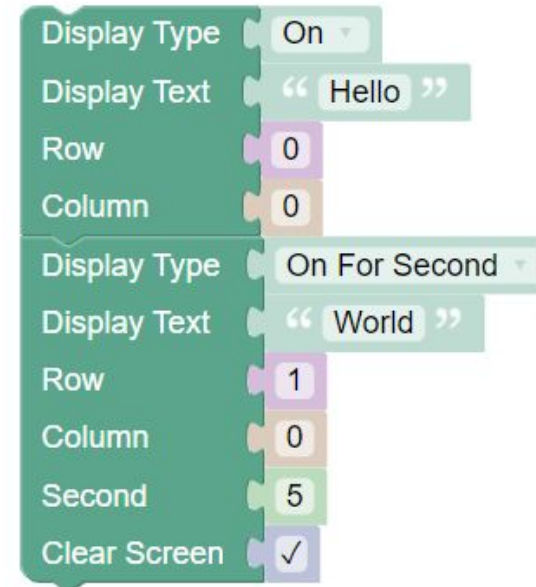
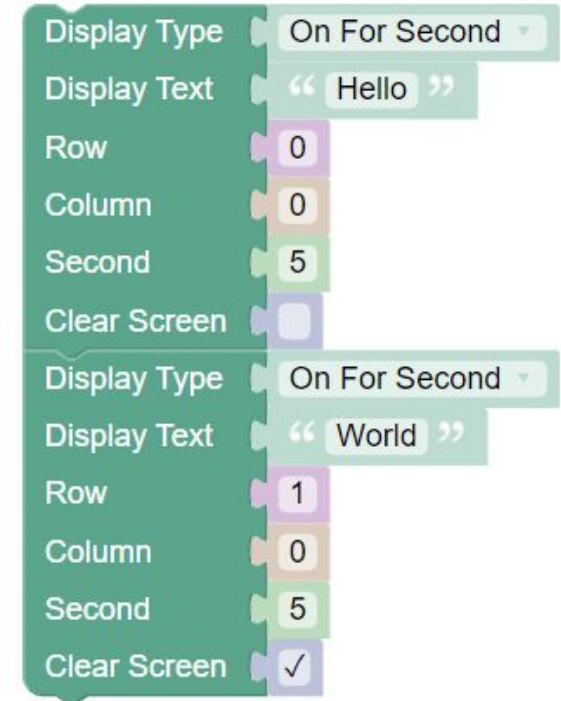


Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) Diagrams 2 and 3 only

Exercise Q2

Which of the following code snippets will display “**I have \$50**” in a **single row**?

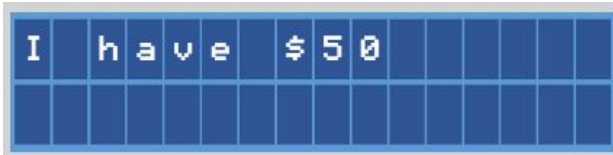
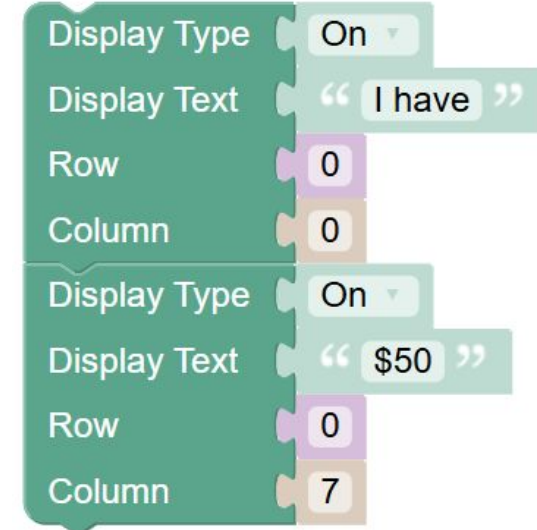


Diagram 1



Diagram 2



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 1 and 2
- D) None

Exercise Q2

Which of the following code snippets will show **"Beep"** and **make a sound at the same time**?

Diagram 1

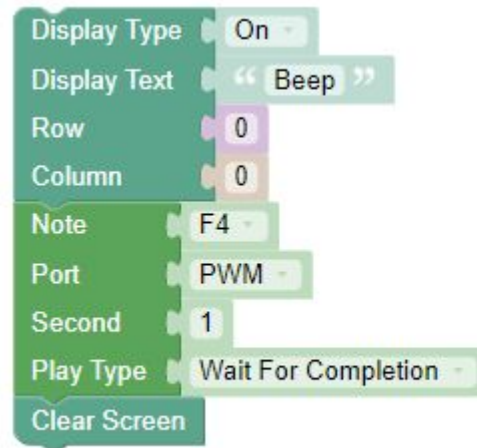


Diagram 2

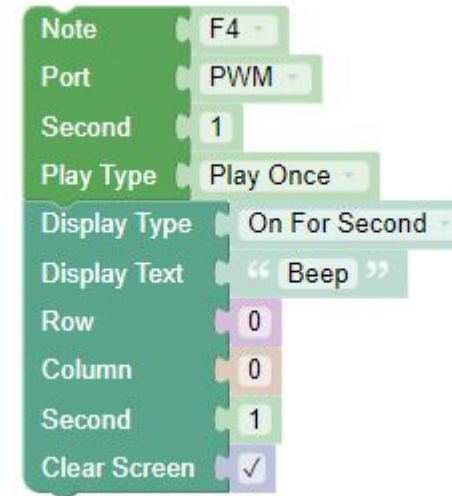
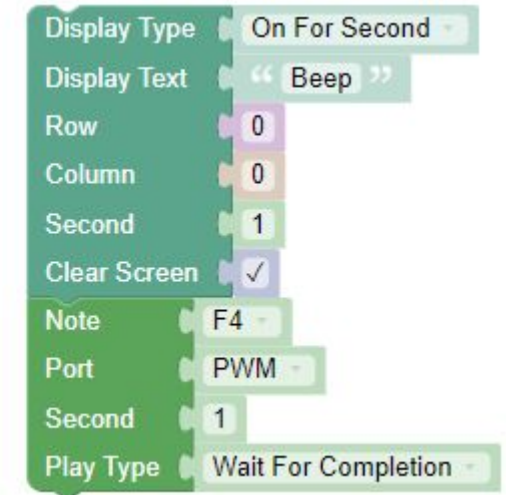


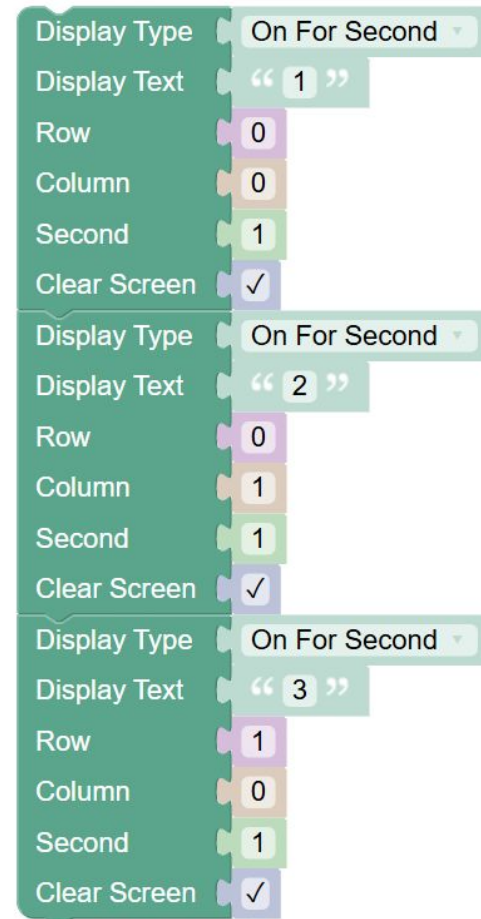
Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) Diagrams 1 and 2 only

Exercise Q4

The code below shows 1, 2 and 3 one by one. Adjust the setting in the blocks to make all the numbers appear together and stay for 4 seconds before disappearing.



Answers

Q1: B

Diagram 1 shows "Hello World" in a single row.
Diagram 3 will wait for 5 seconds before "World" appears.

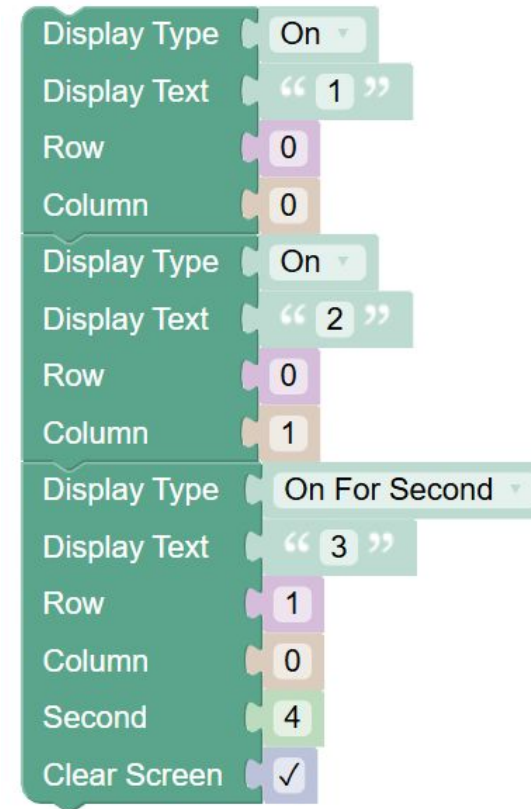
Q2: C

Create-text-with block will join values into a single row.

Q3: D

In Diagram 1, "Beep" will be shown and then the sound block is run immediately for 1 second. After 1 second, the screen will clear. In Diagram 2, the "Play Once" will enable the sound to be played and the next block to be run immediately and the sound will stop by itself after 1 second. In Diagram 3, it will show "Beep" for 1 second before playing the beep sound. So, they are not done at the same time.

Q4:



3004 - Repeat Loops

Learning

When solving a problem, you first observe the sequence, then identify **repeating patterns**. These patterns can then be repeated as a set of codes using **loop**.

In this unit, we will learn about using loop to repeat codes for a fixed number of times, as well as to repeat infinitely.

In class, we will explore the following:

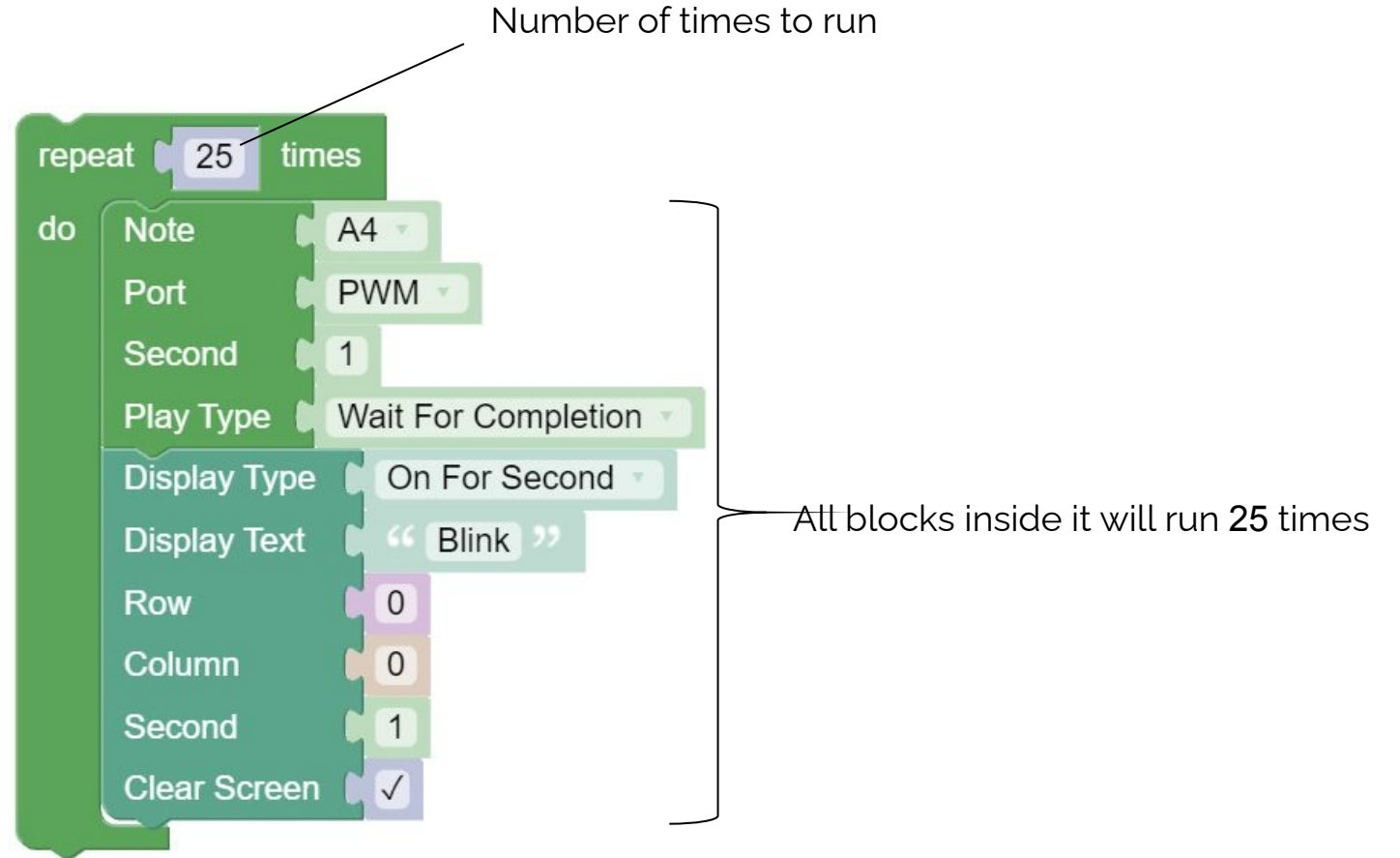
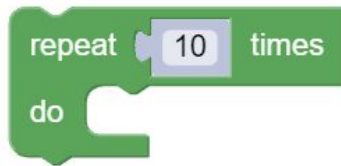
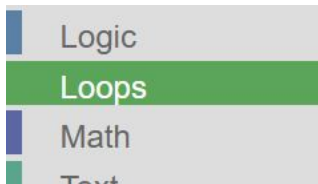
- Identifying patterns and using loops to simplify code.
- Fun applications involving emoticons and animations.
- Practice drawing a flowchart depicting loop.
- Practical applications such as Morse code transmission and a bomb-disarming program.



3004 - Repeat Loops

Repeat

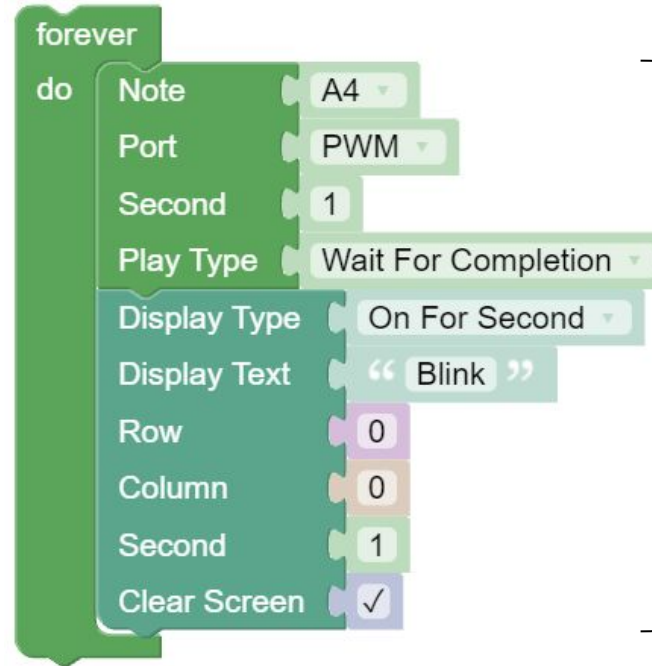
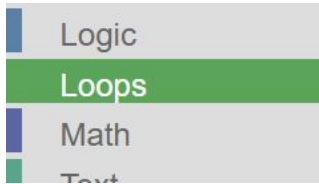
Repeat codes for a number times



3004 - Repeat Loops

Repeat *forever*

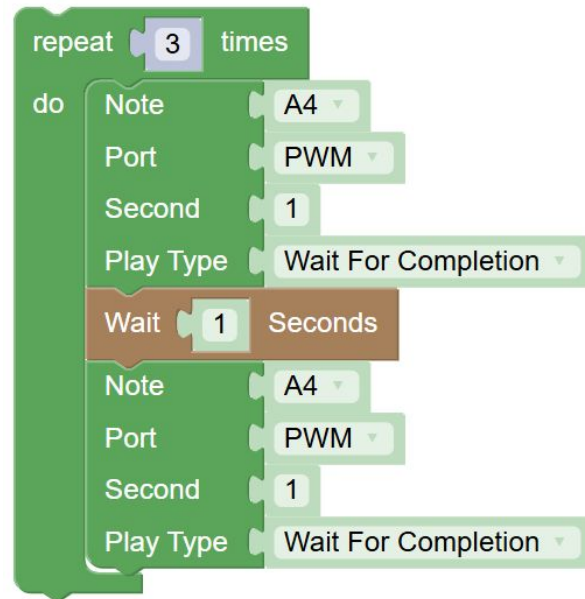
Repeat codes forever



All blocks inside it will repeat forever

Exercise Q1

How many beeps will you hear when the code is run?



- A) 3
- B) 4
- C) 5
- D) 6

Exercise Q2

Which of the following code snippets will show a **flashing "Beep"** and make a **beeping sound** at the **same time** (the word and sound appear and disappear together)?

Diagram 1

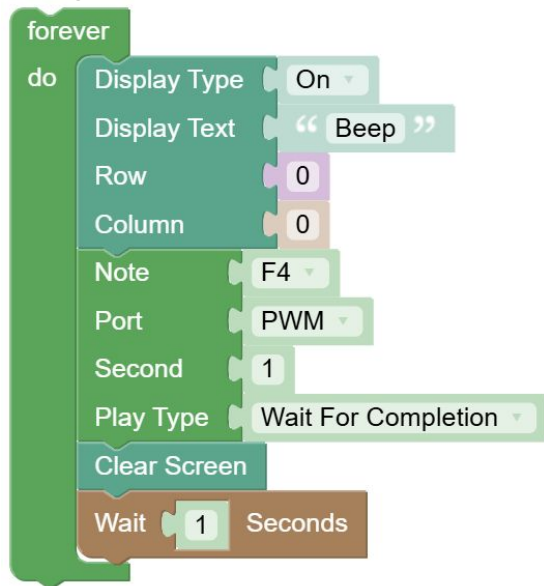


Diagram 2

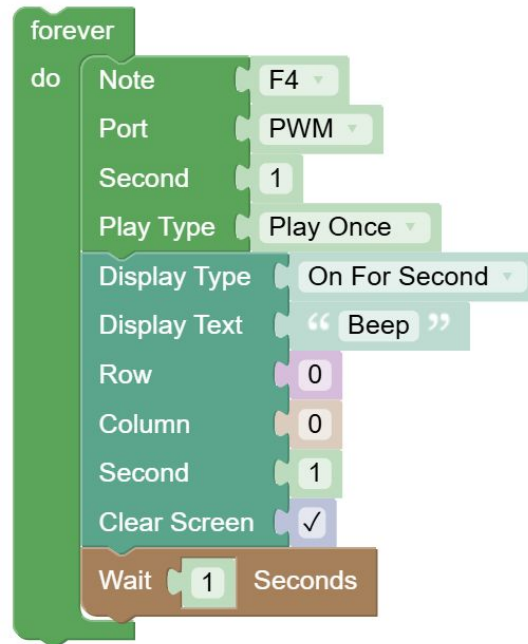
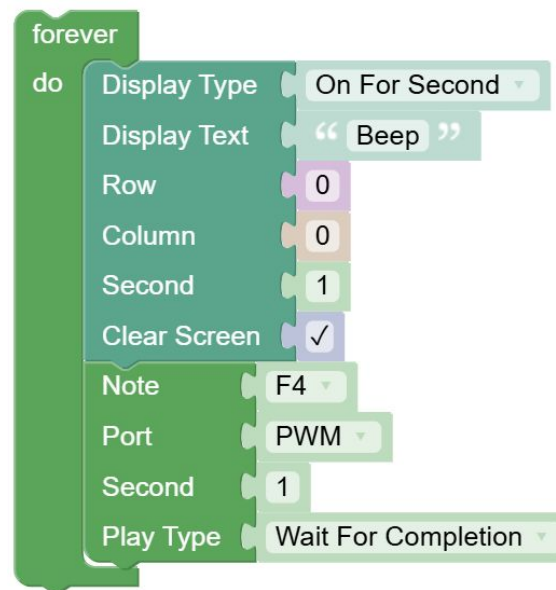


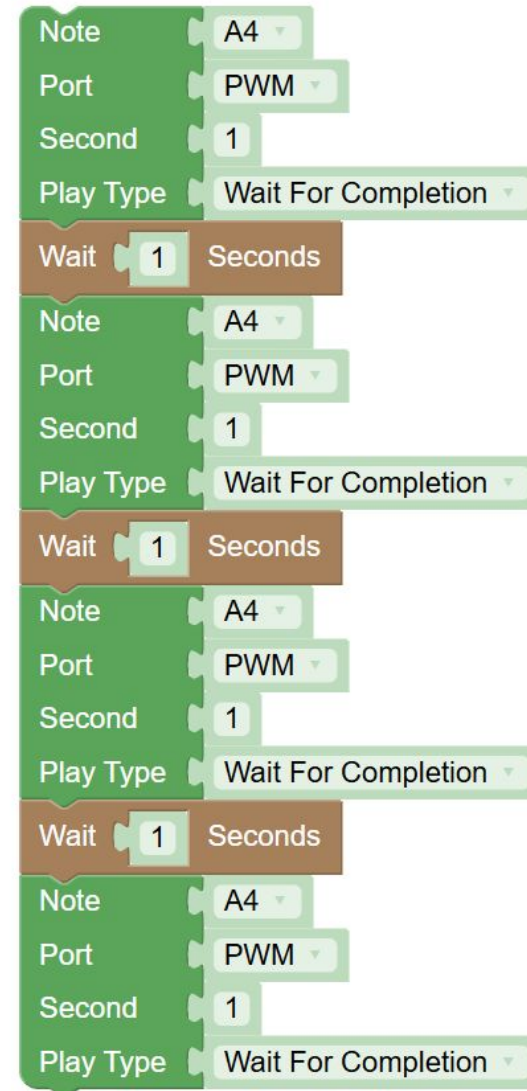
Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagrams 1 and 2 only
- D) Diagrams 1, 2 and 3

Exercise Q3

Simplify the following code using a loop.



Exercise Q4

Form a code that performs the following sequence:

1. Make 10 beeping sounds.
2. Display blinking text “Attention” for 10 times.

Answers

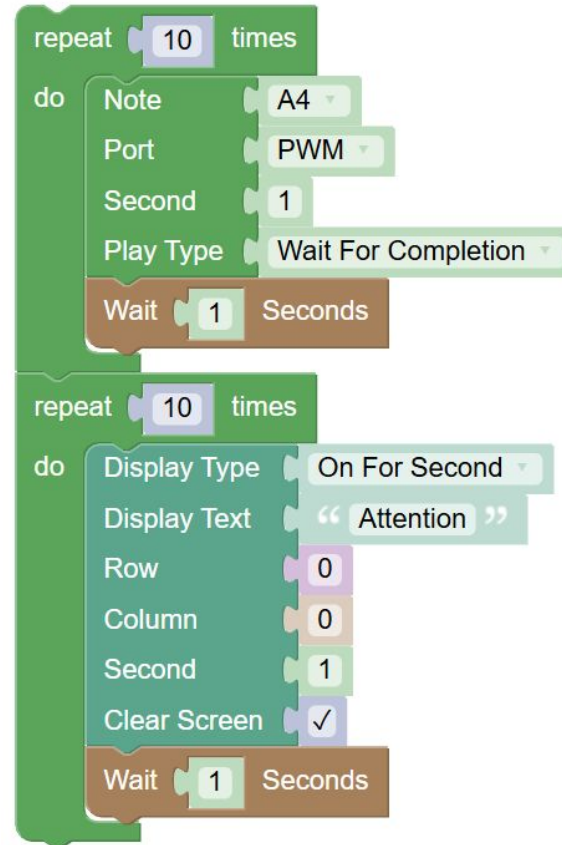
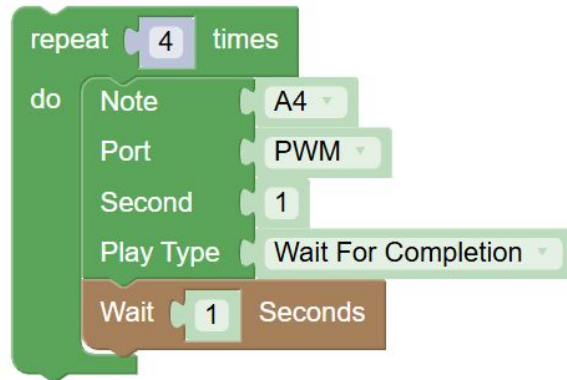
Q1: B

As it loops, the bottom sound block joins with the top one to form a continuous sound.

Q2: C

In diagram 3, the display and sound will take turn to appear instead of together.

Q3:



3005 - Conditional Loops & Button

Learning

A program handles logic by checking **conditions** to decide what it should do. Conditions are used in **Repeat-while/Repeat-until loops** and **If control statements**.

This unit will focus on **conditional loops**. We'll explore the **Button sensor** and apply logical reasoning to create engaging applications.

In class, we will explore the following:

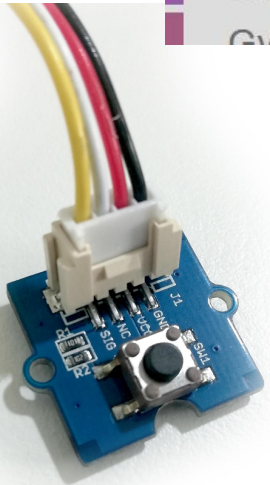
- Understand about Button sensor and its blocks.
- Forming conditions with Button sensor.
- Identifying patterns and using loops to simplify code.
- Conceptualising program flow by analysing sample output.
- Practical applications such as self-ordering kiosk and smart home control.



3005 - Conditional Loops & Button

Button Sensor

To interact with the robot



Send a value to the robot:

- 0 if the button is not pressed
- 1 if the button is pressed

Return Button Sensor Value At Port D22

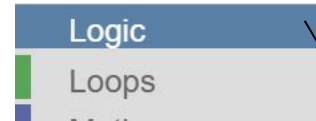
The port where the sensor is plugged
Remember to change to the correct port

Wait Until Button Sensor at Port D22 Is Pressed

If the button is currently not pressed,
wait until button is pressed

Wait Until Button Sensor at Port D22 Is Released

If the button is currently pressed,
wait until button is not pressed

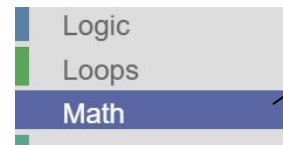


Return Button Sensor Value At Port D22 = 1

Condition to **check** if button is pressed
It is **not** setting D22 to 1

Return Button Sensor Value At Port D22 = 0

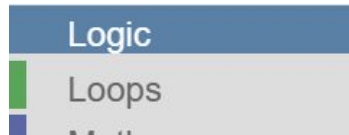
Condition to **check** if button is not pressed
It is **not** setting D22 to 0



3005 - Conditional Loops & Button

Equal to = / Not Equal to ≠

For checking if a value is the same as or different from another.



This condition will be **true** if the left and right have the **same** value
(Button block will be 1 when pressed and 0 when not)

True/False
returned ←



This condition will be **true** if the left and right have **different** value
(Button block will be 1 when pressed and 0 when not)

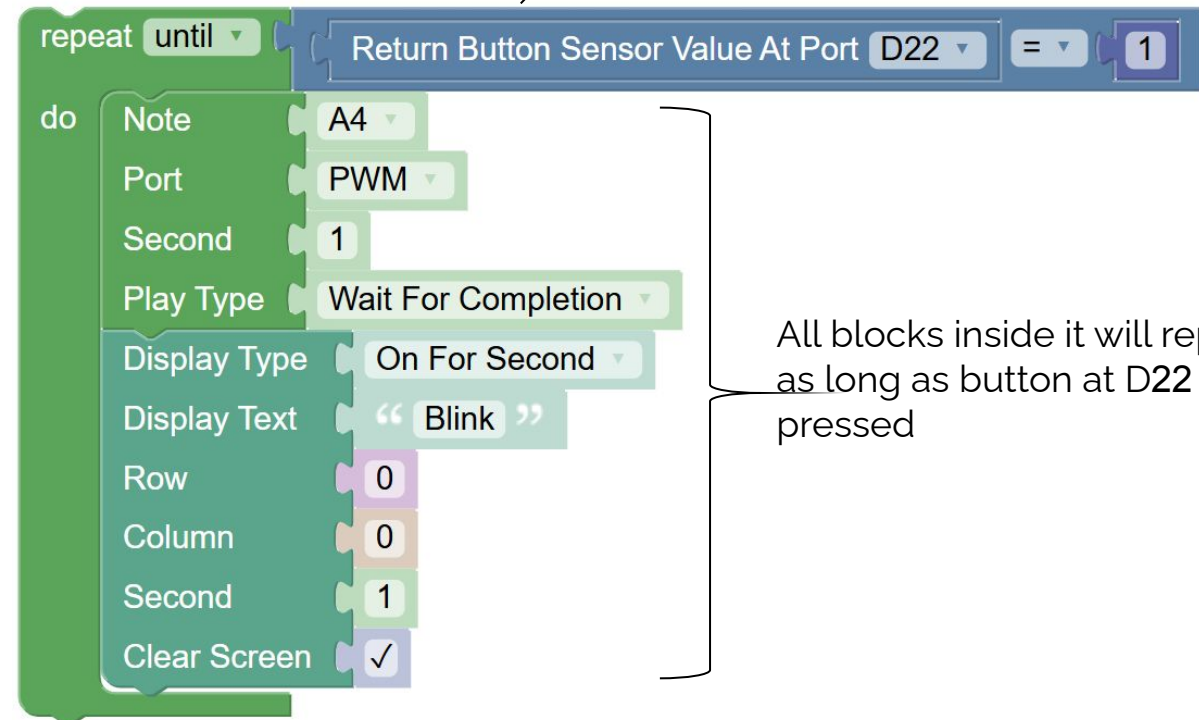
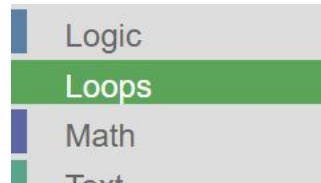
True/False
returned ←



3005 - Conditional Loops & Button

Repeat *until*

Repeat codes and stop when a condition is true



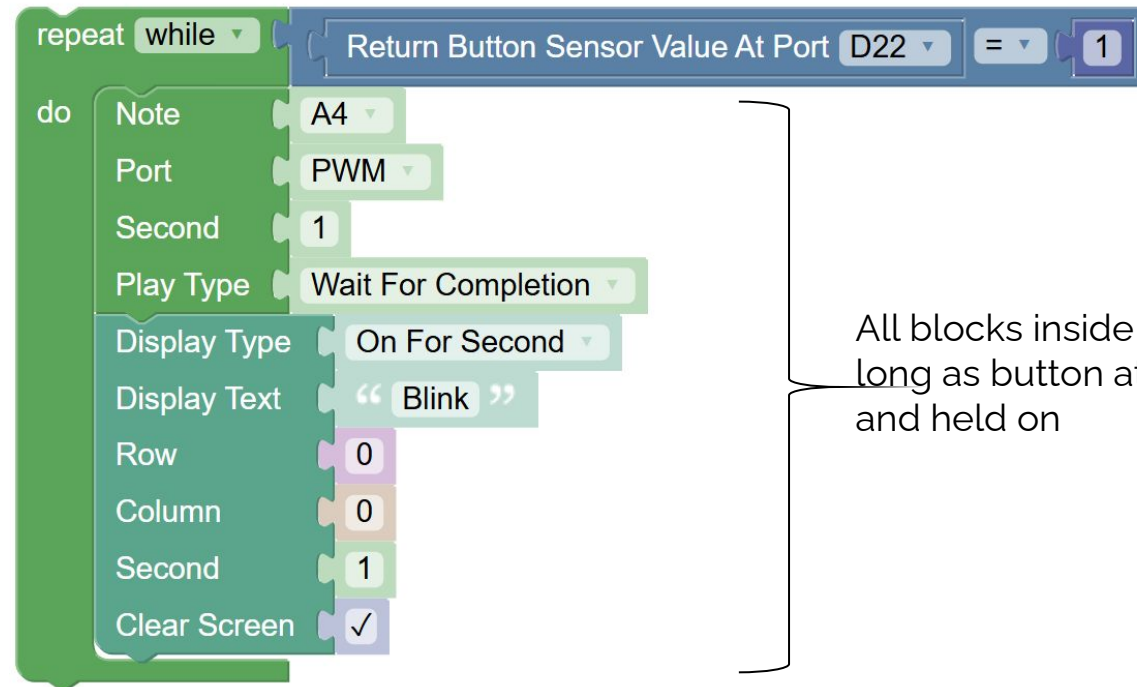
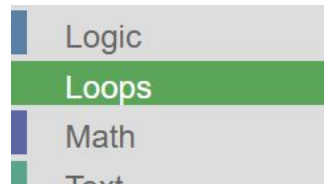
When this condition is true, the loop will stop repeating

All blocks inside it will repeat as long as button at D22 is not pressed

3005 - Conditional Loops & Button

Repeat *while*

Repeat codes while a condition is true



When this condition is true,
the loop will keep repeating

All blocks inside it will repeat as
long as button at D22 is pressed
and held on

Exercise Q1

Which of the following code snippets will make a sound **every time** the button is pressed?

Diagram 1

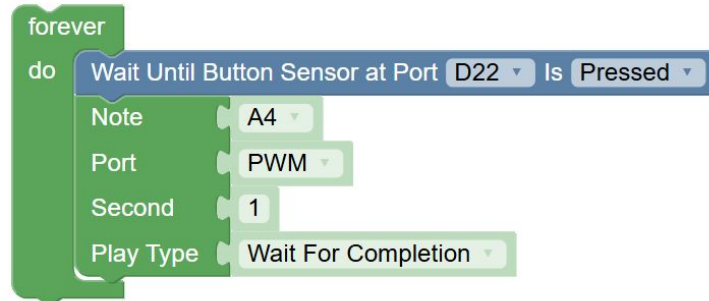


Diagram 2

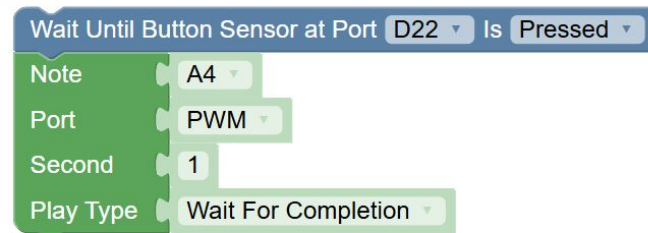
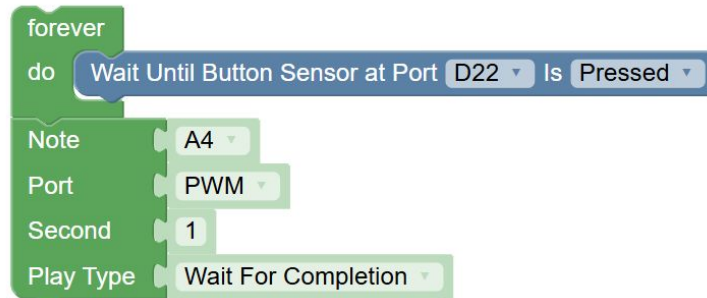


Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) Diagrams 1 and 2 only

Exercise Q2

Which of the following code snippets will make a sound when the button is pressed?

Diagram 1

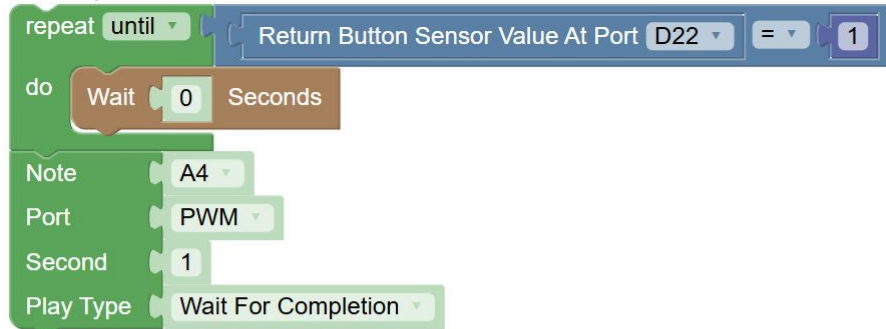


Diagram 2

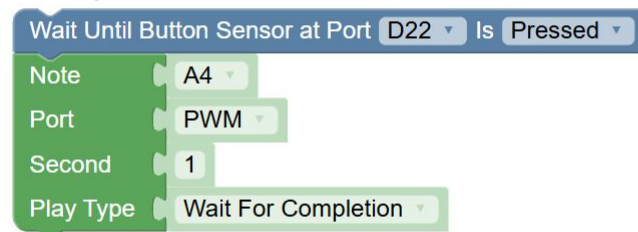
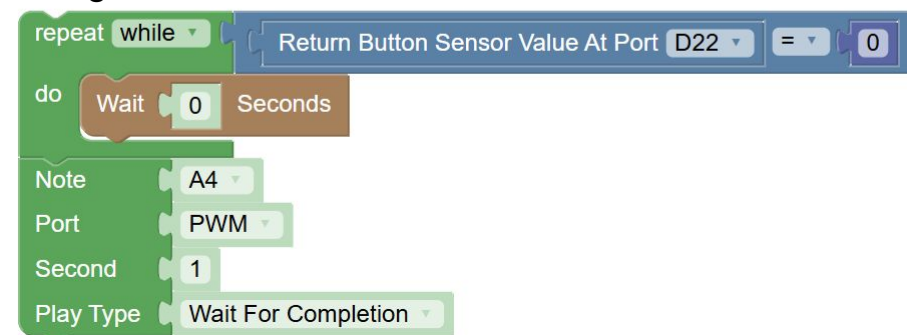


Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) All of the above

Exercise Q3

Which of the following code snippets will make a sound **after** the button is **pressed** and then **released**?

Diagram 1

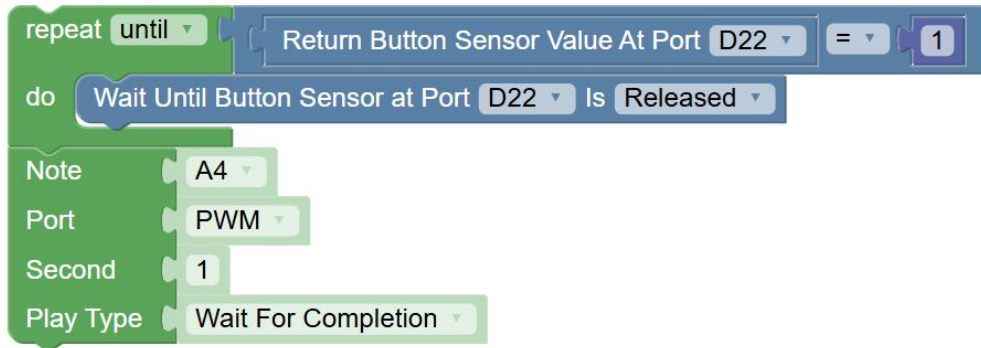
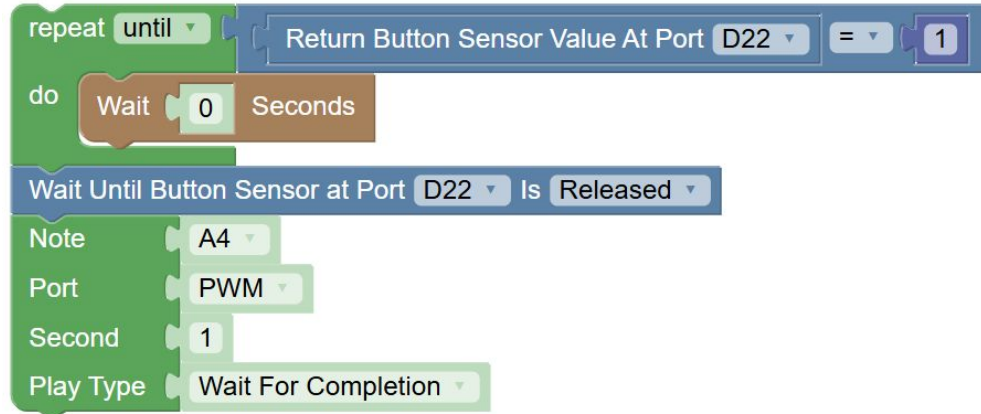


Diagram 2



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagrams 2 & 3
- D) None

Exercise Q4

Create a code that performs the following sequence:

1. Show the following until Button D22 is held down.

H	o	l	d		D	2	2								
t	o		o	p	e	n									

2. Show the following until Button D22 is released.

R	e	l	e	a	s	e		D	2	2					
t	o		c	l	o	s	e								

3. The program clears screen and ended.

Answers

Q1: A

Diagram 2 can only work once while diagram 3 is trapped in a loop.

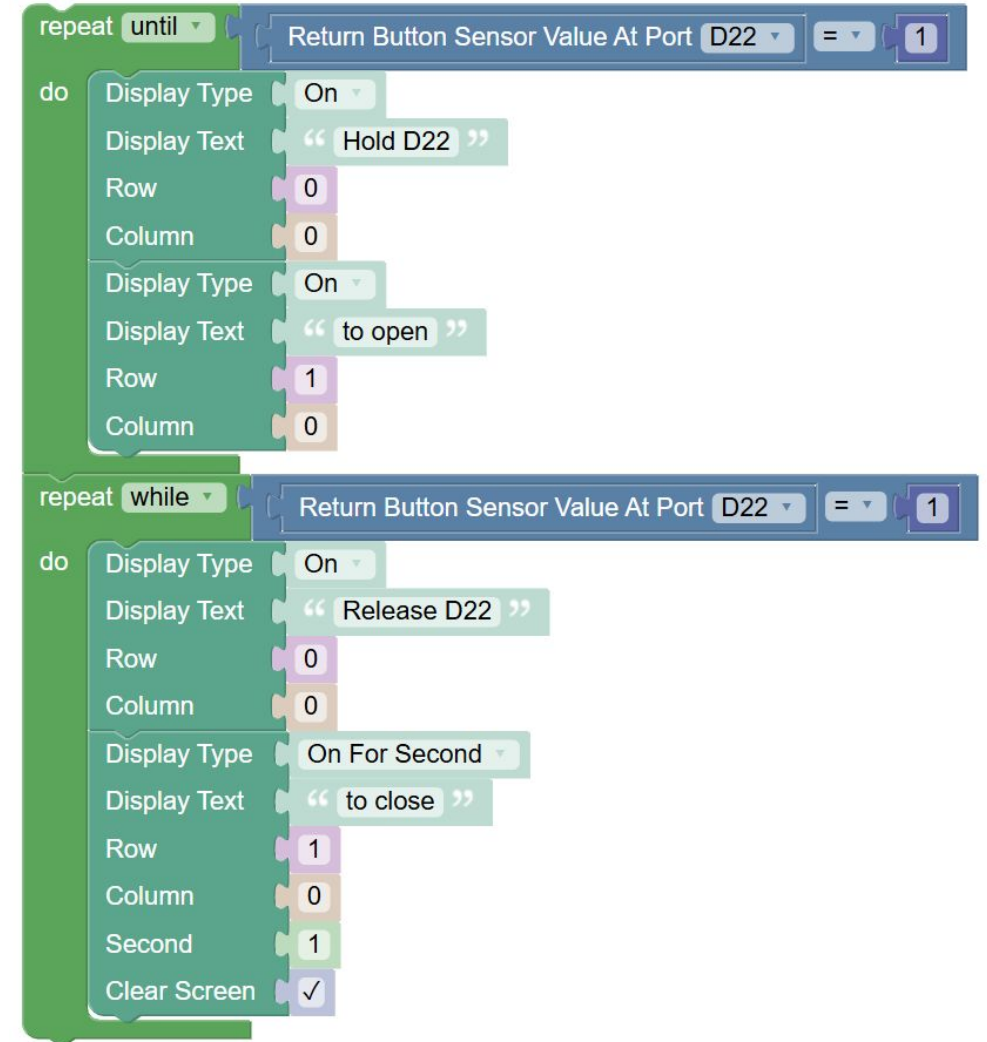
Q2: D

The repeat blocks work like the "wait until button pressed" block. Only when the button is pressed, then it will run the next block.

Q3: B

In diagram 1, once the button is pressed, it will exit the loop and not wait for the button to be released.

Q4:



3006 - If Statement

Learning

You have learnt about conditional loops that can be used to add logic to a program. The **If control statement** is the other **main logic** component in coding.

An **If control statement** is used to execute a block of code only when a specified condition is true. It lets the program make decisions and respond differently depending on varying inputs or situations. If control statement can be structured in various ways to form simple or complex logic.

In this unit, we will learn about the different structures of the If control statement.

In class, we will explore the following:

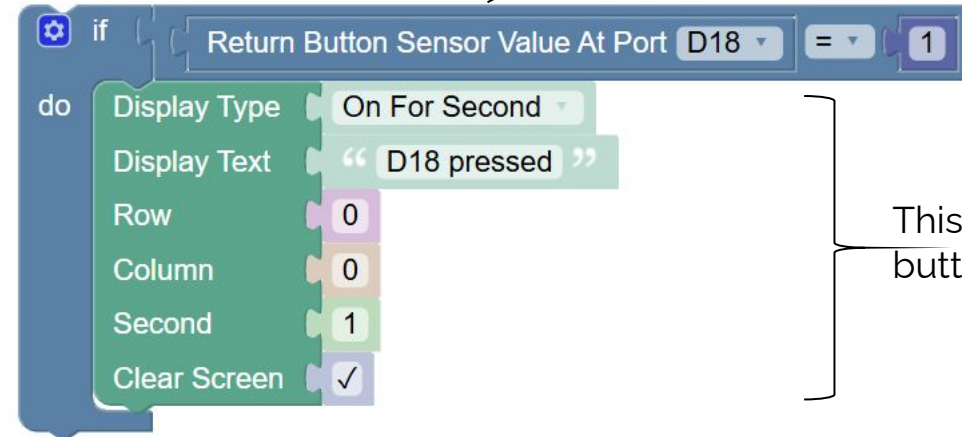
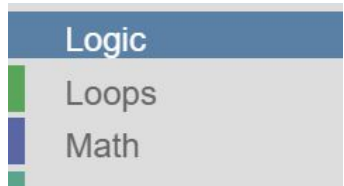
- Understand the different If control structures (e.g. If, If-Else and If-Else If. etc).
- Combining conditional loops and If statements.
- Conceptualising program flow by analysing sample output.
- Practise drawing flowchart depicting logic structure.
- Applications including a fun speed training program and a digital water dispenser with animated display.



3006 - If Statement

If

If a condition is true, then run some codes



If this condition is true,
the blocks in "do" will run.

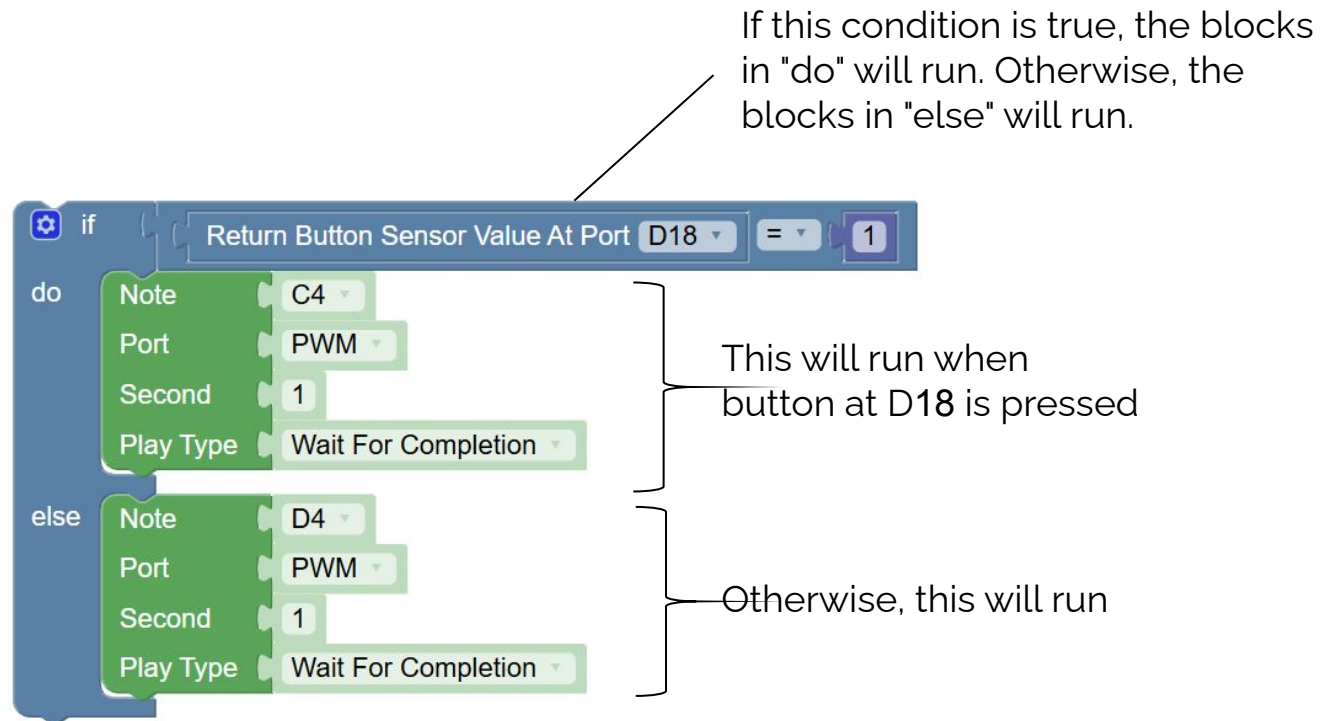
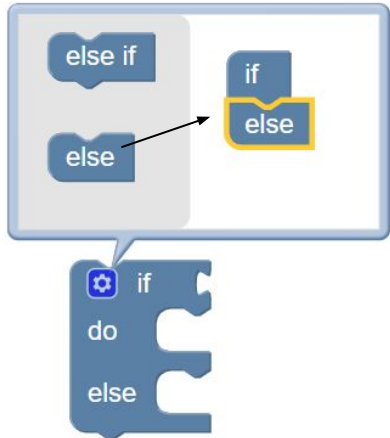
This will run when
button at D18 is pressed

3006 - If Statement

If...else

If a condition is true, then run some codes.

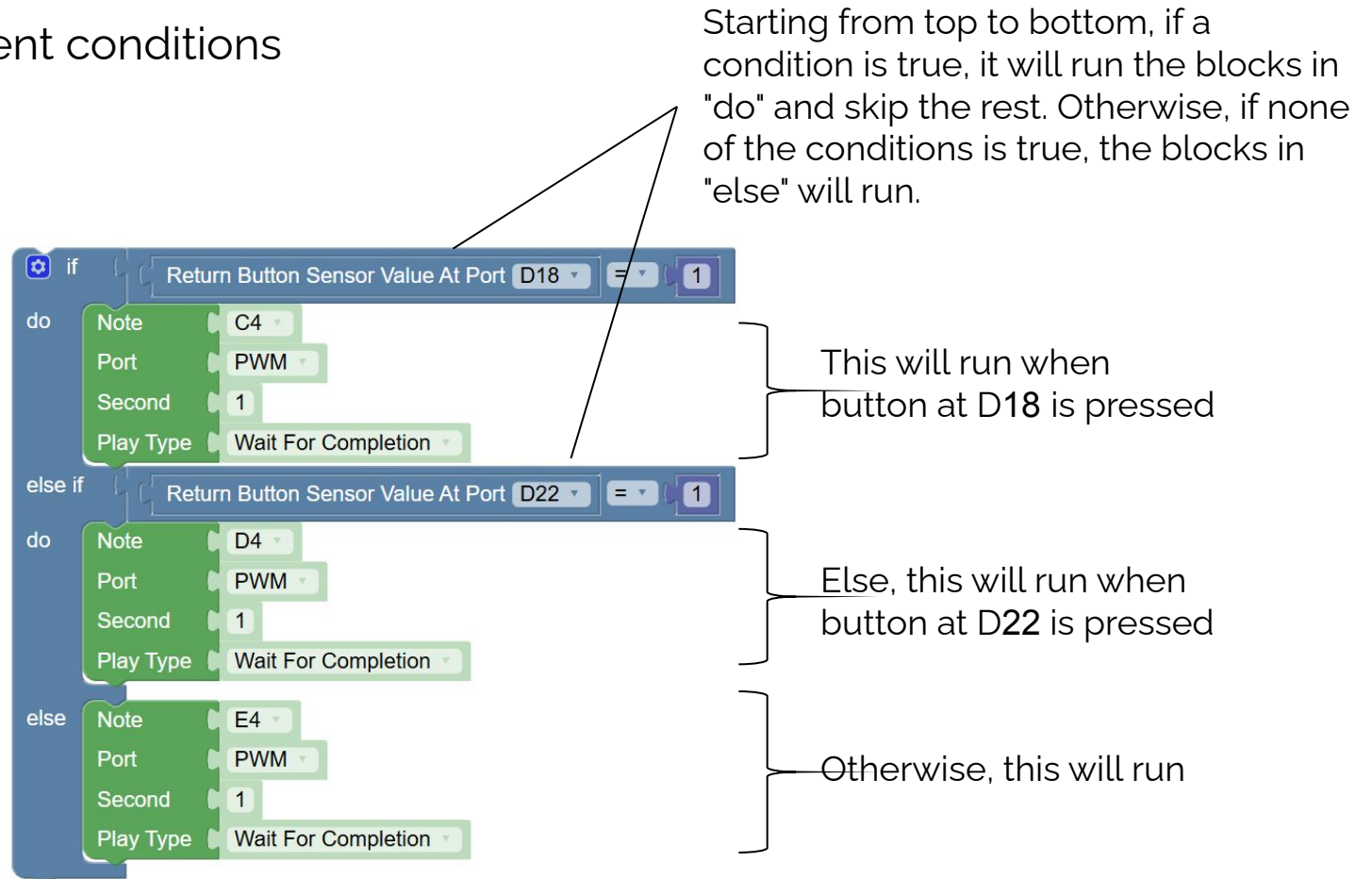
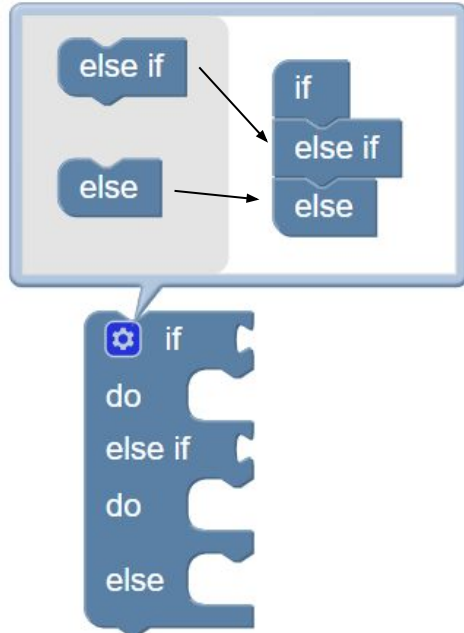
Otherwise, run some other codes.



3006 - If Statement

If...else if...else

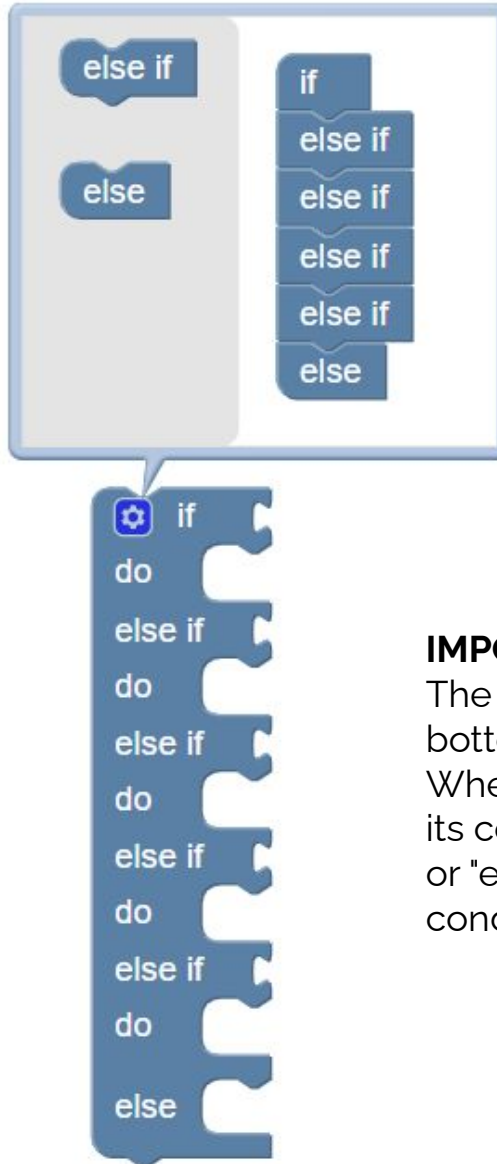
Run different codes based on different conditions



3006 - If Statement

If...else if...else

You can add many "else if"

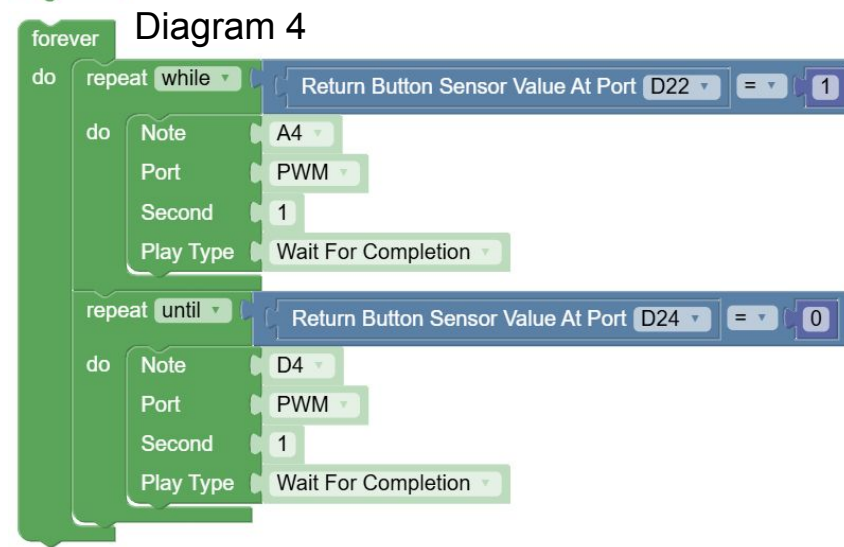
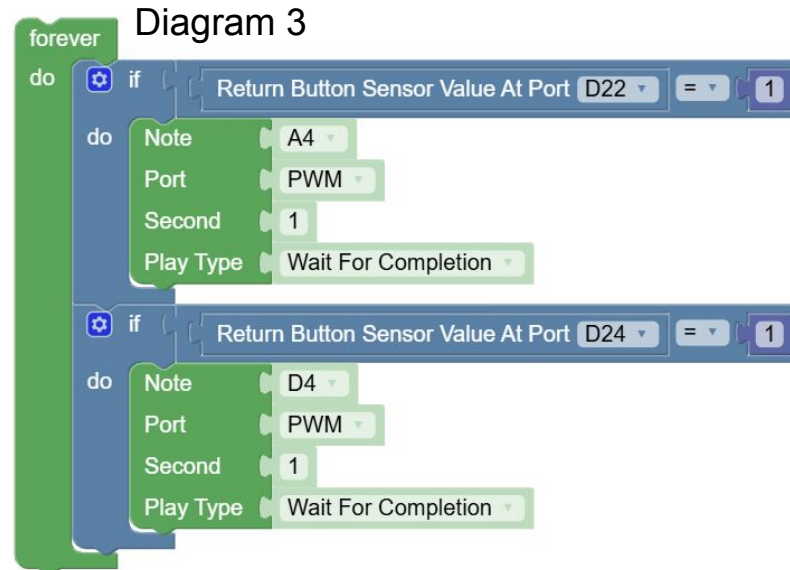
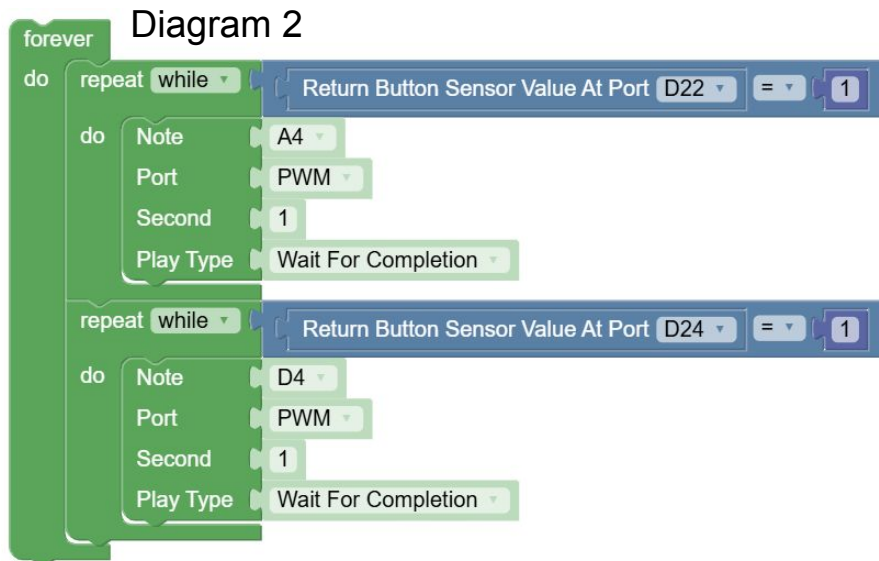
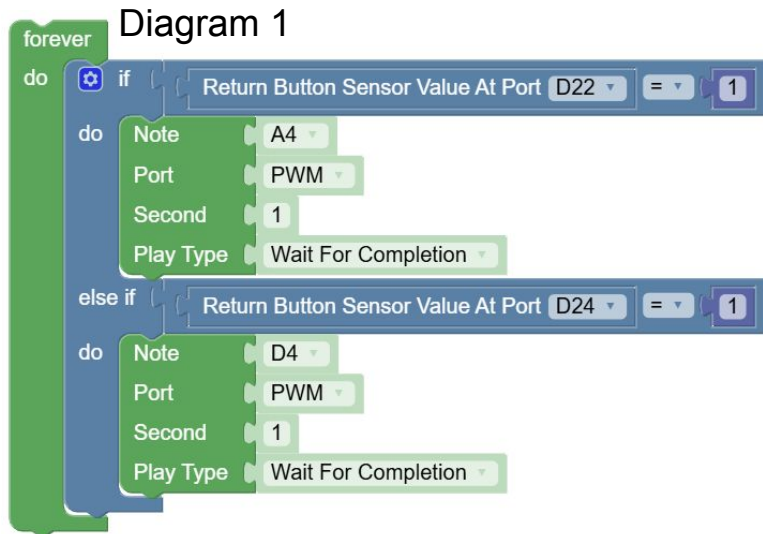


IMPORTANT:

The program will check from top to bottom to see which condition is true. When one of the conditions is true and its codes are run, the remaining "else if" or "else" will be skipped (even if the conditions in other "else if" is true).

Exercise Q1

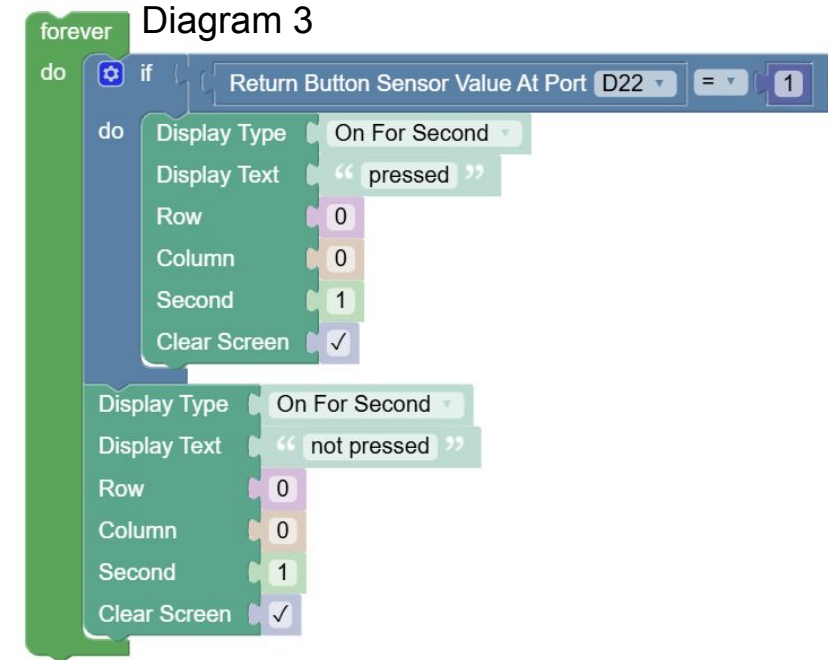
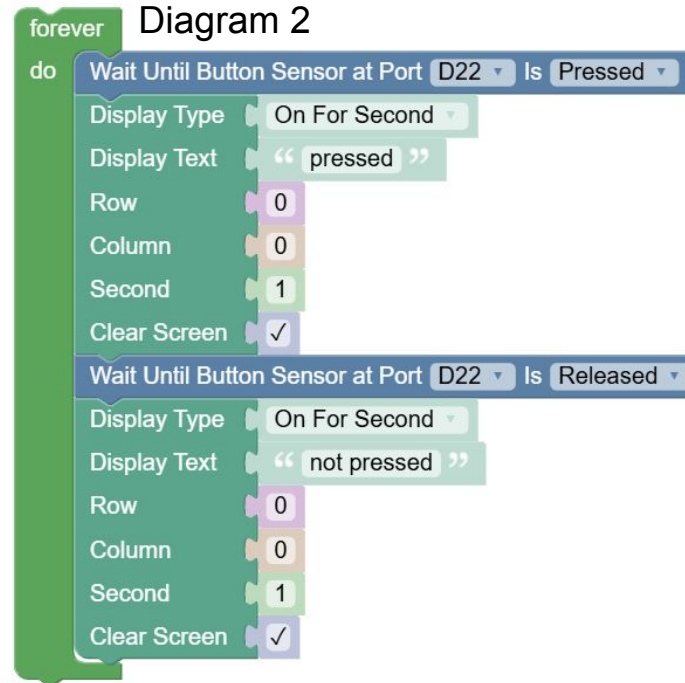
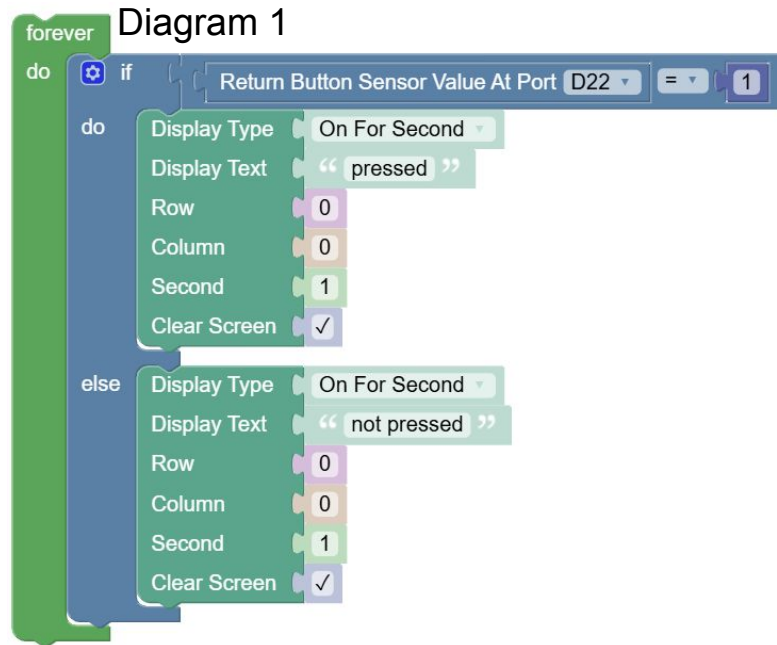
Which of the following code snippets will produce the **same** results regardless of how the buttons are pressed?



- A) Diagrams 1 and 3 only
- B) Diagrams 2 and 4 only
- C) Diagrams 1, 2 and 4 only
- D) All of them

Exercise Q2

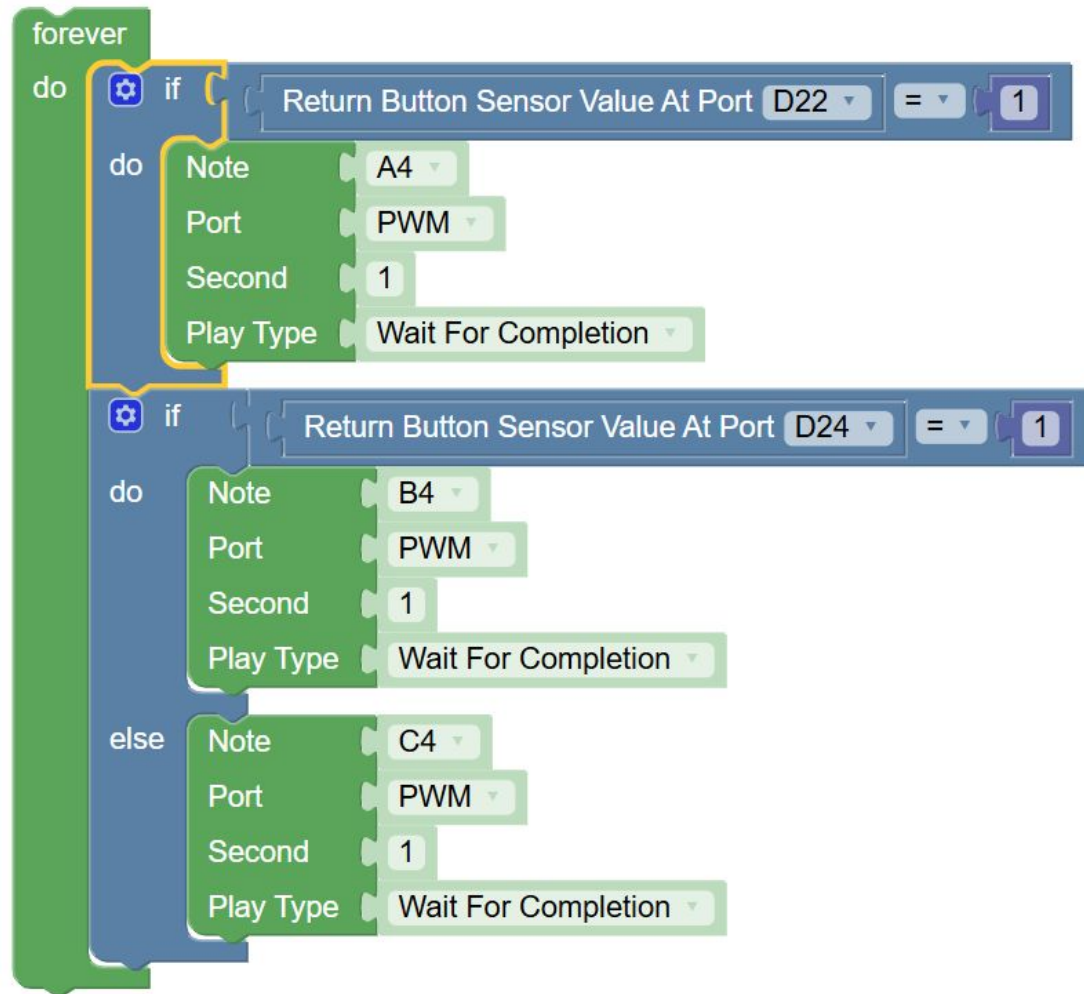
Which of the following code snippets will show “press” **whenever** Button D22 is held on and “not pressed” when it is not?



- A) Diagram 1 only
- B) Diagrams 1 and 2 only
- C) Diagrams 1 and 3 only
- D) Diagrams 1, 2 and 3

Exercise Q3

Which statement(s) is/are correct about the code?



- A) C4 will NOT be played when D22 is held down.
- B) C4 will NOT be played when D24 is held down.
- C) Both statements are true.
- D) None of the statements is true.

Exercise Q4

Create a code that performs the following:

Condition	Display Message
D18 pressed	D18 is working
D22 pressed	D22 is working
D24 pressed	D24 is working
No button pressed	Test a button

Answers

Q1: C

If both buttons are held down, diagram 3 will produce alternating A4 and D4 as both If blocks will be run. The others will keep repeating Note A4.

Q2: A

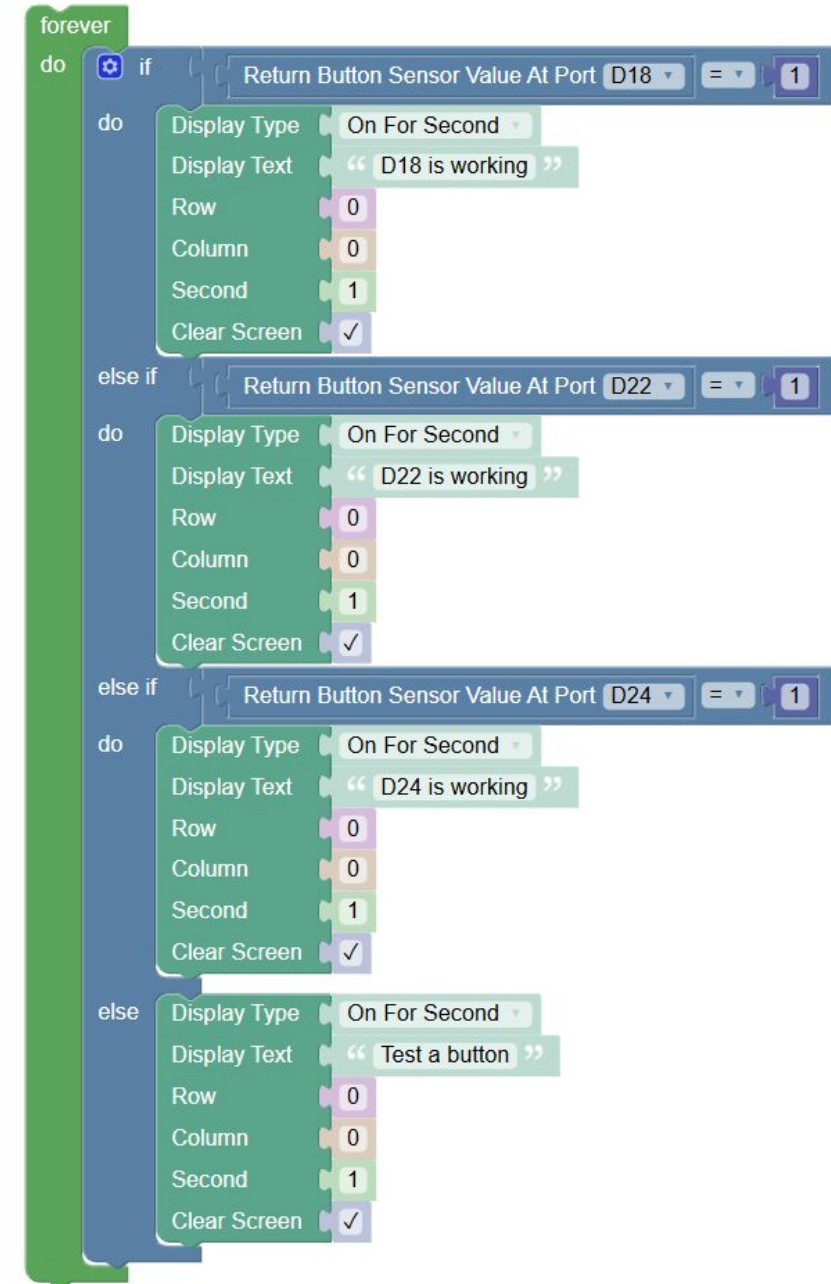
In diagram 2, you can only alternate press and release to work.

In diagram 3, when you hold down the button, “press” and “not pressed” will be shown alternately.

Q3: B

When D22 is held down, after playing A4 for 1 second, it will enter the else of the second if statement to play C4 since D24 is not pressed.

Q4:



3007 - Comparison & AND Operators & Ultrasonic Sensor

Learning

Not only the If control statement can take on different structures, its conditions can vary widely by using different combinations of operators.

Comparison operators are use to compare values. **AND operator** is used to combine multiple conditions. The combined condition is true only when all the individual conditions are true.

In this unit, we will learn about the **Ultrasonic distance sensor** and build programs using logic structures and operators. We will also learn to read tabulated data and forming code from flowchart.

In class, we will explore the following:

- Understand Ultrasonic distance sensor and its blocks.
- Learn about range and how to form conditions to check for range.
- How to break from loops.
- Practise reading tabulated data.
- Practise forming code from a flowchart.
- Practical applications involving facial recognition system and car proximity sensors.
- Fun applications including Shuffleboard game and cool DJ controller.



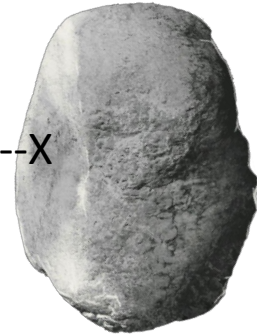
3007 - Comparison & AND Operators & Ultrasonic Sensor

Ultrasonic Distance Sensor

For detecting object in front of the sensor



It can measure distance from 0 to about 500cm



Button Sensor

Ultrasonic Sensor

Gyro Sensor

Return Distance in Centimetres At Port D26

Send the distance value to the robot

The port where the sensor is plugged

Wait Until Distance at Port D26 > 1 CM

Wait until the range condition is met
(in this example is distance less than 5 cm)

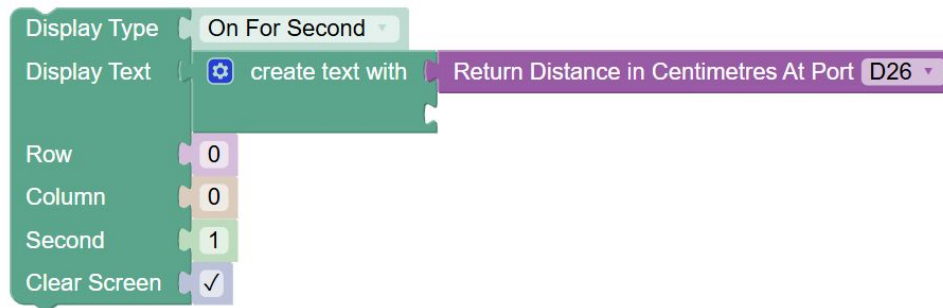
Select your comparison operator:

- = equal to
- ≠ not equal to
- < smaller than
- > greater than
- ≤ smaller than or equal to
- ≥ greater than or equal to

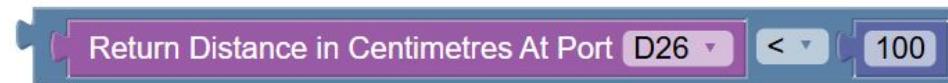
3007 - Comparison & AND Operators & Ultrasonic Sensor

Ultrasonic Distance Sensor

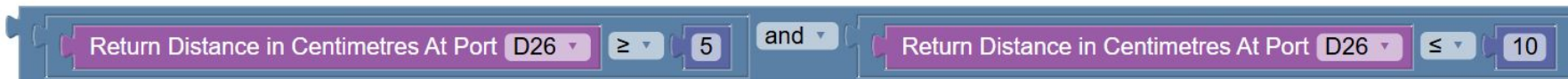
Checking range



The distance scanned often changes rapidly. Hence, instead of using the equal operator, we normally use the smaller/bigger than operators to check for ranges as shown below.



Condition to check if ultrasonic distance is less than 100cm

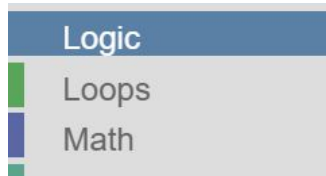


Condition to check if ultrasonic distance is from 5 to 10cm away

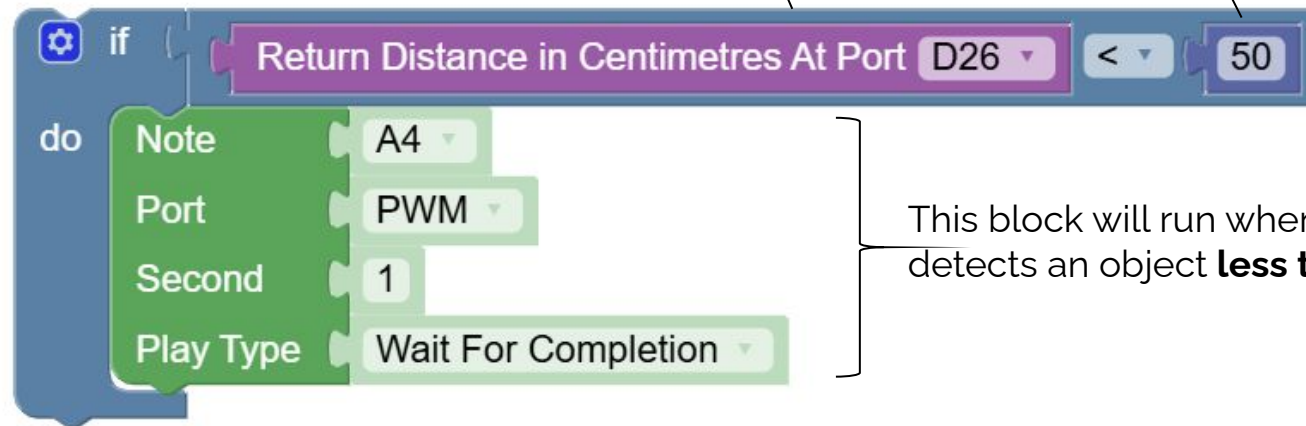
3007 - Comparison & AND Operators & Ultrasonic Sensor

Smaller than <

For checking if a value is less than another



This condition will be true if the left value is less than the right value

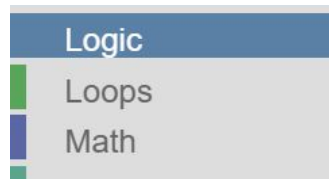


This block will run when the ultrasonic sensor detects an object **less than 50cm** away

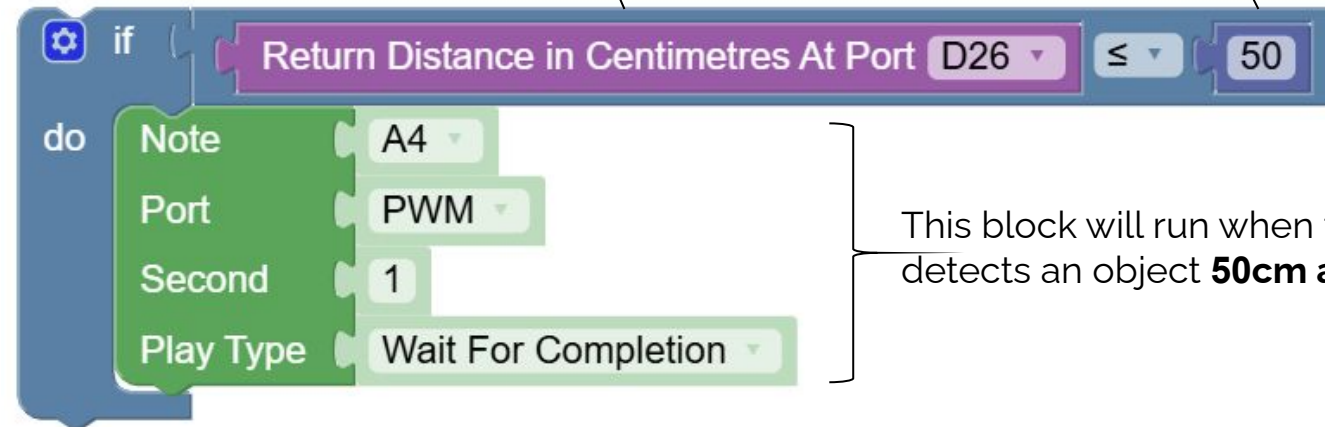
3007 - Comparison & AND Operators & Ultrasonic Sensor

Smaller than or Equal to $\leq$

For checking if a value is less than or equal to another



This condition will be true if the left value is less than or equal to the right value

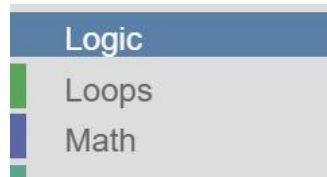


This block will run when the ultrasonic sensor detects an object **50cm away or less**

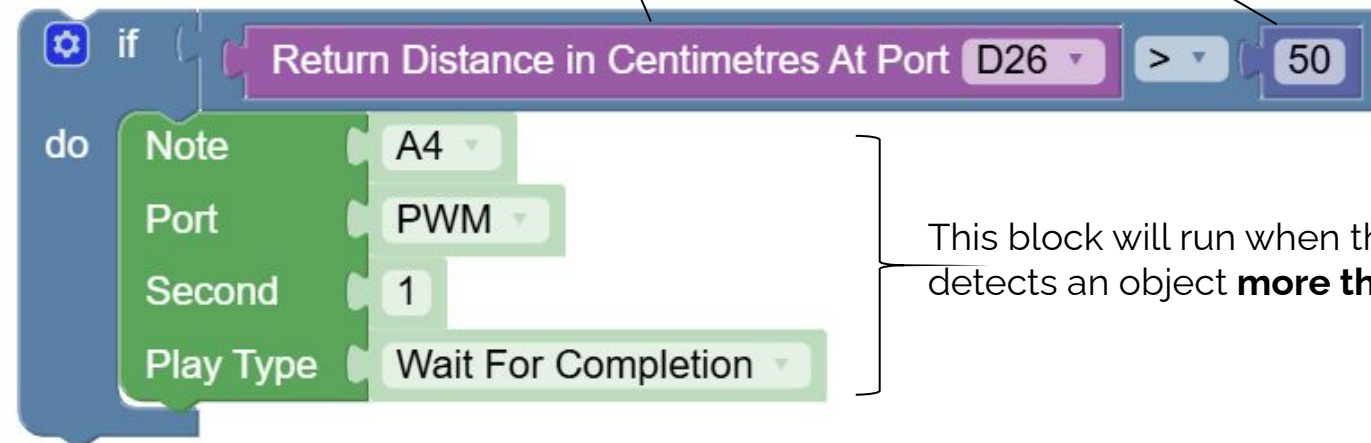
3007 - Comparison & AND Operators & Ultrasonic Sensor

Greater than >

For checking if a value is more than another



This condition will be true if the left value is more than the right value

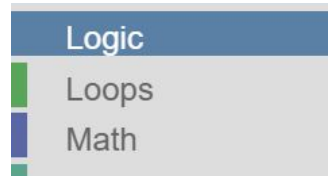


This block will run when the ultrasonic sensor detects an object **more than 50cm** away

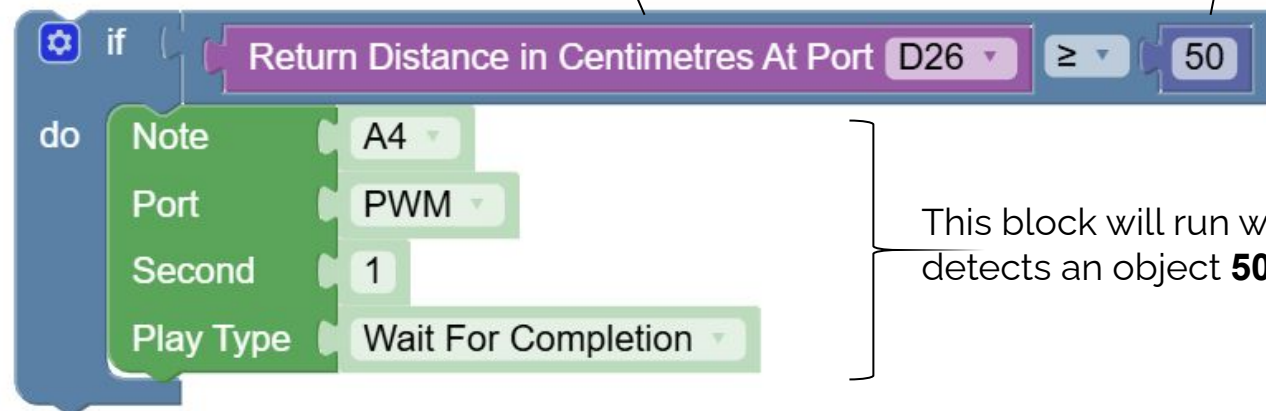
3007 - Comparison & AND Operators & Ultrasonic Sensor

Greater than or Equal to $\geq$

For checking if a value is more than or equal to another



This condition will be true if the left value is more than or equal to the right value

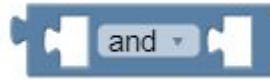
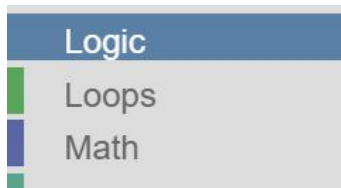


This block will run when the ultrasonic sensor detects an object **50cm away or more**

3007 - Comparison & AND Operators & Ultrasonic Sensor

AND

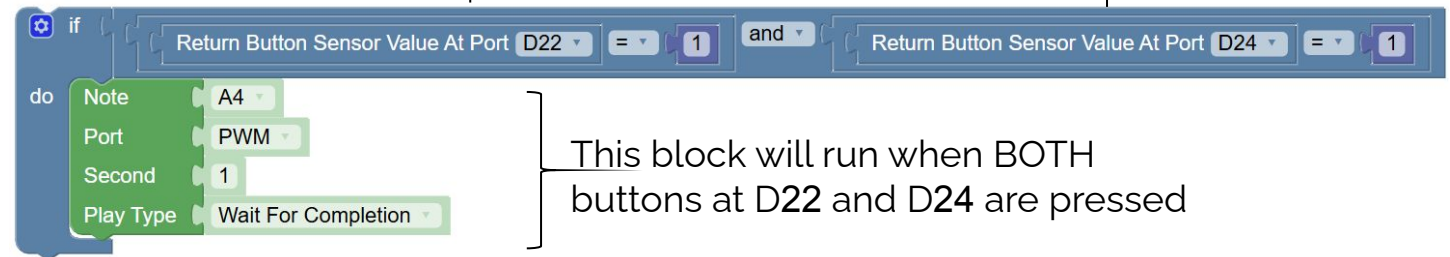
For checking if two conditions are **BOTH** true



This condition will be true if the left and right conditions are **BOTH** true

Left condition: button at D22 pressed

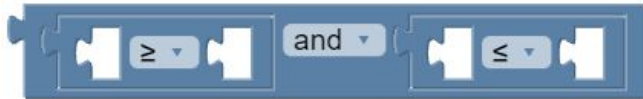
Right condition: button at D24 pressed



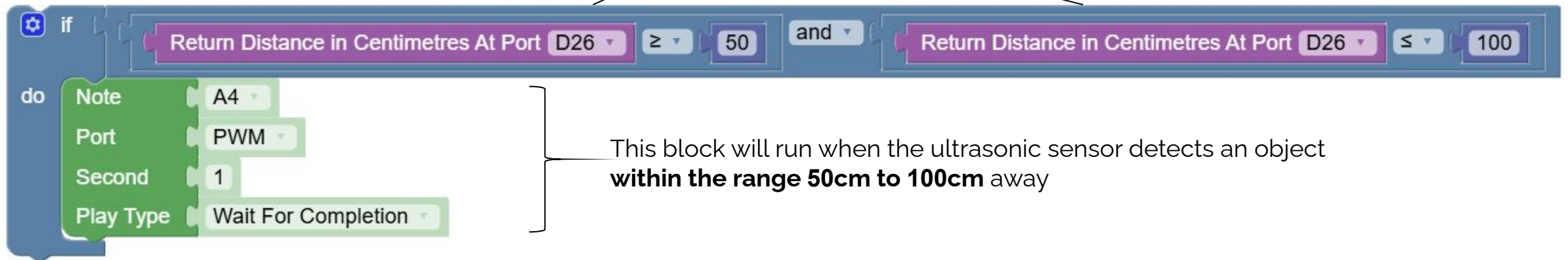
3007 - Comparison & AND Operators & Ultrasonic Sensor

Checking within a range

For checking if a value within a range



This condition will be true if both the left and right conditions are true

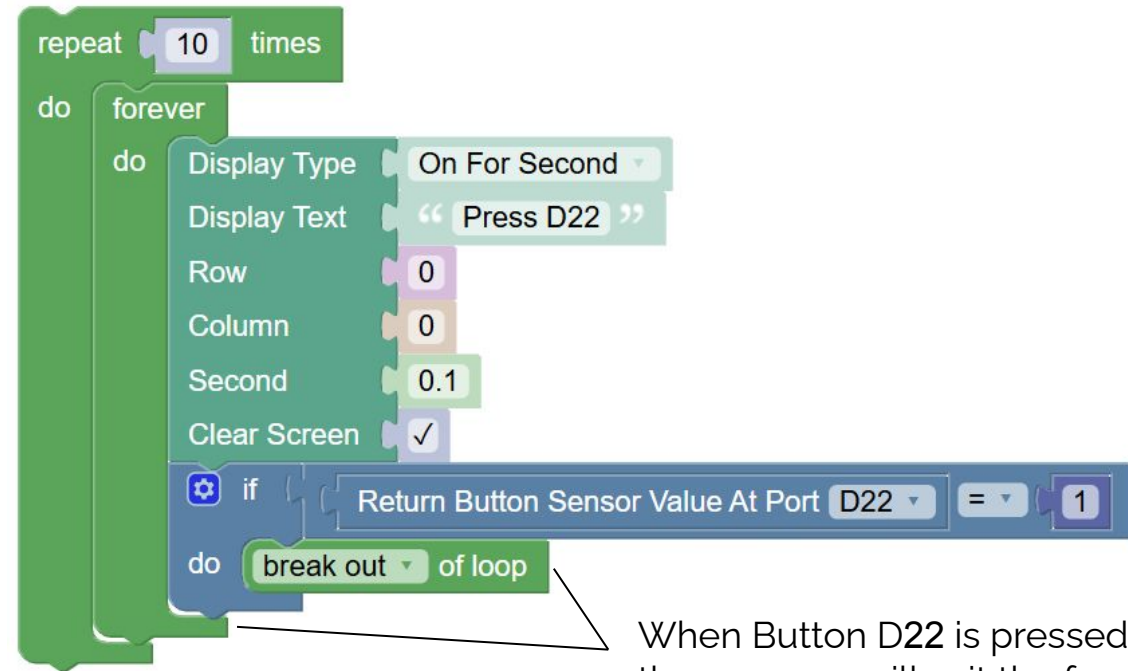
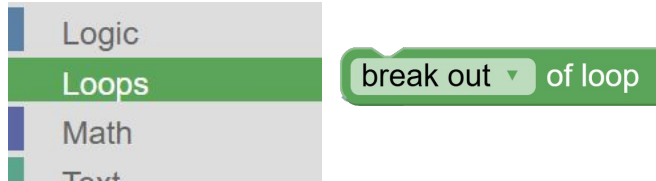


This block will run when the ultrasonic sensor detects an object **within the range 50cm to 100cm** away

3007 - Comparison & AND Operators & Ultrasonic Sensor

Break from loop

Exit from the **nearest** loop

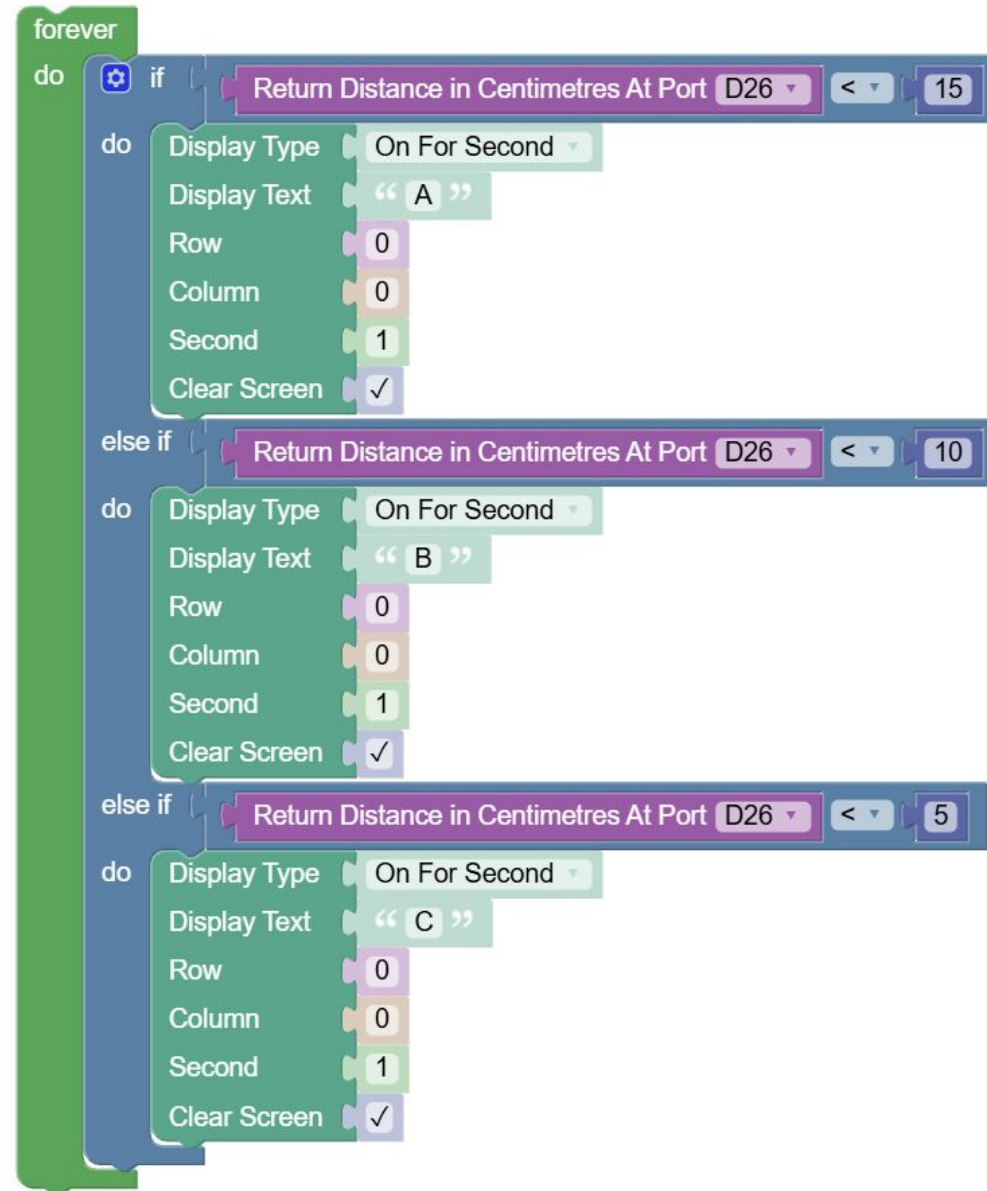


When Button D22 is pressed, the program will exit the forever loop only and gets back into for 10 times. Hence, you need to press 10 times to end the program.

Exercise Q1

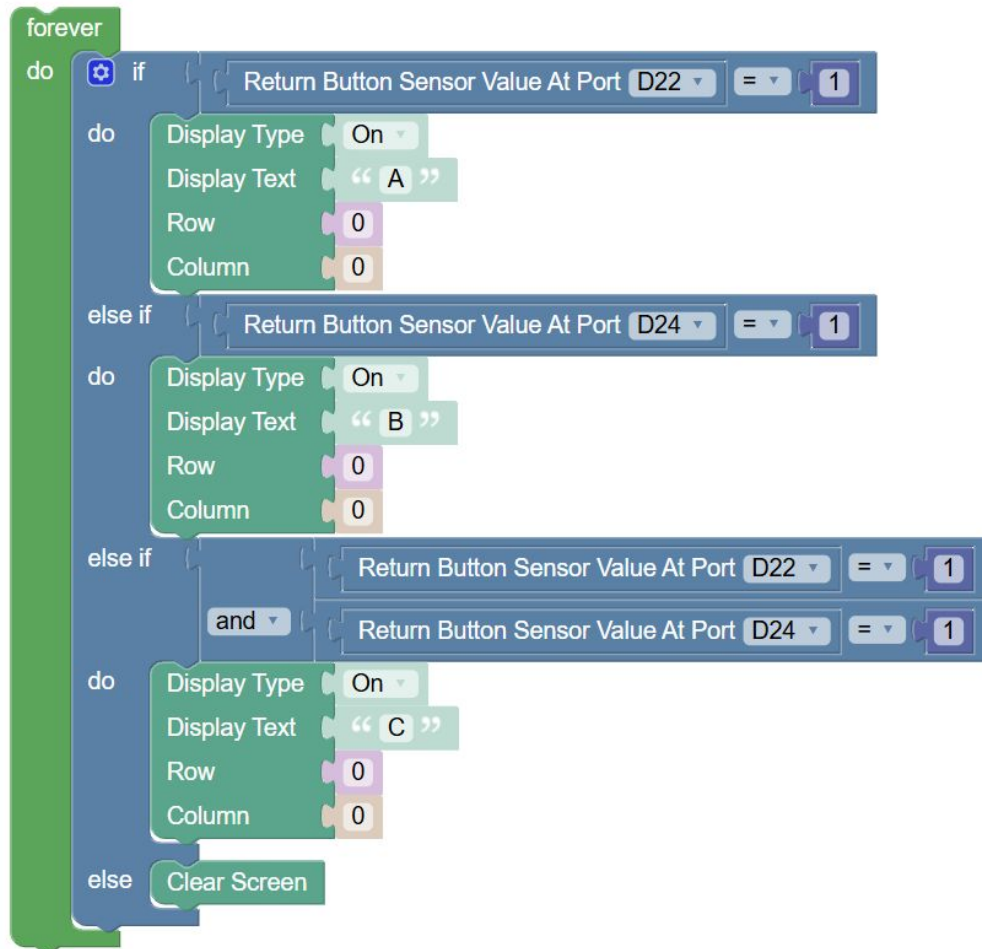
When the obstacle distance is **constantly at 3cm**, which of the following will be shown when the code is run.

- A) A
- B) B
- C) C
- D) A, B then C repeatedly



Exercise Q2

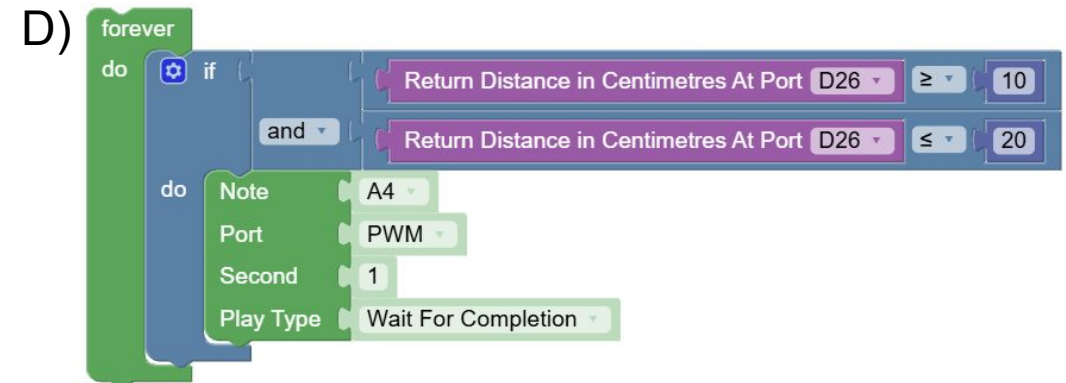
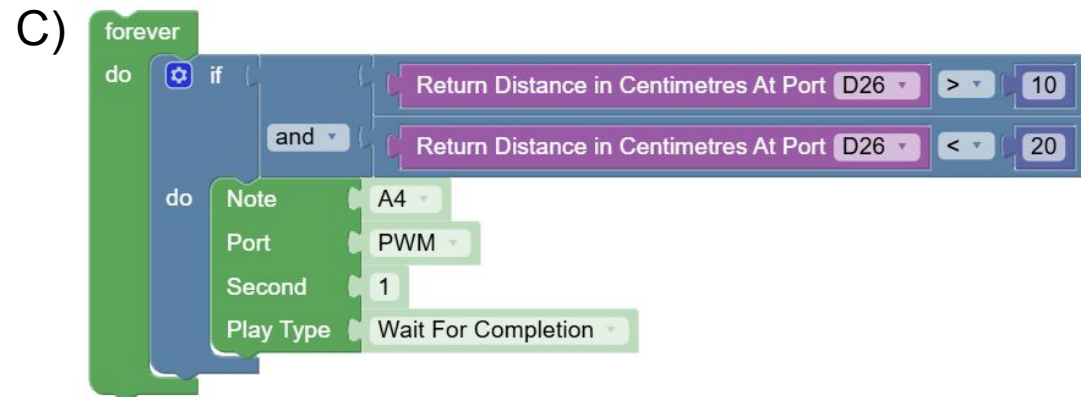
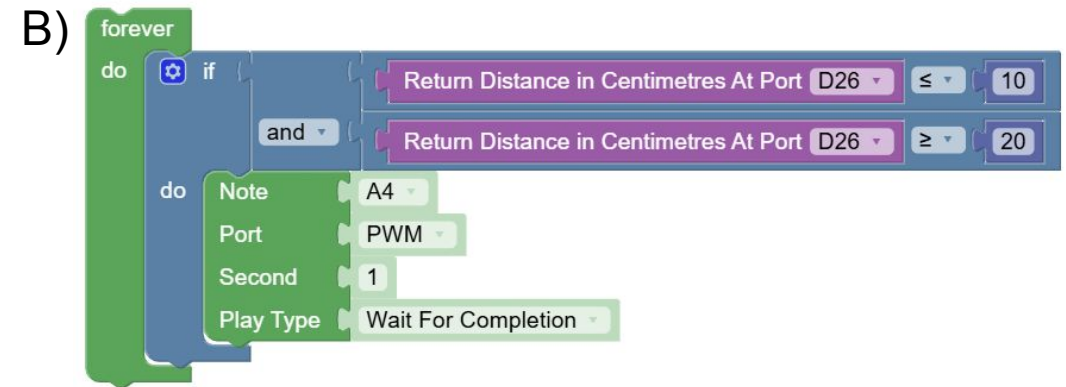
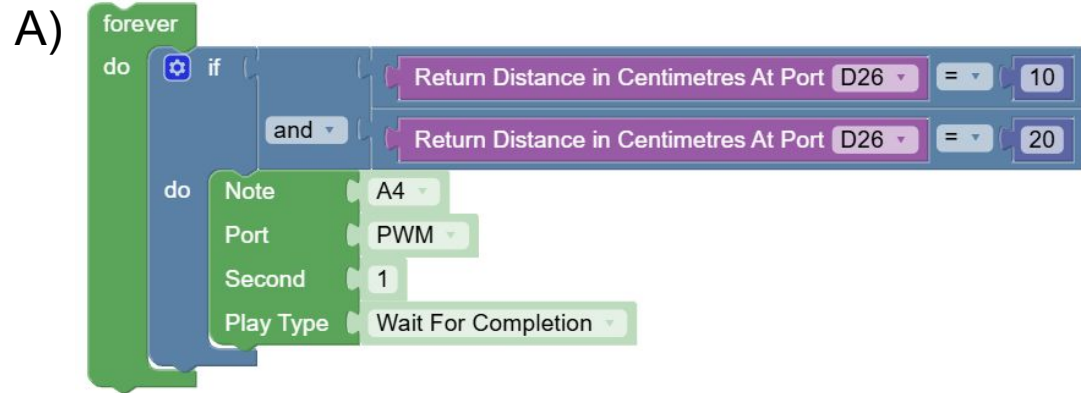
What will be displayed when the code is run and **both buttons** are held down.



- A) A
- B) B
- C) C
- D) Nothing

Exercise Q3

Which of the following code will produce a sound when an obstacle is **10cm to 20cm** away?



Exercise Q4

Create a code that performs the following:

Obstacle Distance	Display Message
Less than 5cm away	Zone A
10cm to 20cm away	Zone B
Otherwise	Zone C

Answers

Q1: A

The first condition is true so it will keep displaying A and skip checking the remaining conditions.

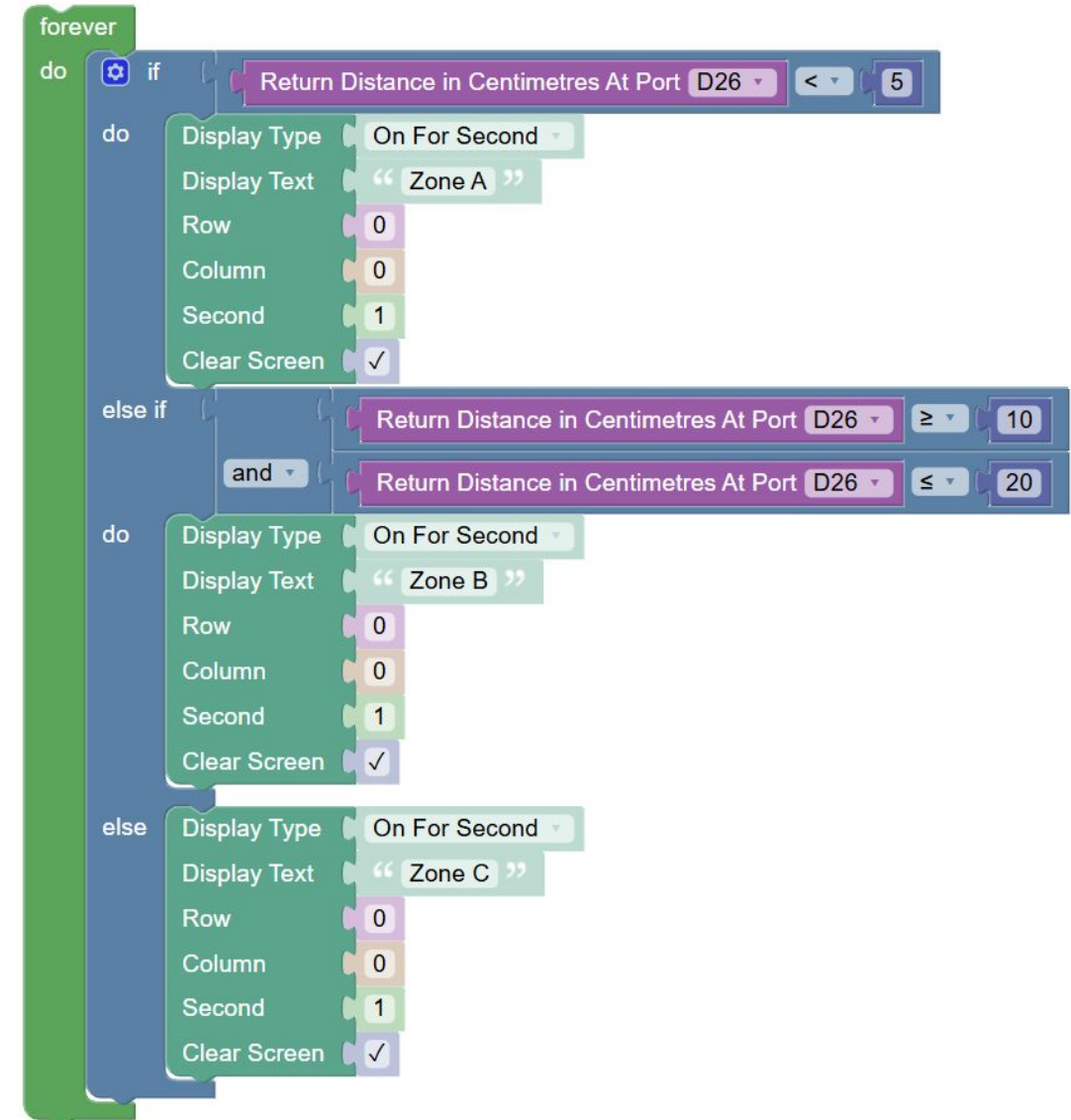
Q2: A

When both buttons are held down, all three conditions are true and 'A' will be shown as its condition is checked first.

Q3: D

To be within a range, it must be “bigger or equal” ($\geq$) to the smaller number AND “smaller or equal” ($\leq$) to the bigger number.

Q4:



3008 - Comparison & OR Operators

Learning

An **OR operator** is used to check multiple conditions. The combined condition is true when at least one of the individual conditions is true.

In this unit, you will learn about the difference between OR and AND operators, and how to combine multiples them to check for more conditions.

In class, we will explore the following:

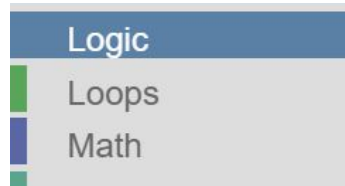
- Understand the difference between OR and AND operators and apply them in different situations.
- More practices on tabulated data reading and sample output analysis.
- Practise forming code from a flowchart.
- Applications including science experiment support app and polling app.



3008 - Comparison & OR Operators

OR

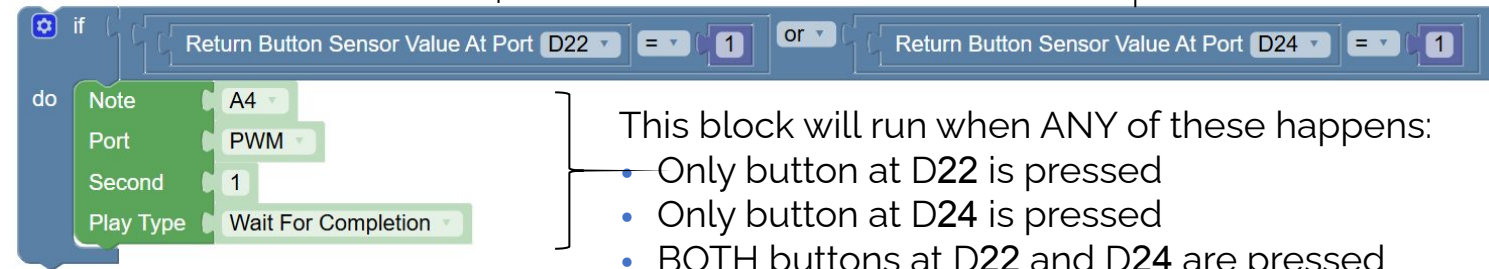
For checking if **AT LEAST ONE** of the conditions is true



This condition will be true if either the left or right or both conditions are true

Left condition: button at D22 pressed

Right condition: button at D24 pressed



3008 - Comparison & OR Operators

COMBINING OPERATORS

You can check if multiple conditions are **ALL** true



You can also check if **AT LEAST ONE** of multiple conditions is true

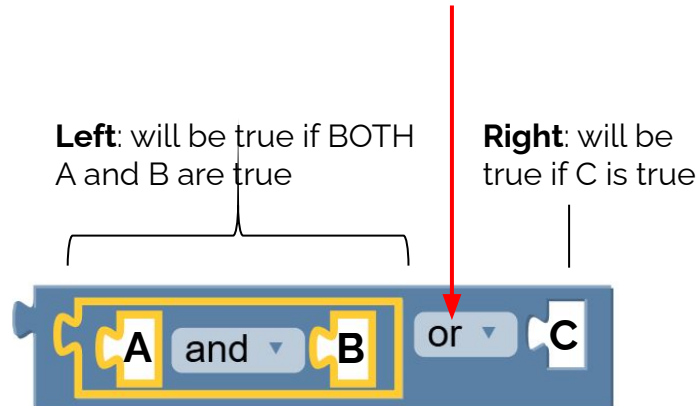


3008 - Comparison & OR Operators

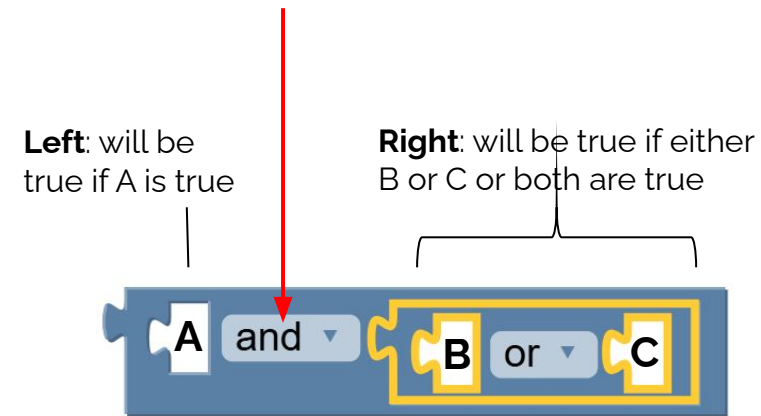
AND + OR

You can mix AND and OR blocks to form complex conditions but be careful of how the logic works

This condition will be true if either the left **or** right **or** both are true

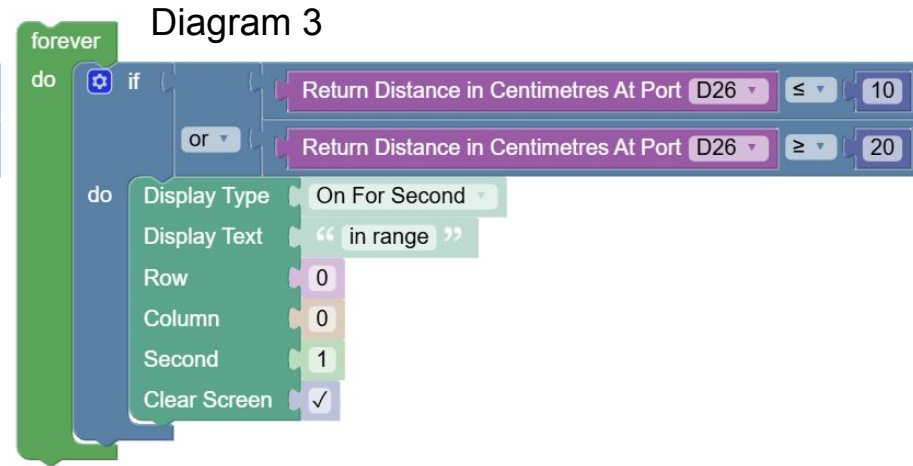
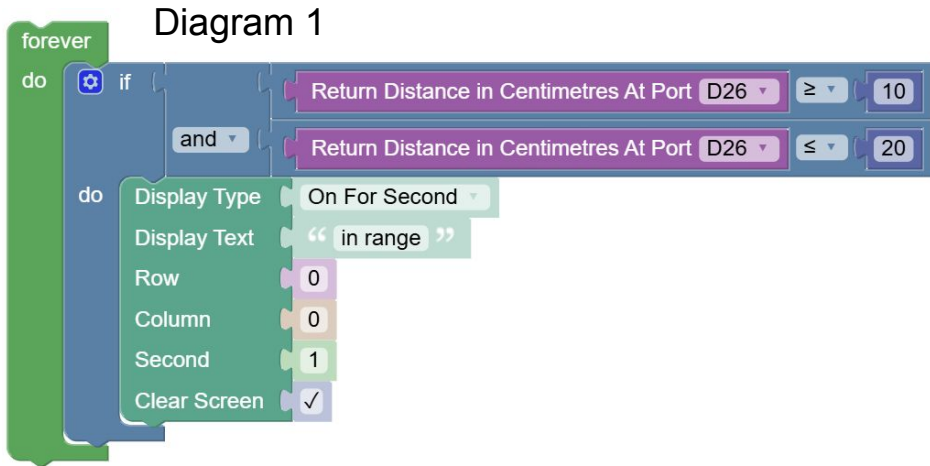


This condition will be true if both left **and** right are true

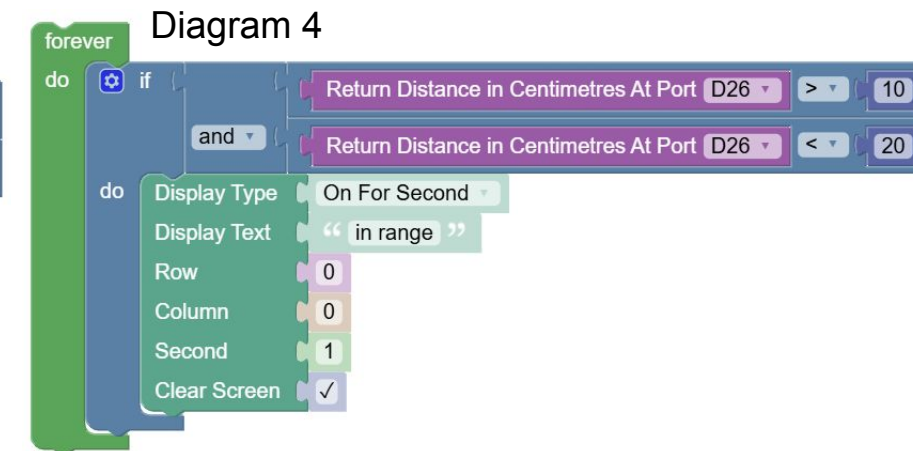
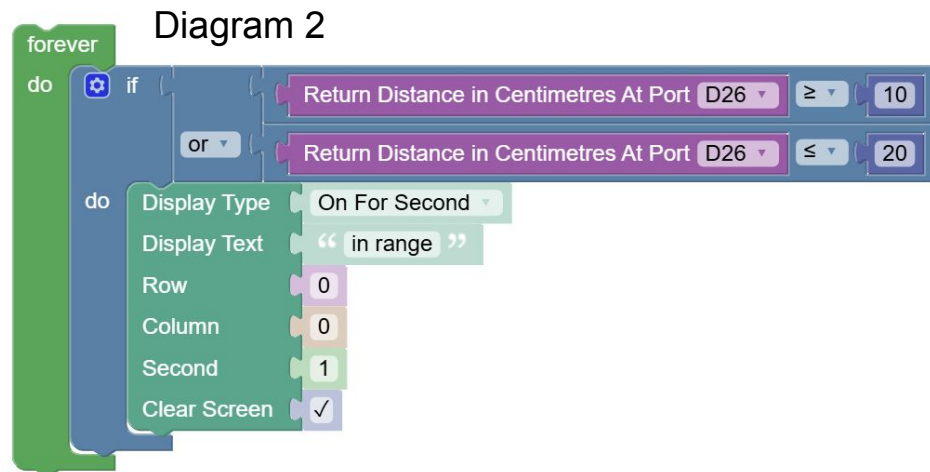


Exercise Q1

Which of the following will show “in range” **only when** the ultrasonic sensor detects an obstacle at 25cm away

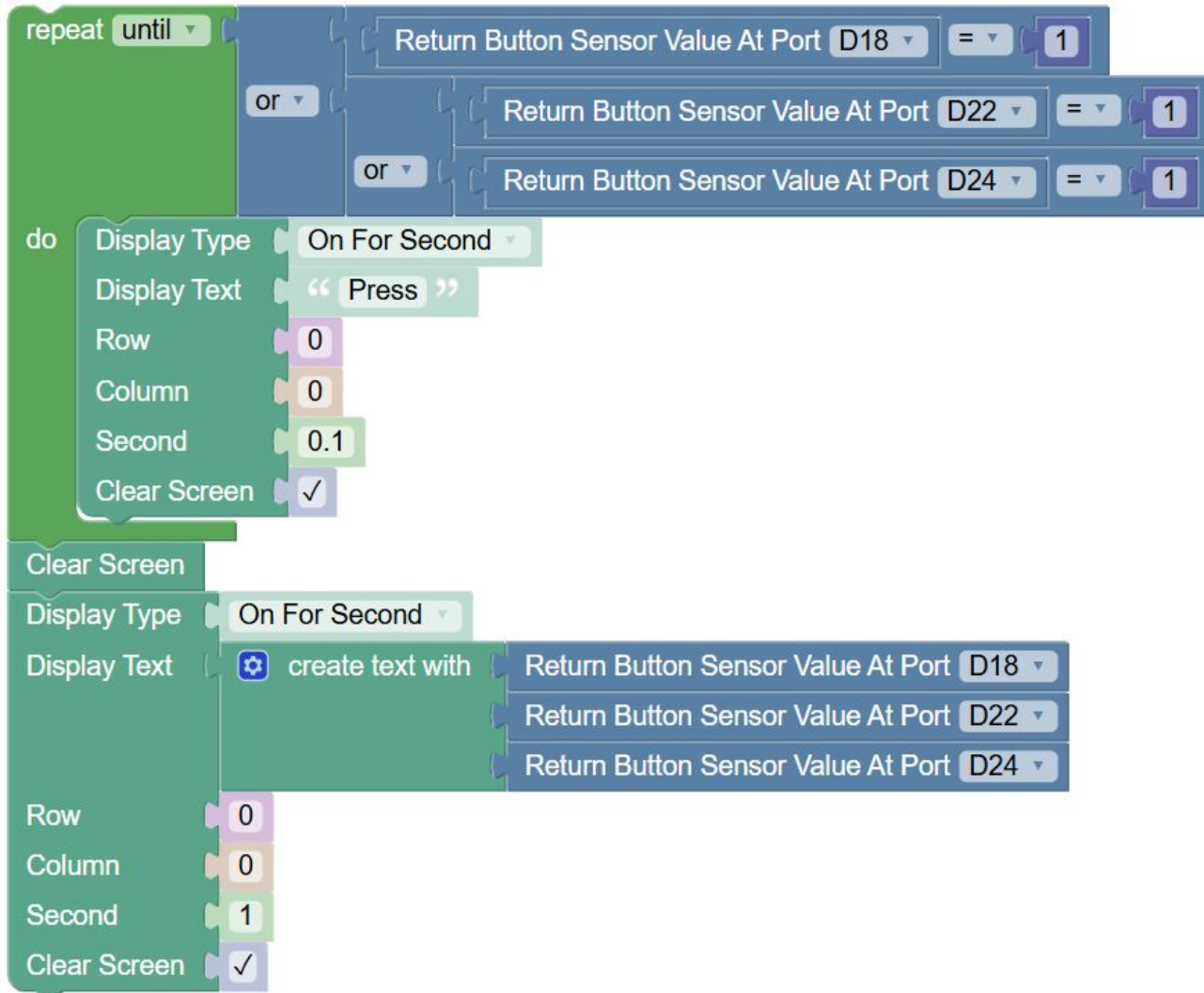


- A) Diagram 1
- B) Diagrams 1 & 4
- C) Diagram 3
- D) Diagrams 2 & 3



Exercise Q2

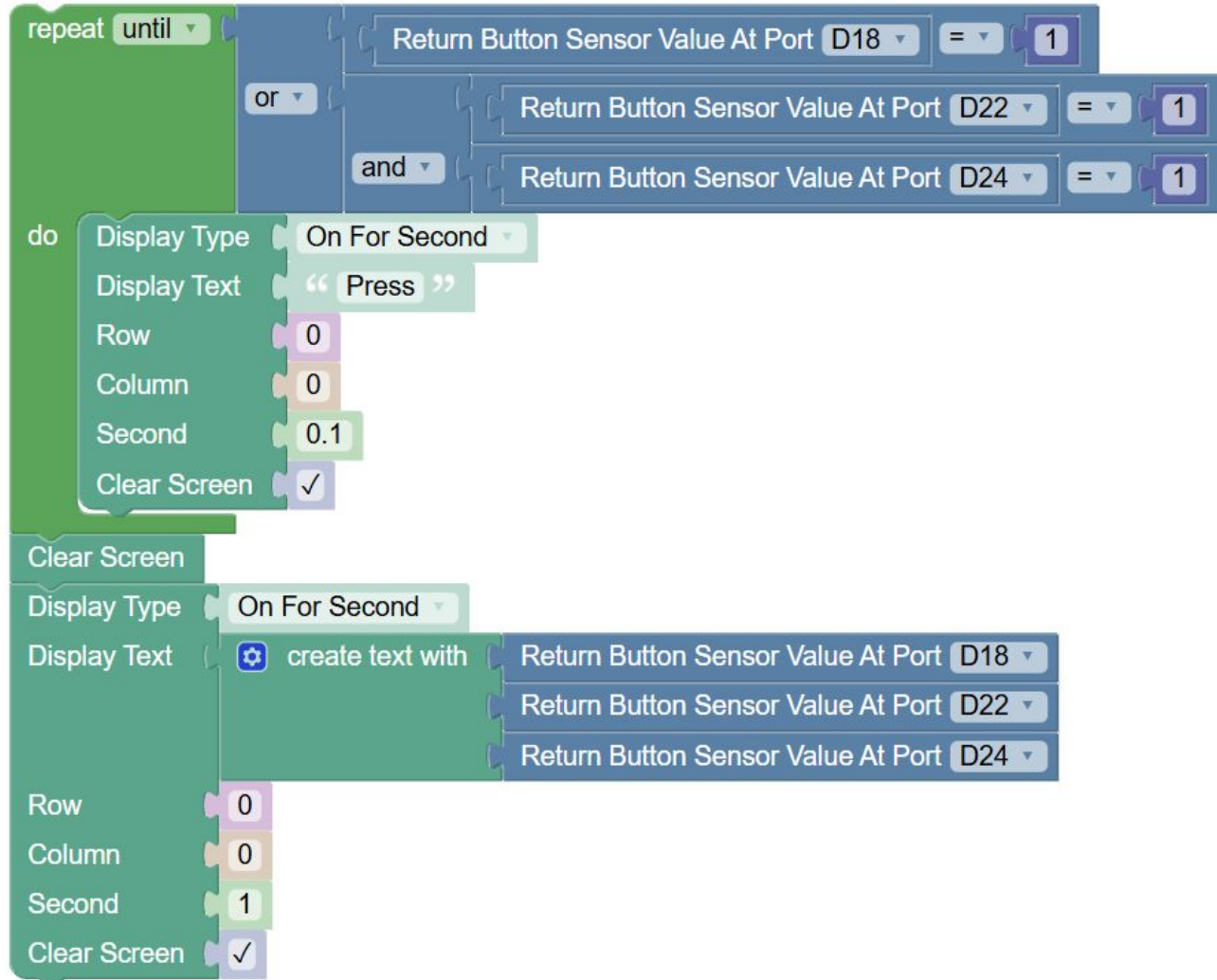
In the following code, what will be shown on the LCD when **D22** and **D24** are **pressed together**?



- A) FalseTrueTrue
- B) 011
- C) 100
- D) Press

Exercise Q3

In the following code, what will be shown on the LCD when **D18** and **D22** are **pressed together**?



- A) TrueFalseFalse
- B) 100
- C) 110
- D) Press

Exercise Q4

Create a code that performs the following:

Condition	Display Message
Button 18 pressed and Obstacle distance is less than 30cm	Sprinkle water
Button 22 and 24 pressed	Pump water

Answers

Q1: D

Diagram 1: 10 to 20

Diagram 2: All distances

Diagram 3: 0 to 10 OR 20 and above

Diagram 4: 11 to 19

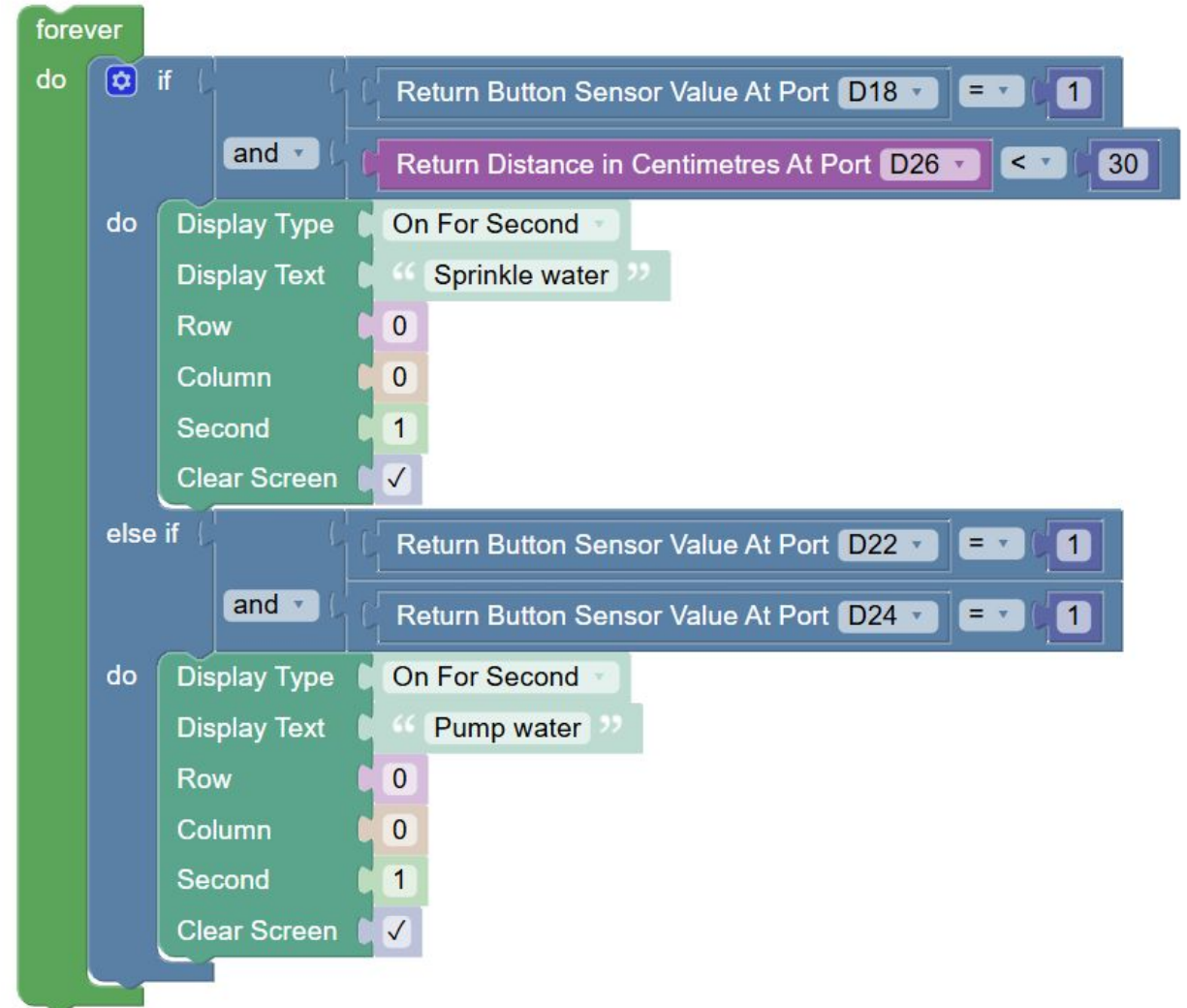
Q2: B

Button block returns 1 when the button is pressed and 0 when not.

Q3: C

Pressing D18 will exit the loop regardless of D22 and D24. Button block returns 1 when the button is pressed and 0 when not.

Q4:

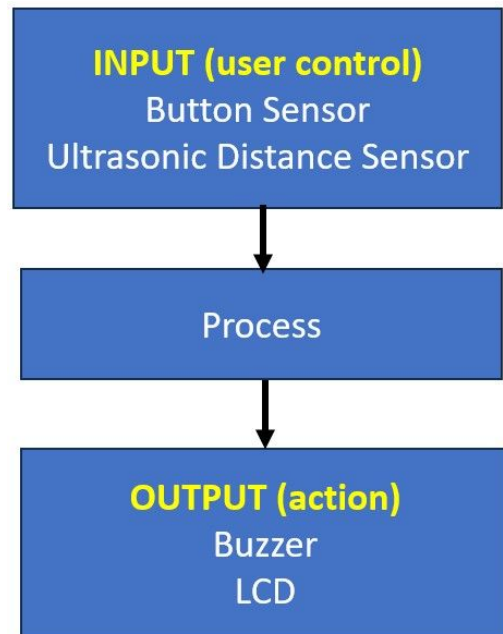


3009 - Basic 1 Project

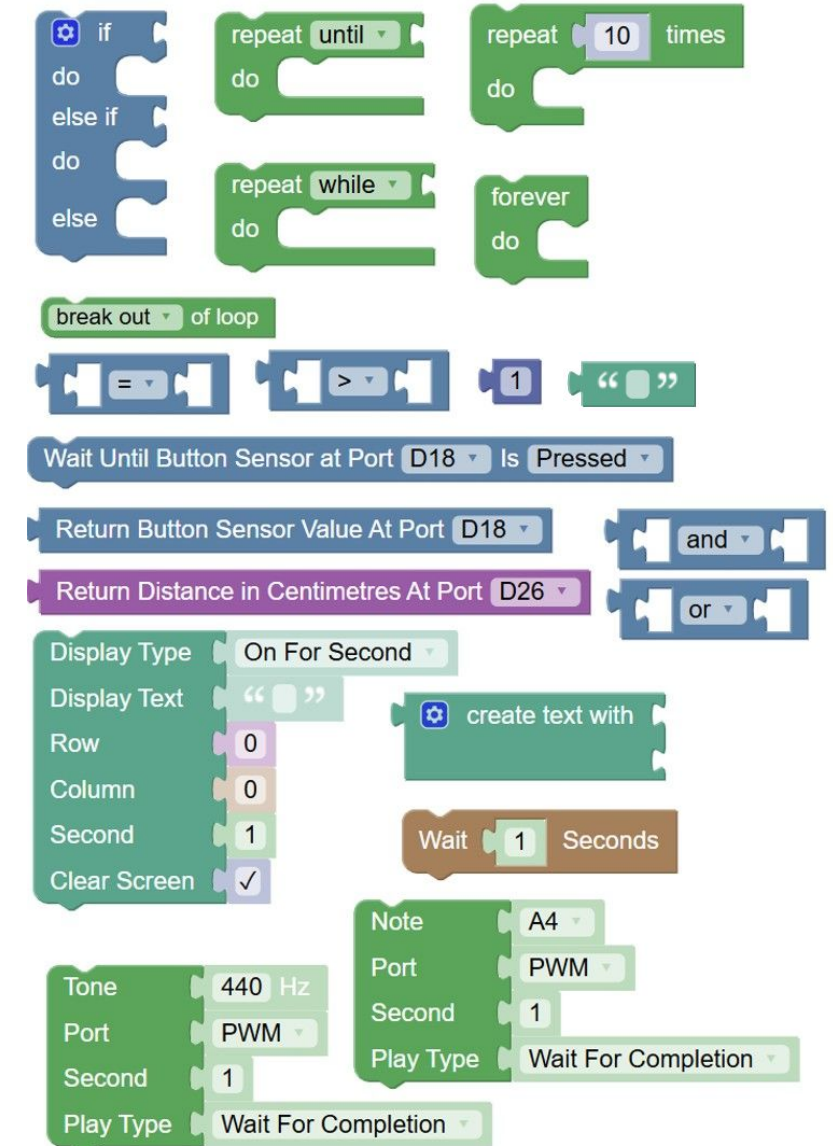
Learning

A great programmer is not one who just follow instructions and think a within a provided scope. In this unit, you will be given the chance to think out of the box and let your creativity flow. Dare to dream and execute is where miracles happen!

Hardware learnt



Blocks learnt



Exercise Q1

Imagine you are an inventor who wants to entertain your friends. Using the programming concepts that you've learnt in class, design and implement a simple game that they can play. You can save your project as a snippet and let your friends try out in class.

To create a project, you need to:

1. Understand individual **block usage**
you can revise the notes and do the exercises to build a strong foundation
2. Practice **combining and applying** them to solve problems
you need to be attentive and diligent in class to master it
3. Think of a concept and **outline the sequence** of tasks
you can get creative or use an idea you came across before
4. Implement the outline **one part at a time**
all big problems are solved in small steps

3010 - Logic & Motor

Learning

In this unit, you will explore an essential component of robotics, the **motor**. You will also apply concepts learnt from previous units to perform both manual control and autonomous navigation tasks.

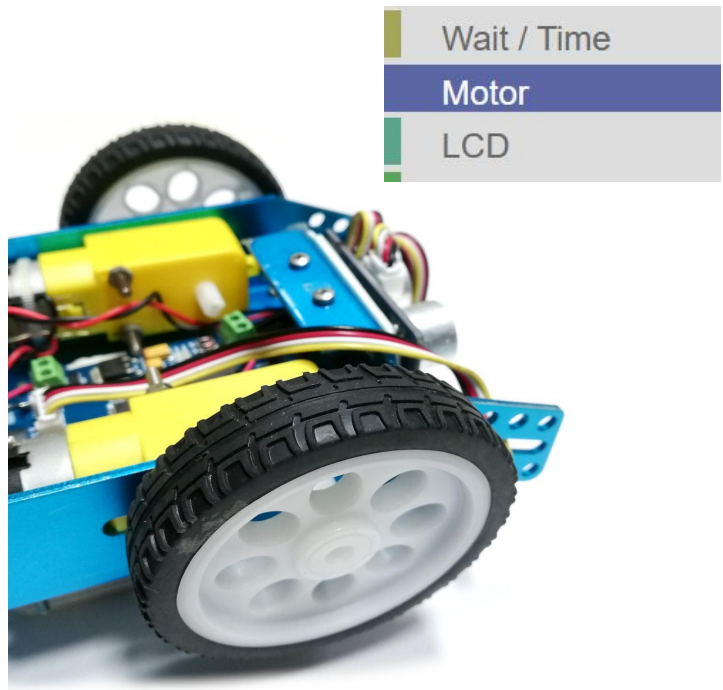
In class, we will explore the following:

- Learn about the Motor module and its sensors.
- Learn how to control directions.
- Applications including remote control and driverless cars.



Motor

For moving the robot car



Motor Type	On For Second ▾
Left Speed	75
Right Speed	75
Second	1

Move the car and wait for a duration before moving to next block

Left motor speed (-100 to 100).
Positive value moves forward.
Negative value moves backwards.

Left motor speed (-100 to 100).
Positive value moves forward.
Negative value moves backwards.

Duration of move in seconds

Motor Type	On ▾
Left Speed	75
Right Speed	75

Move the car at the indicated speeds and immediately move on to the next block.
The motors will keep turning.

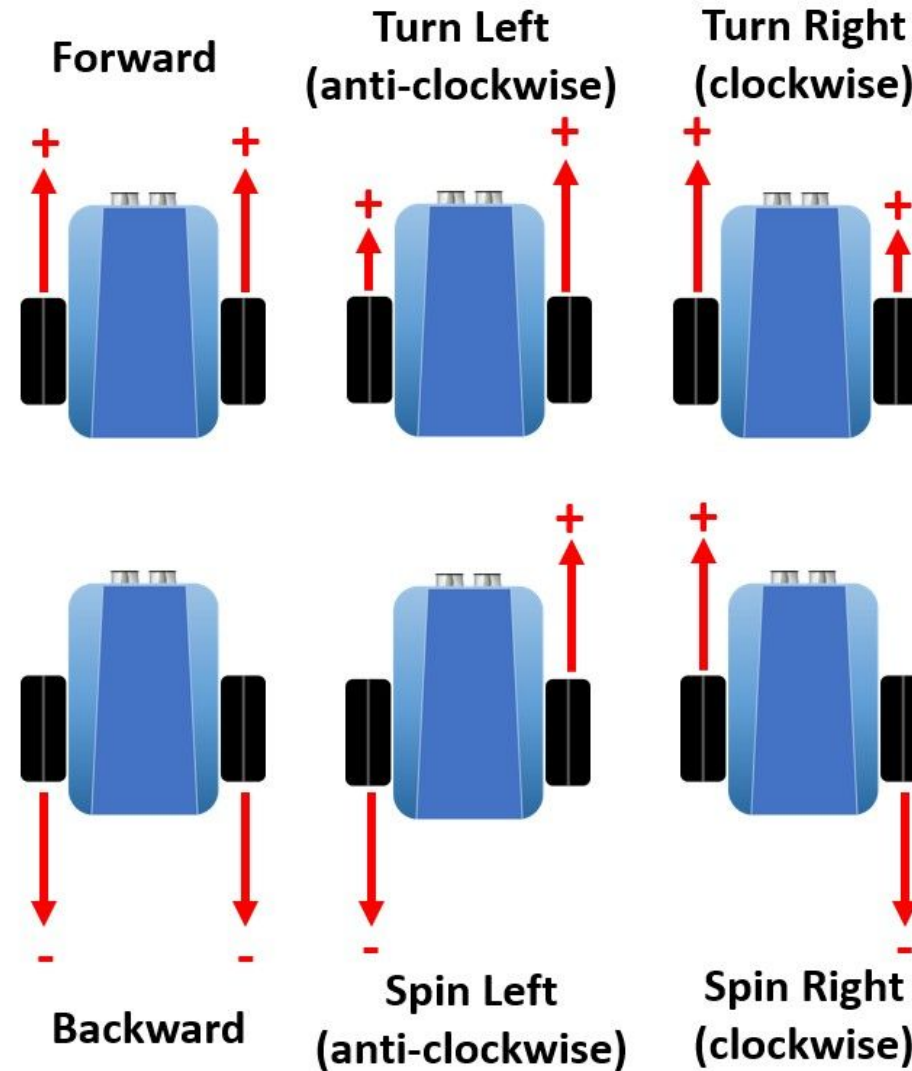
Stop Moving

Stop all motors from turning

3010 - Logic & Motor

Motor

Applying different speeds to each wheel controls the direction.



Exercise Q1

Which of the following will make the robot car turn **clockwise**?

Diagram 1

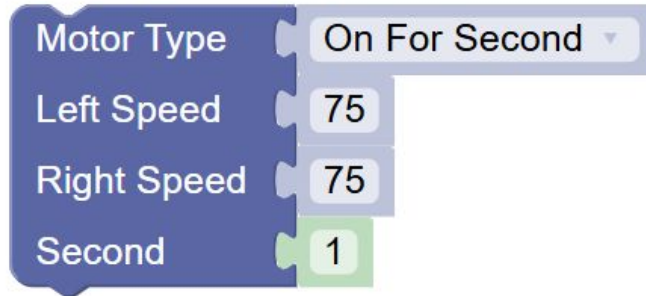


Diagram 2

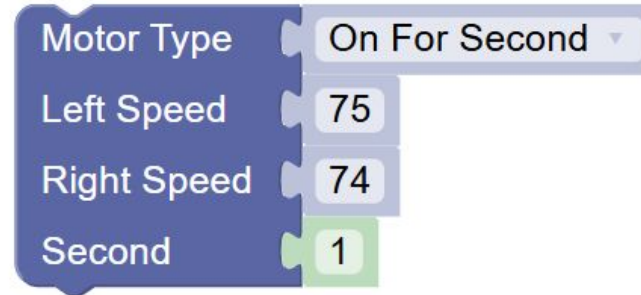


Diagram 3

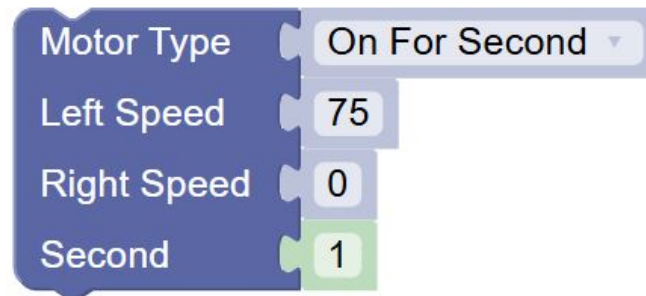
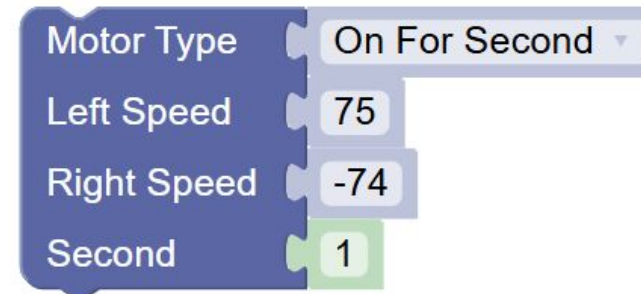


Diagram 4



- A) Diagram 1
- B) Diagram 3
- C) Diagrams 2 and 3
- D) Diagrams 2, 3 and 4.

Exercise Q2

In which of the following code snippets will the robot start and end at the same point (assuming the turning block will make a 180 degree turn)?

Diagram 1



Diagram 2

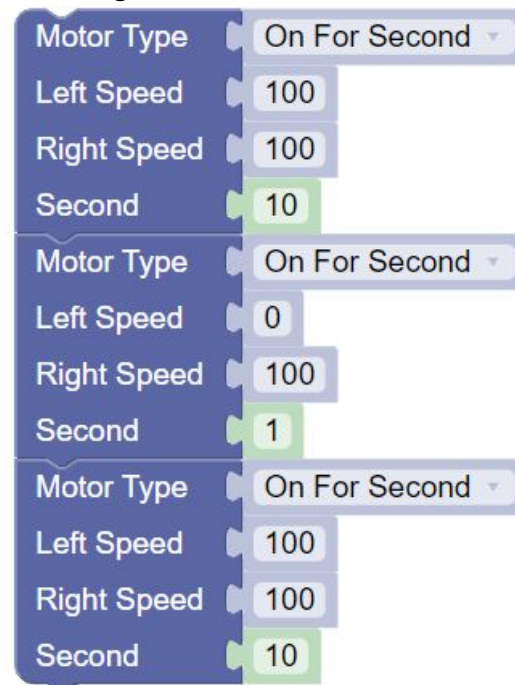
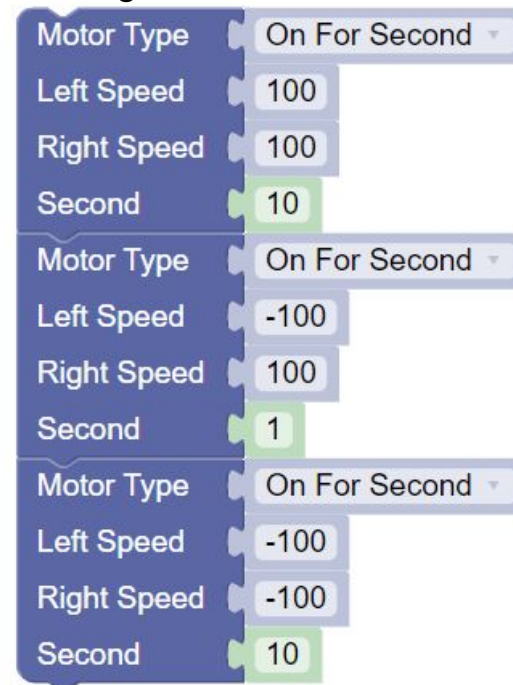


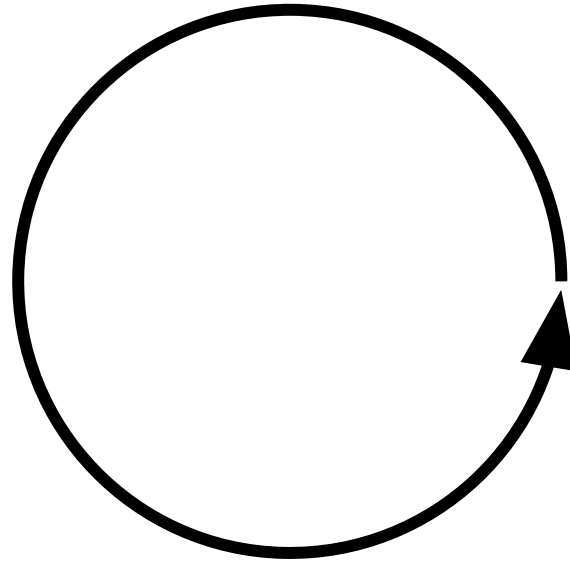
Diagram 3



- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagram 3 only
- D) Diagrams 1 and 2 only

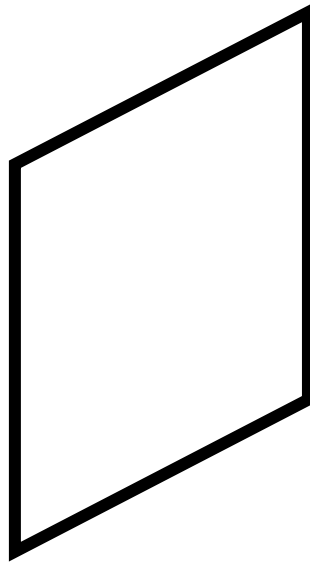
Exercise Q3

Create a code to make the robot repeatedly trace a circular path in the anti-clockwise direction.



Exercise Q4

Using loops, create a code to follow a path shown below. You can put cones to estimate the turning points.



Answers

Q1: D

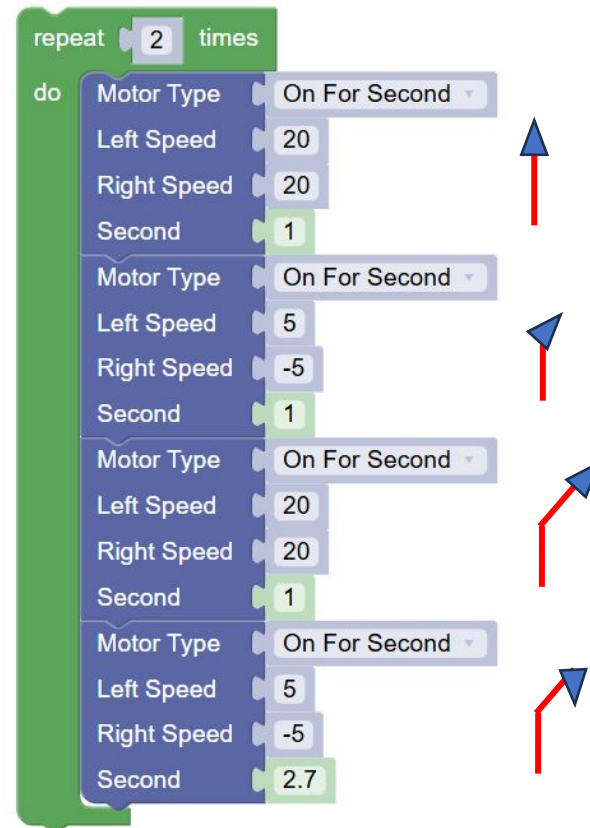
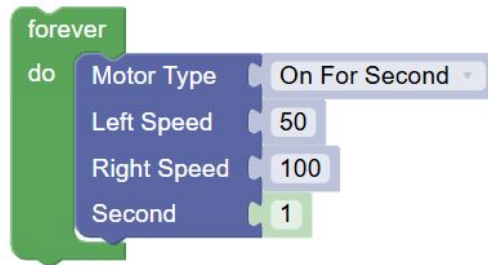
As long as the left speed is more than the right speed, it will turn clockwise (right).

Q2: A

The turning in diagram 2 is a U-turn so it will end up by the side of the start point.

In diagram 3, after turning back, it reverses away farther from the start point.

Q3:



3011 - Logic & Colour Sensor

Learning

In this unit, you will learn about the **Colour sensor** and apply logic blocks with various sensors.

You learnt that flowchart can be used to design a program before implementing. But you need to be proficient with drawing complex flowcharts to do that. A simpler way to start planning your program is using the **Comment blocks** to breakdown and plan your code sequence.

In class, we will explore the following:

- Learn about Colour sensor and its blocks.
- Learn how to use Comment blocks to plan code sequence.
- Implement you program in parts with the guide of the Comment blocks.
- Applications including coding a Laundry helper and a Robot server.



3011 - Logic & Colour Sensor

Colour Sensor

For detecting colour



Return Color Name

Send the name of the colour being sensed to the robot

Return Reflected Light Intensity Value

Send the reflected light intensity value to the robot (not required in curriculum)

Return Ambient Light Intensity Value

Send the ambient (surrounding) light intensity value to the robot (not required in curriculum)

Wait Until Color Sensor Detect Red

Wait until the sensor detects the indicated colour

“ Red ”

Colour name: Yellow, Orange, Red, White, Green, Purple, Blue, Black
But only White, Red, Orange, Green and Blue is used.

Return Color Name = “ Red ”

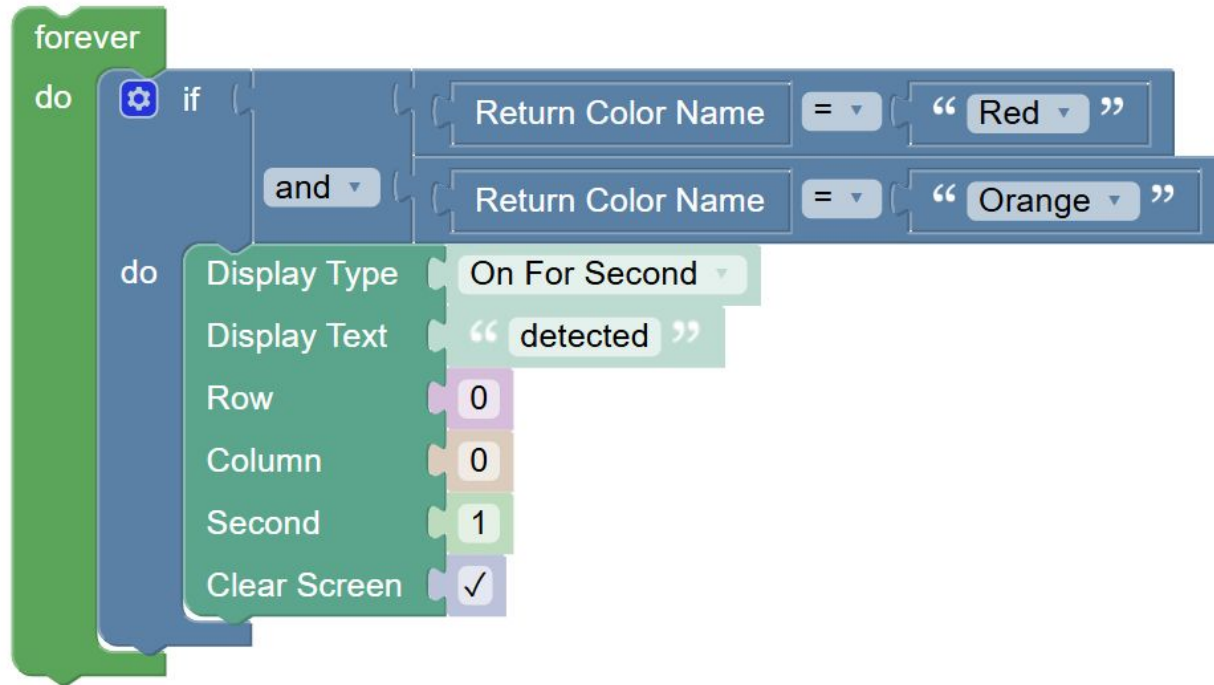
Condition to check that the colour.

Return Color Name = “ Red ”

You can also use text box in place of the colour block if the colour you wanted is not found in the colour block. Use first letter capitalization in your text.

Exercise Q1

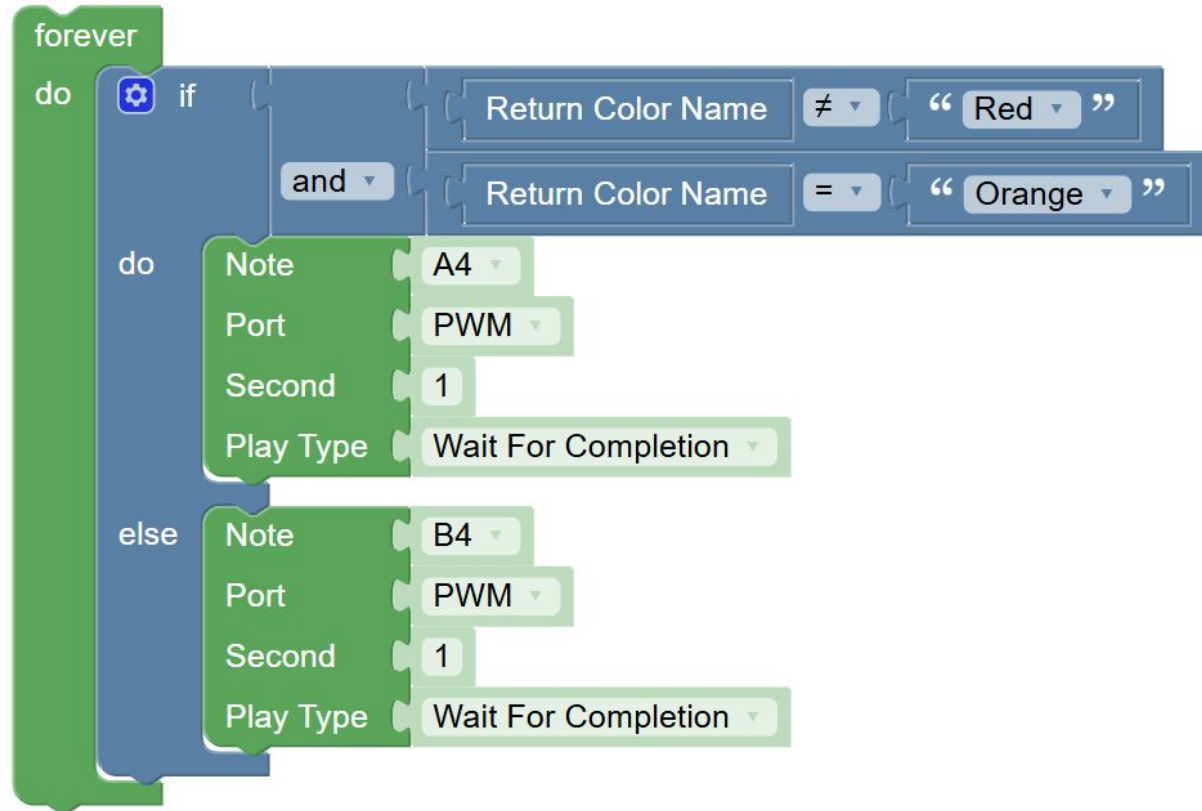
Which condition will cause 'detected' to be shown when the following code is run?



- A) Only when colour sensor detects red.
- B) Only when colour sensor detects orange.
- C) When colour sensor detects both red and orange.
- D) None of the above.

Exercise Q2

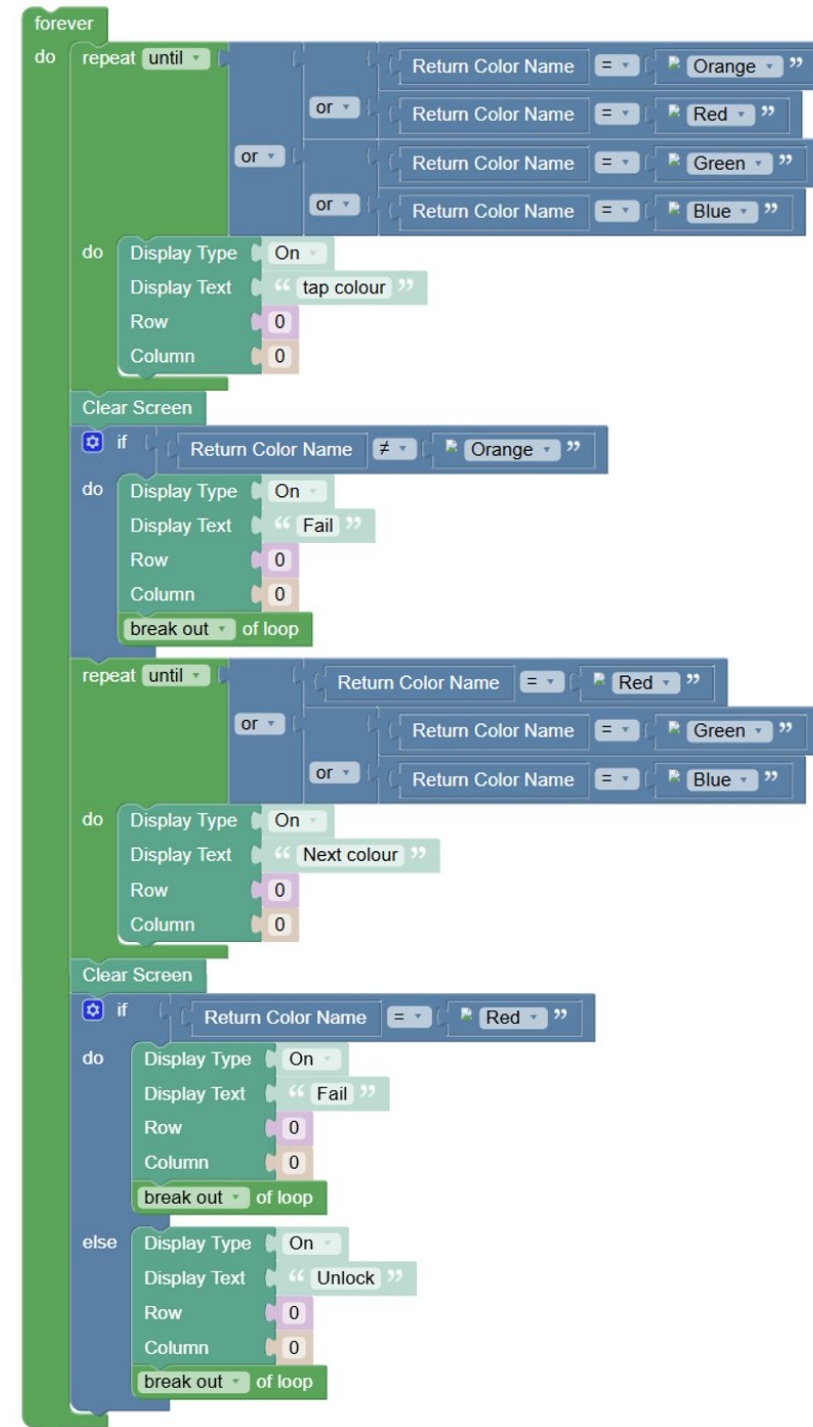
Which colour(s) will cause the code to play note A4?



- A) Orange, Green, Blue or White
- B) Red, Green, Blue or White
- C) Orange only
- D) None

Exercise Q3

Study the code without running it. Write down the colour sequence to “unlock”.



Exercise Q4

Create a code that performs the following:

Colour detected	Display Message
Either Red or Orange	"Warm colour"
Either Green or Blue	"Cool colour"
Otherwise	<i>Nothing is displayed</i>

Answers

Q1: D

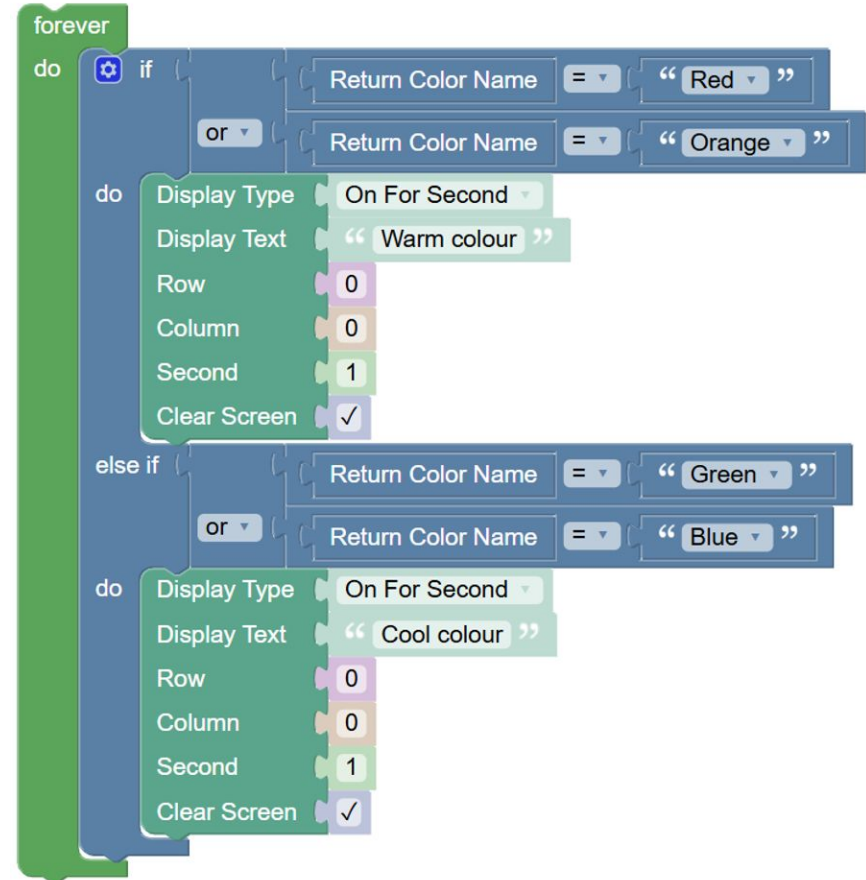
Colour sensor can only detect one colour so logically, the condition can never be true.

Q2: C

Colour sensor can only detect one colour at a time. When it detects orange, which is also not red, note A4 will be played.

Q3: Orange, Green
or Orange, Blue

Q4:



3012 - Nested If Statements

Learning

In this unit, you will learn how **Nested If statements** can be used to create layered logic. Nested If statements allow you to check one condition first, and only if it is true, proceed to evaluate another. This can help make your code more organized and manageable in complex situations.

In class, we will explore the following:

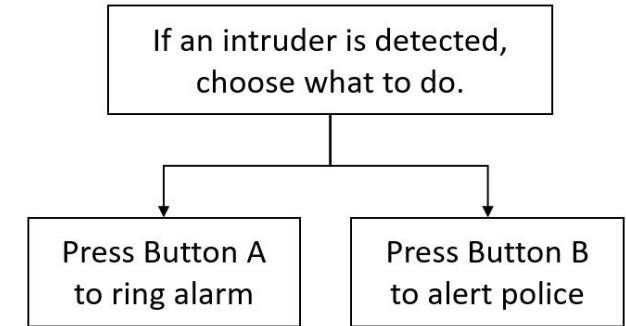
- Practice nested if statements.
- Learn about the LTA School zone initiative for road safety and simulate it with your robot.



3012 - Nested If Statements

Nested If

Combining multiple AND/OR operators can make code hard to debug. Using nested If statements can be a good alternative. Below shows an example how the same program can be written in different ways using nested If.



Multiple AND operators

```

# CODE A
# Check both button press and range
if
  Return Button Sensor Value At Port D22 = 1
  and
  Return Distance in Centimetres At Port D26 > 10
  and
  Return Distance in Centimetres At Port D26 < 20
do
  Note A4
  Port PWM
  Second 1
  Play Type Wait For Completion
else if
  Return Button Sensor Value At Port D24 = 1
  and
  Return Distance in Centimetres At Port D26 > 10
  and
  Return Distance in Centimetres At Port D26 < 20
do
  Display Type On
  Display Text "warning"
  Row 0
  Column 0
  
```

Nested If
(check which button first then check range)

```

# CODE B
# Check button press
if
  Return Button Sensor Value At Port D22 = 1
do
  # Check range
  if
    Return Distance in Centimetres At Port D26 > 10
    and
    Return Distance in Centimetres At Port D26 < 20
  do
    Note A4
    Port PWM
    Second 1
    Play Type Wait For Completion
  else if
    Return Button Sensor Value At Port D24 = 1
  do
    # Check range
    if
      Return Distance in Centimetres At Port D26 > 10
      and
      Return Distance in Centimetres At Port D26 < 20
    do
      Display Type On
      Display Text "warning"
      Row 0
      Column 0
    
```

Nested If
(check range first then check button press)

```

# CODE C
# Check range
if
  Return Distance in Centimetres At Port D26 > 10
  and
  Return Distance in Centimetres At Port D26 < 20
do
  # Check button press
  if
    Return Button Sensor Value At Port D22 = 1
  do
    Note A4
    Port PWM
    Second 1
    Play Type Wait For Completion
  else if
    Return Button Sensor Value At Port D24 = 1
  do
    Display Type On
    Display Text "warning"
    Row 0
    Column 0
  
```

3012 - Nested If Statements

Nested If statements are also layered logic structure, that is, they make decisions at different levels.

The program first check if there is an obstacle ahead. If yes, it will let user decide whether to blast it off.



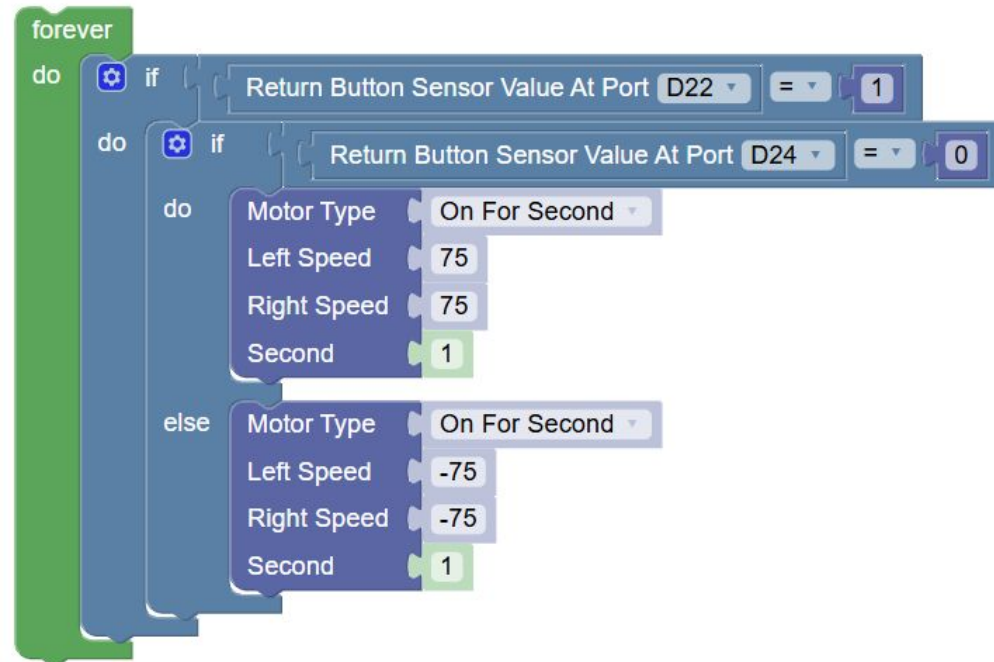
— (First layer logic)
Check if an obstacle is within 5 to 90cm ahead

— (Second layer logic, nested inside an if statement)
If obstacle is in range,
then check if Button at D22 is pressed to fire.

— (First layer logic)
Check if an obstacle is less than 5cm ahead

Exercise Q1

Which button press combination will cause the robot to move backwards?



- A) D22-pressed, D24-pressed
- B) D22-pressed, D24-released
- C) D22-released, D24-pressed
- D) D22-released, D24-released

Exercise Q2

Which of the following code snippets will perform the following:

- When **only one** button is pressed, Note A4 will be played.
- Otherwise, Note C4 will be played.

Diagram 1

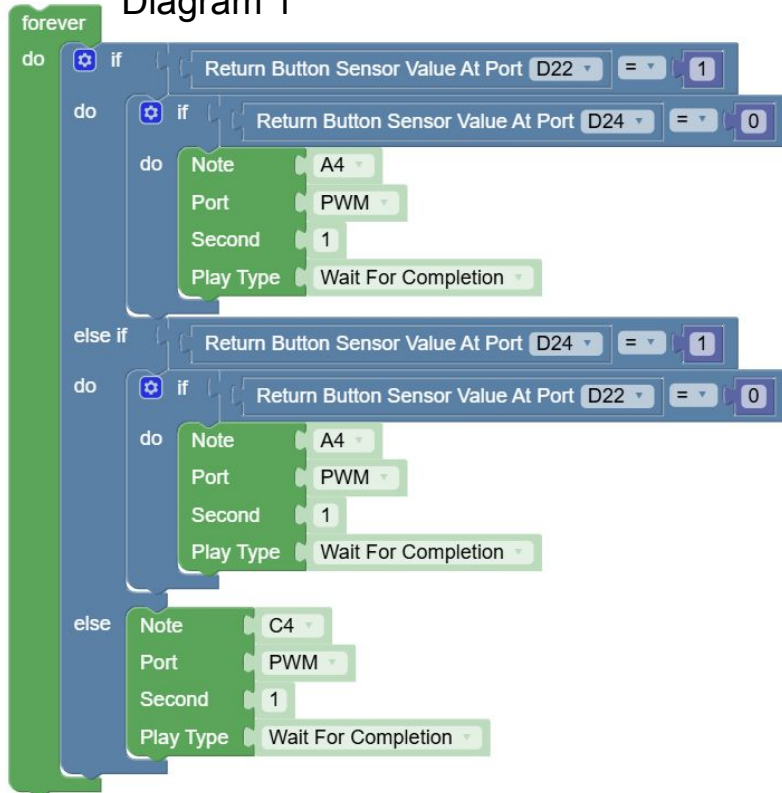
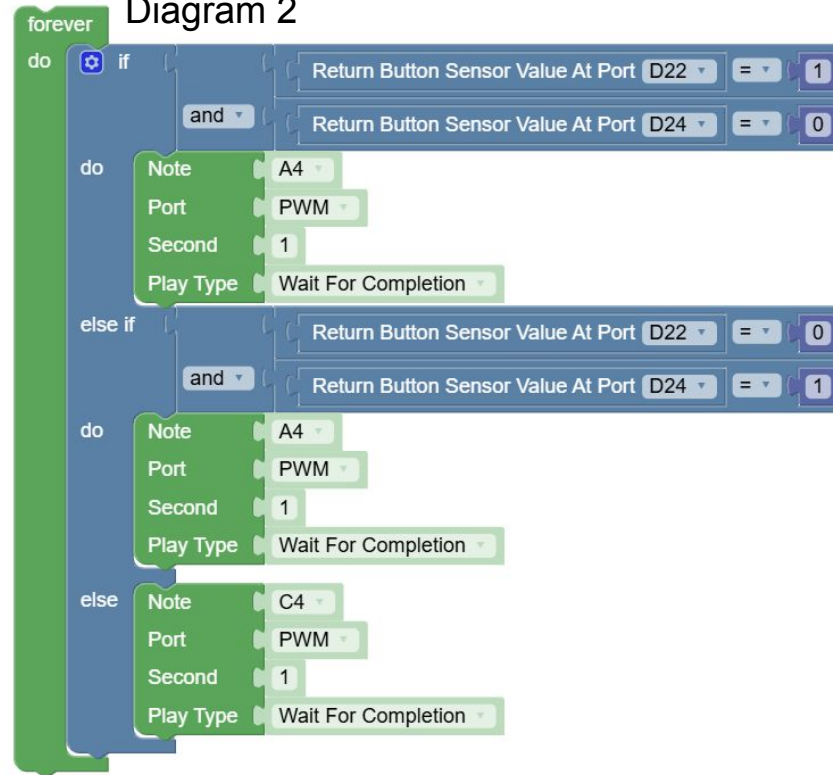


Diagram 2

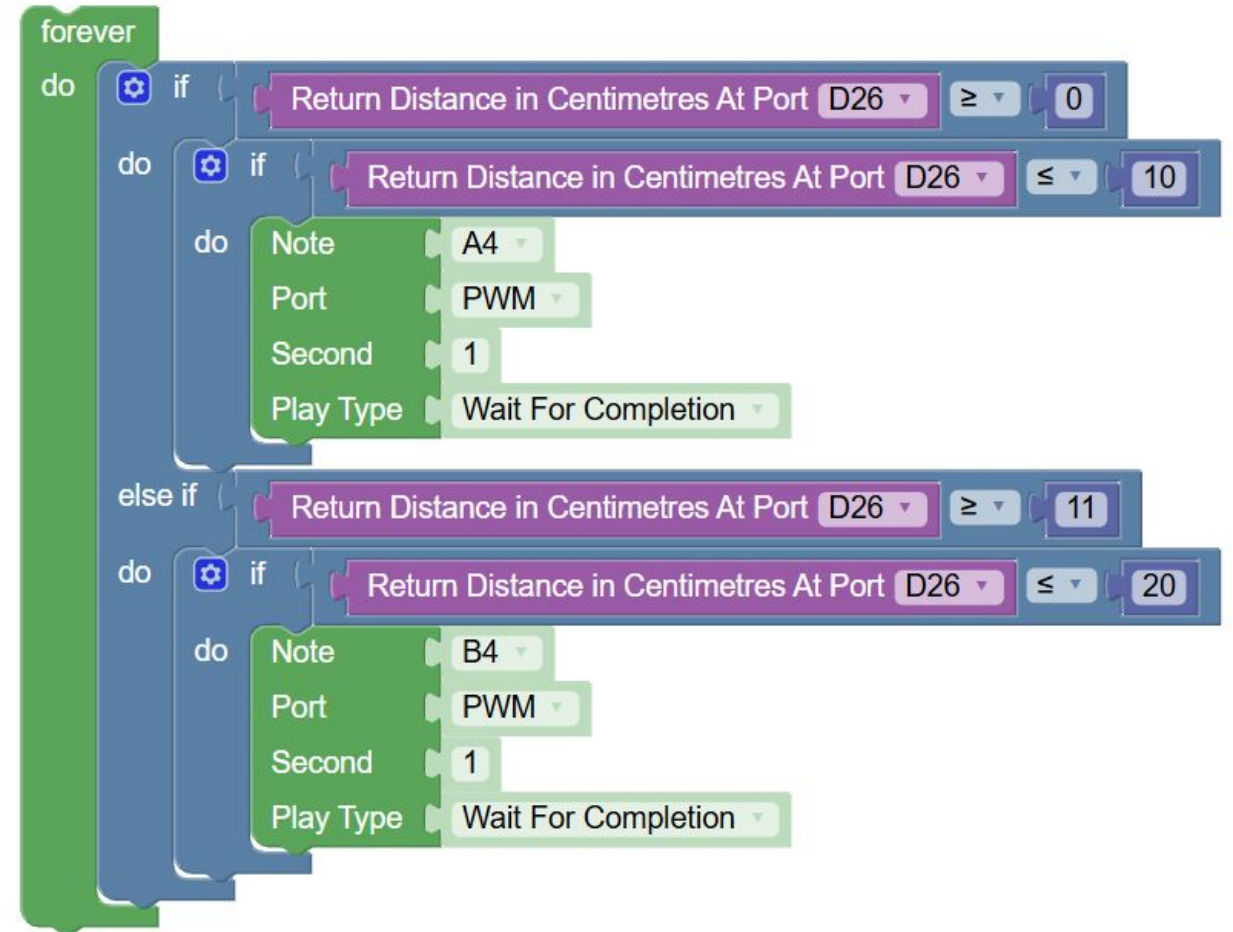


- A) Diagram 1 only
- B) Diagram 2 only
- C) Diagrams 1 & 2 only
- D) None of them

Exercise Q3

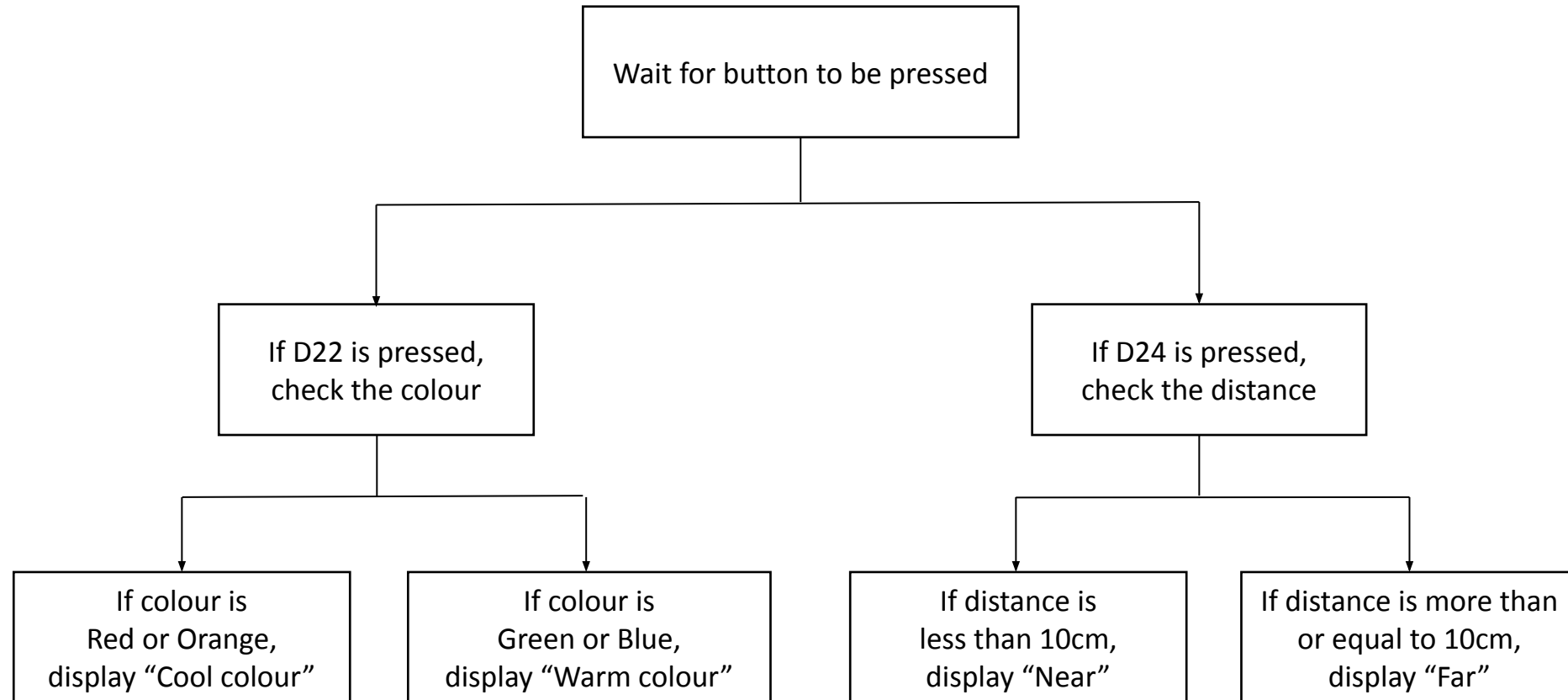
When the obstacle distance is **constantly at 15cm**, which note will be played when the code is run?

- A) A4
- B) B4
- C) A4 followed by B4 repeatedly
- D) None



Exercise Q4

Using nested If blocks, create a program to follow the logic control as shown below.



Answers

Q1: A

When D22 is pressed, it will check the inner If block, whose condition is $D24=0$. So if D24 is pressed, the else will be run, causing the motor to reverse.

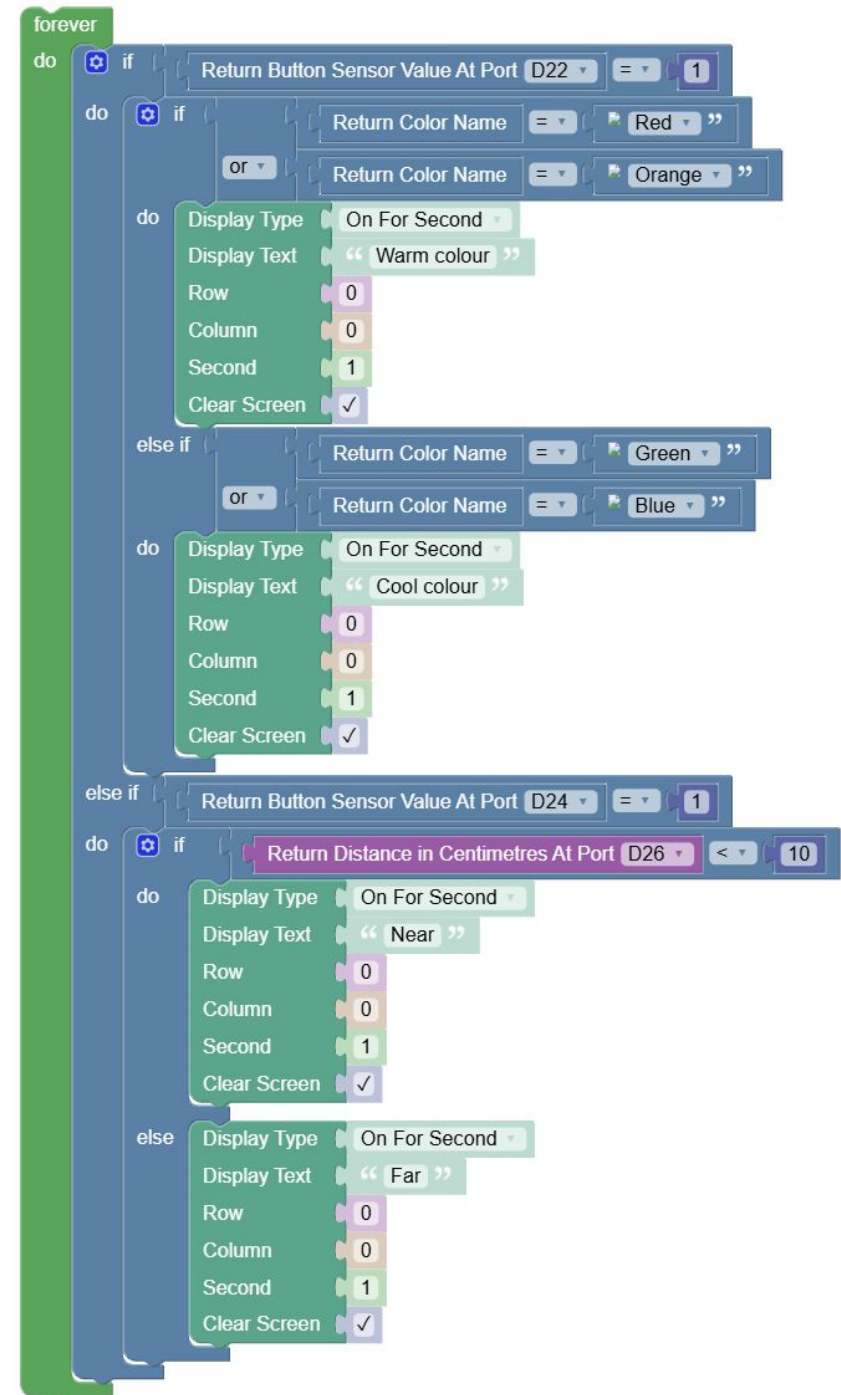
Q2: B

In diagram 1, when both buttons are pressed, the first condition " $D22=1$ " will be true and it will proceed to check the inner if condition " $D24=0$ ". It is not true since both buttons are pressed and nothing will happen. The loop will restart from the top.

Q3: D

When D26 is 15cm, $D26 \geq 0$ will be true and the program will proceed to check the inner condition $D26 \leq 10$. This will be false and nothing will happen (it will skip checking $D26 \geq 11$).

Q4:



3013 - 3015 More Applications

Learning

As you advance to code longer and more complex programs, you will need learn how to breakdown your programs logically. You have tried using comment blocks as a simple method to plan the flow of a program. A program won't normally be a direct sequence of events. It includes looping and branching. The exercises in these units will show you how programs can be represented as flow diagrams. They are not as detailed as flowcharts, which requires more proficiency to use, but a good start for you to plan more complex codes.

You will also learn about code optimisation and user-friendliness, as well as, how to add randomness to your program.

In class, we will explore the following:

- Study examples of programs broken down into parts and represented as flow diagrams. Practice implement and check each part progressively.
- Learn about code optimisation and user-friendliness.
- Learn about different ways to use random number generator.
- Introduction to two emerging fields: Equity in Engineering and Neuroinclusive Design. Through hands-on application development, you'll deepen your understanding of how inclusive thinking can shape innovative, human-centered solutions.
- Fun applications implementation to inspire and prepare you for project development.

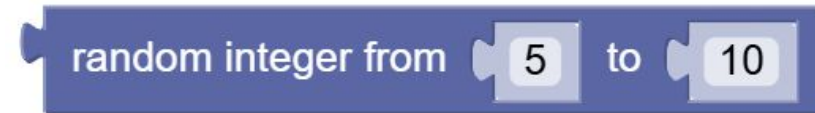


3012 - 3015 More Applications

Generate Random Number

The ***random block*** can generate a random number within a range. Each time the block is used, a new random number is generated (the number may repeat).

In the example below, a random number from 5 to 10 is generated. That is, the random number can be 5, 6, 7, 8, 9, 10.

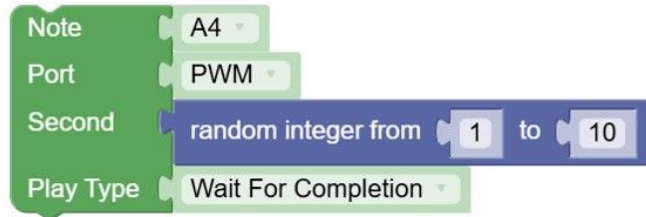


3012 - 3015 More Applications

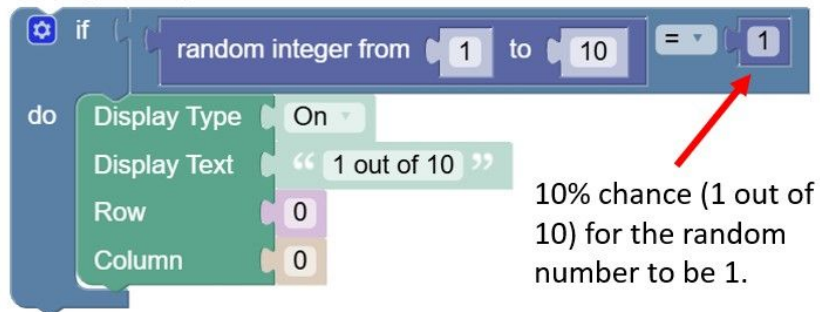
Generate Random Number

Different ways of using random block

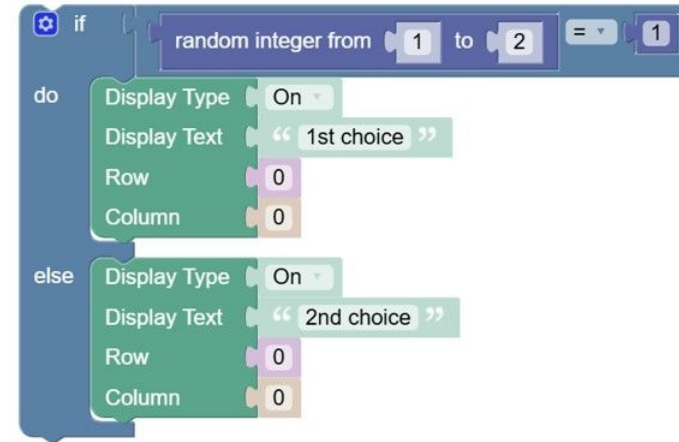
Using it as values



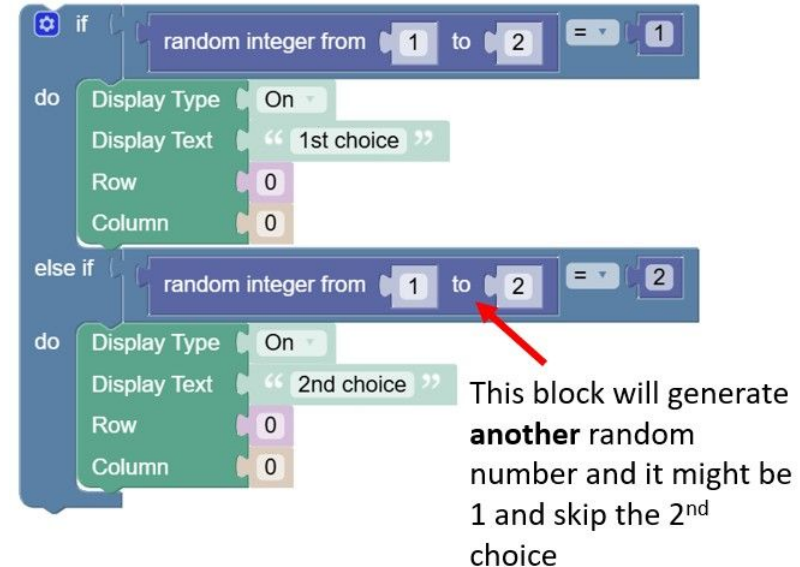
Using it to generate chance



Using it to make choices

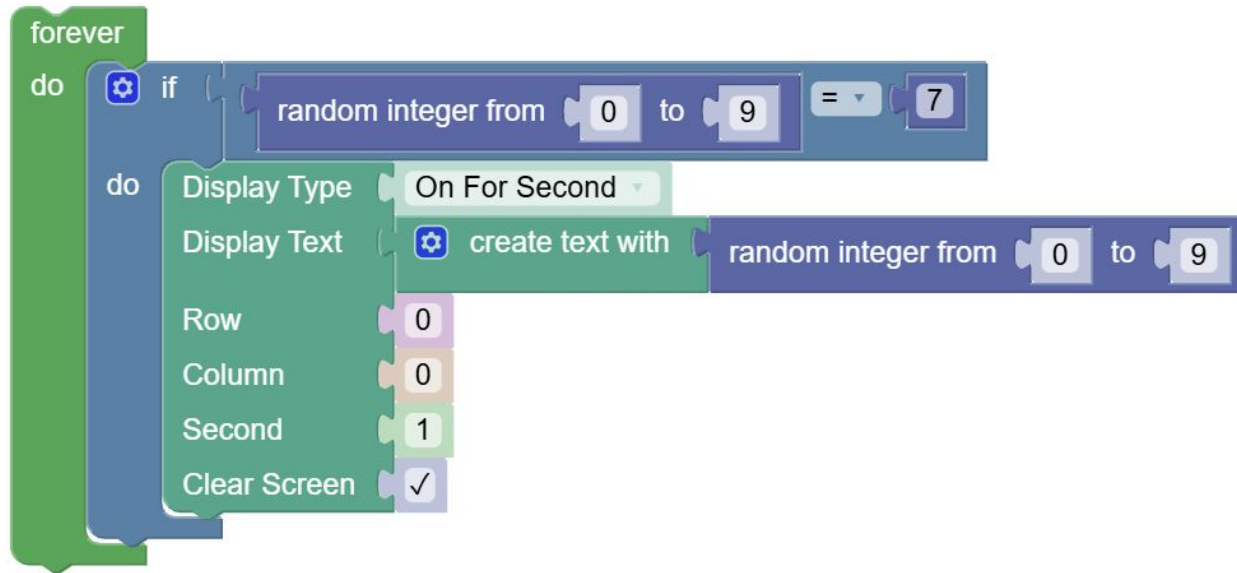


Warning! Below is **INCORRECT**.



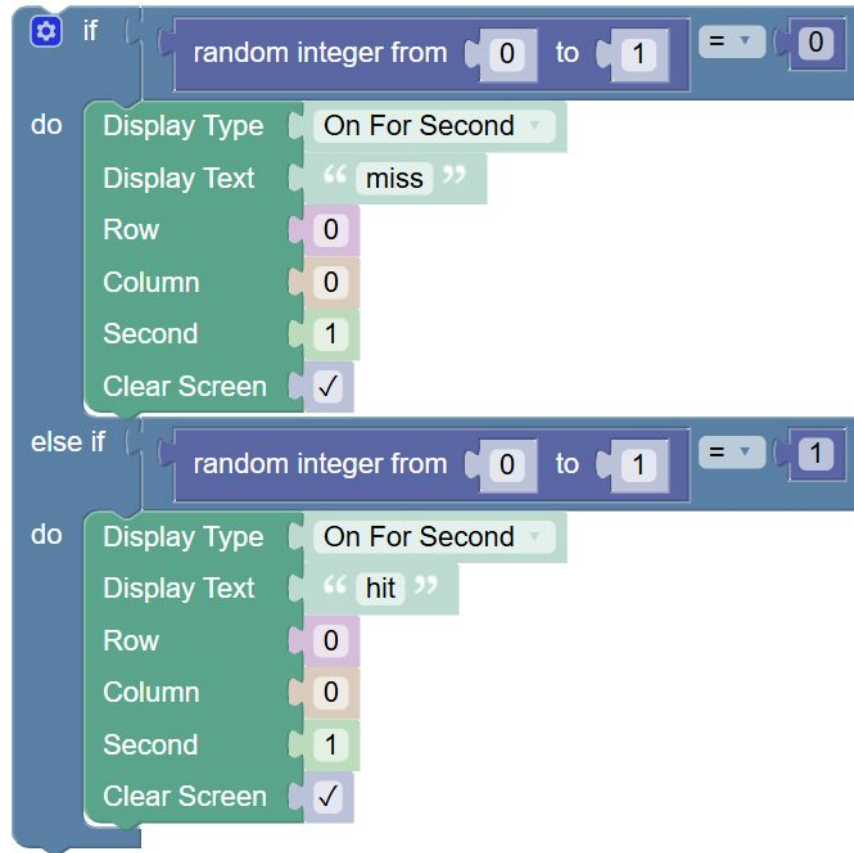
Exercise Q1

Write down all the possible number(s) that may appear on the LCD.



Exercise Q3

What will be shown when the code below is run?



- A) “miss” only
- B) “hit” only
- C) “miss” or “hit” only
- D) “miss” or “hit” or nothing

Exercise Q3

Which of the following structure can be used to generate a random option of 1 to 3 (need not be even chances).

Diagram 1

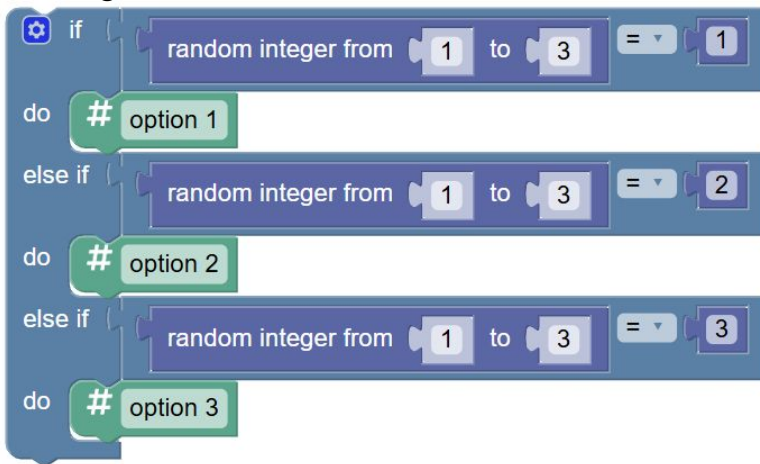


Diagram 2

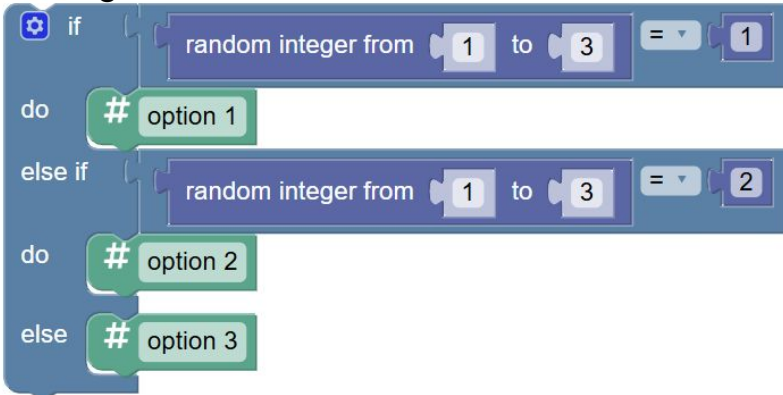
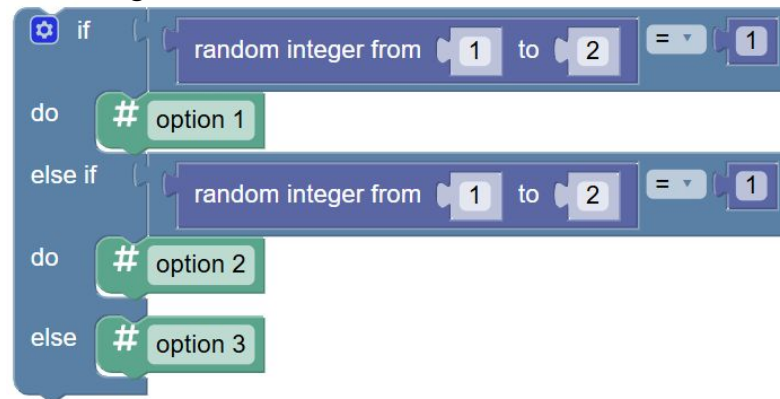


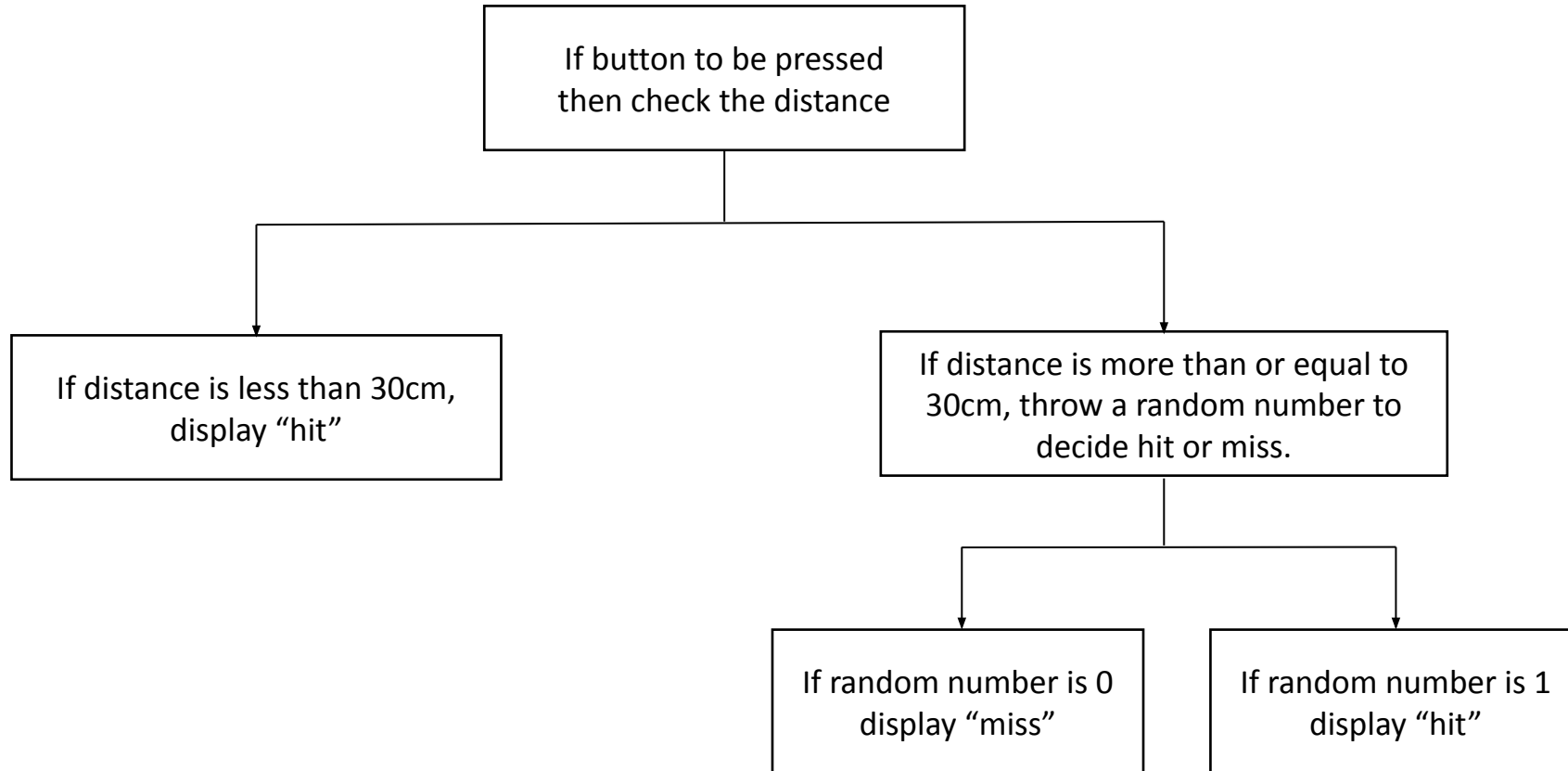
Diagram 3



- A) Diagram 1 only.
- B) Diagram 2 only.
- C) Diagrams 1 and 2 only.
- D) Diagrams 2 and 3 only.

Exercise Q4

Code a program to simulate a paint-ball game with random hits using the logic control below.



Answers

Q1: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

When number 7 is generated in the If condition, it will run the LCD block, which uses another Random block to generate digits 0 to 9. Hence, any number from 0 to 9 may appear.

Q2: D

If the first random block generates 1 and the second random block generates 0, then nothing will be shown.

Q3: D

Diagram 1 has a possibility on not executing any of the options.

Q4:



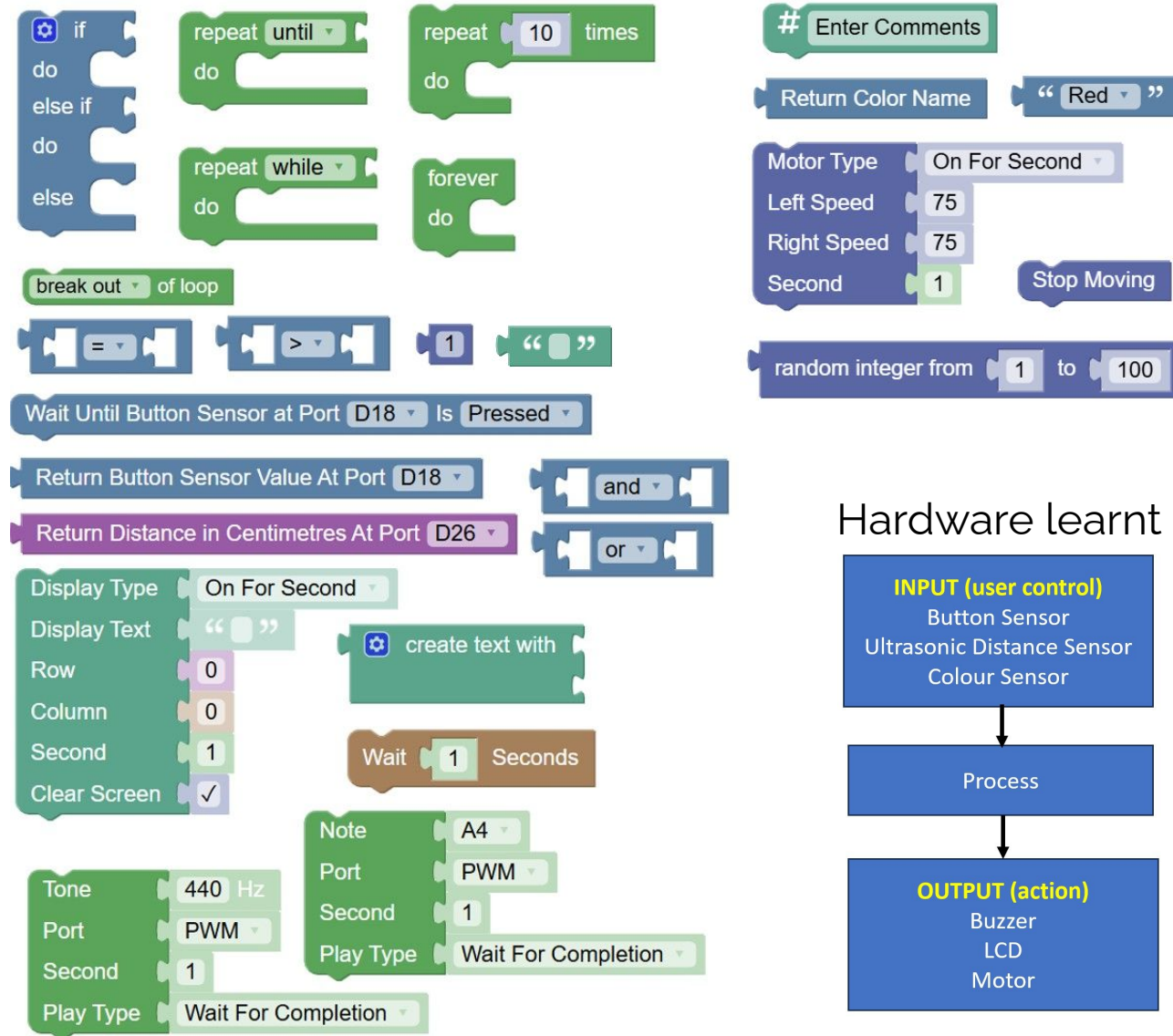
3016 - Basic 2 Project

Learning

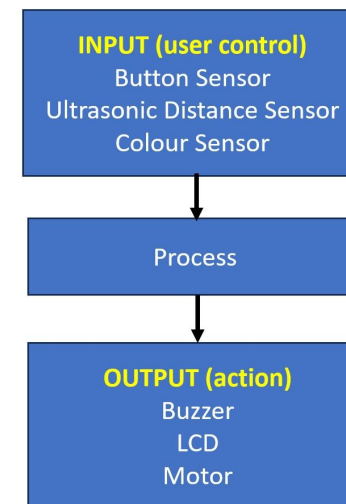
Increase in knowledge base gives you more power to create greater things.

Every new idea starts with a spark. Don't hold back, share your wildest, smartest, silliest thoughts. Creativity isn't about being perfect; it's about being brave enough to try.

Blocks learnt



Hardware learnt



Exercise Q1

Now that you have learnt all the Basic module concepts, use them to design and implement another project. You can save your project as a snippet and let your friends try it out in class.

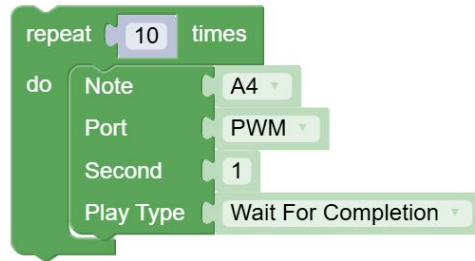
To create a project, you need to:

1. Understand individual **block usage**
you can revise the notes and do the exercises to build a strong foundation
2. Practice **combining and applying** them to solve problems
you need to be attentive and diligent in class to master it
3. Think of a concept and **outline the sequence** of tasks
you can get creative or use an idea you came across before
4. Implement the outline **one part at a time**
all big problems are solved in small steps

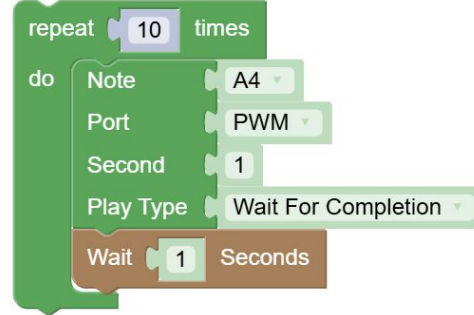
Useful Structures

The chapter lists down code sequences that students often make mistakes with.

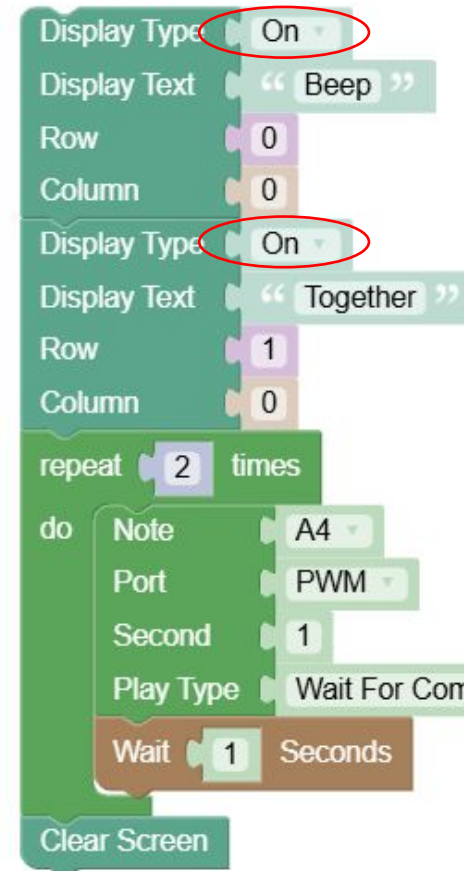
Continuous sound



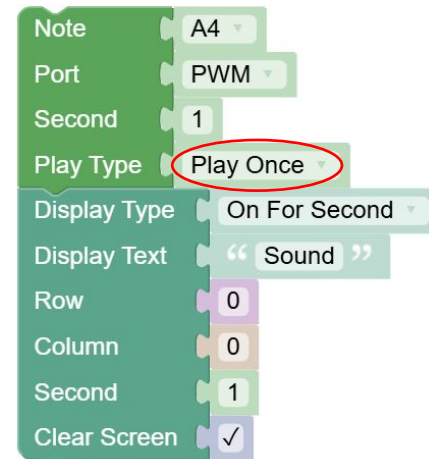
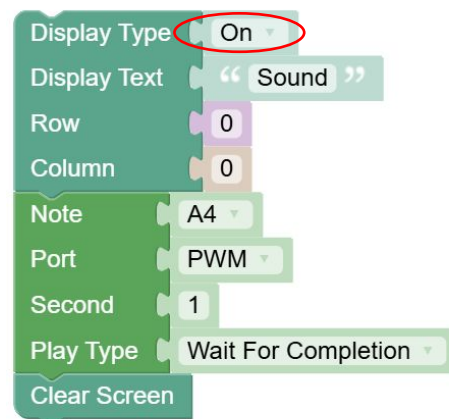
Beeping sound



Multiple displays and sounds together



Display and sound together

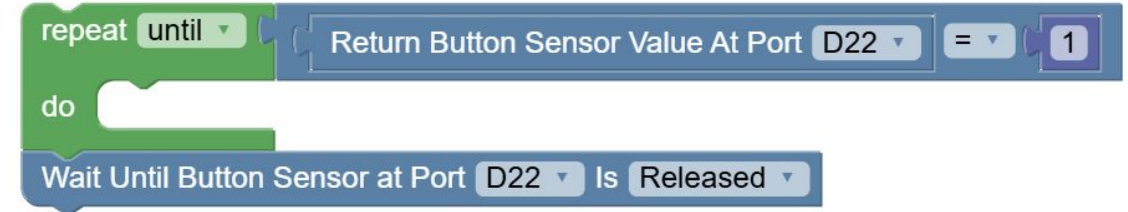


Useful Structures

Press and release



Press and release



Code here runs after press and release

Prioritise overlapping conditions



END